



CPE 334

SOFTWARE ENGINEERING · WEEK 9

Git workflows, containers and CI/CD

INSTRUCTOR

Dr. Santawat

DEPARTMENT

Computer Engineering, KMUTT

 PART 01

Branching workflows



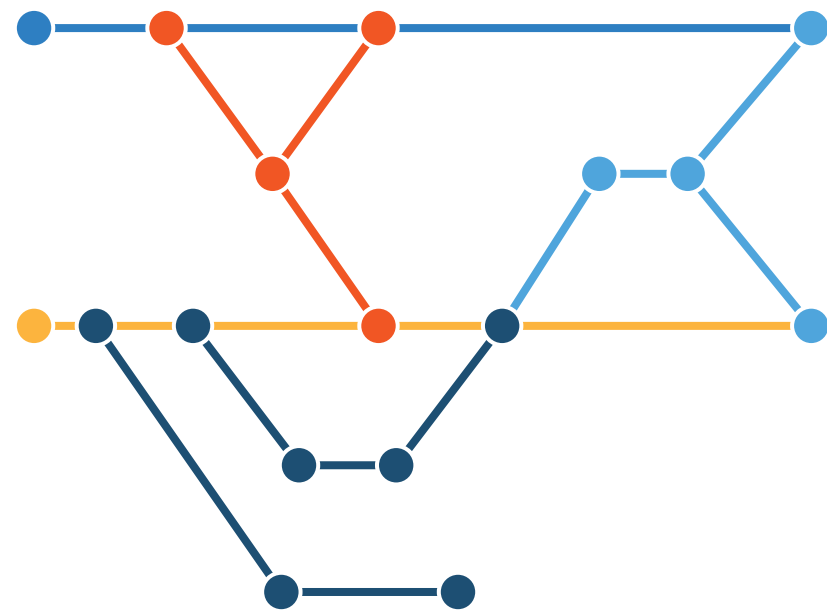
A Written Convention

Agree on a Branching Workflow in Your Team

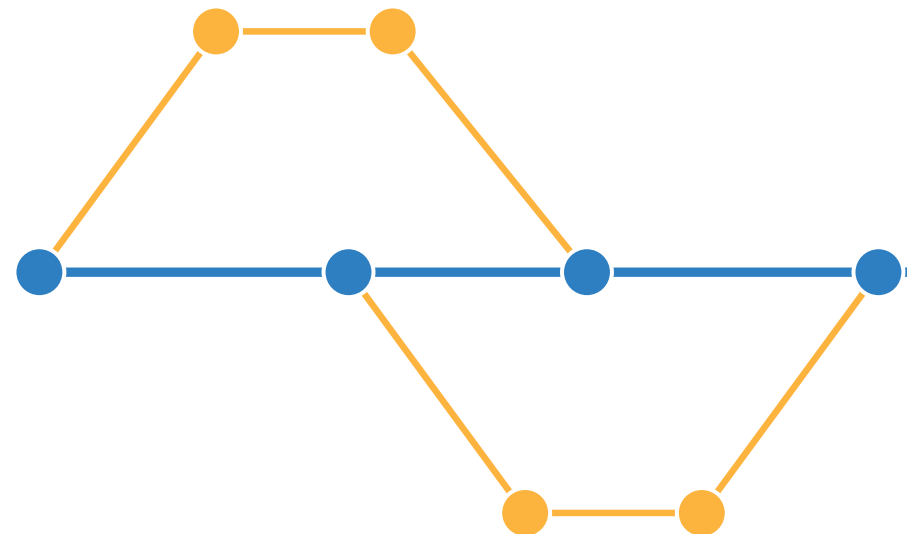
- 1 Git allows you to *create branches* - but it doesn't tell you *how to use* them!
- 2 You need a written best practice of how work is ideally structured in your team - to avoid mistakes & collisions.
- 3 It highly depends on your team / team size, on your project, and how you handle releases.
- 4 It helps to onboard new team members ("this is how we work here").

Different workflow

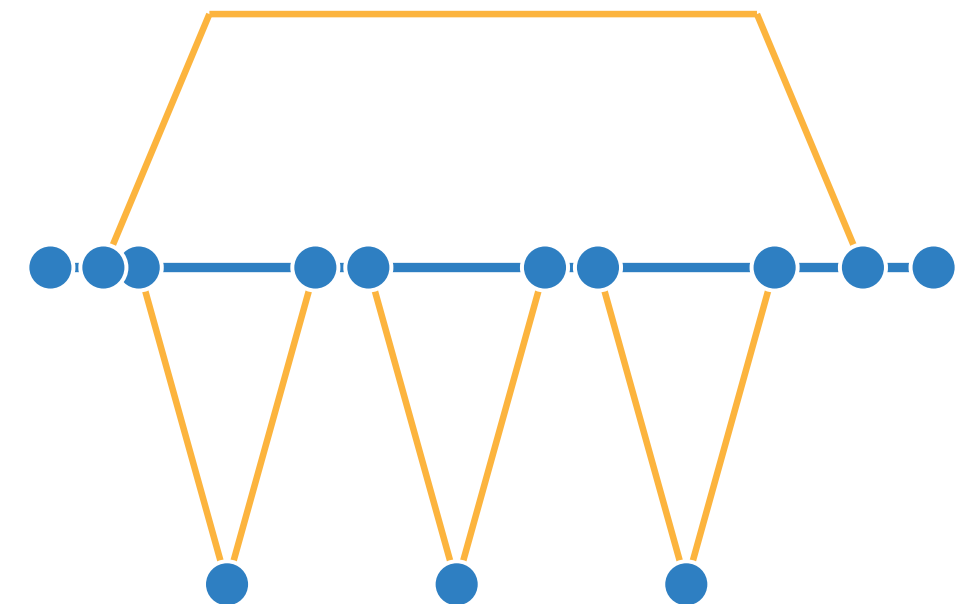
GitFlow



GitHub Flow



Trunk-Based Development



How GitFlow Works



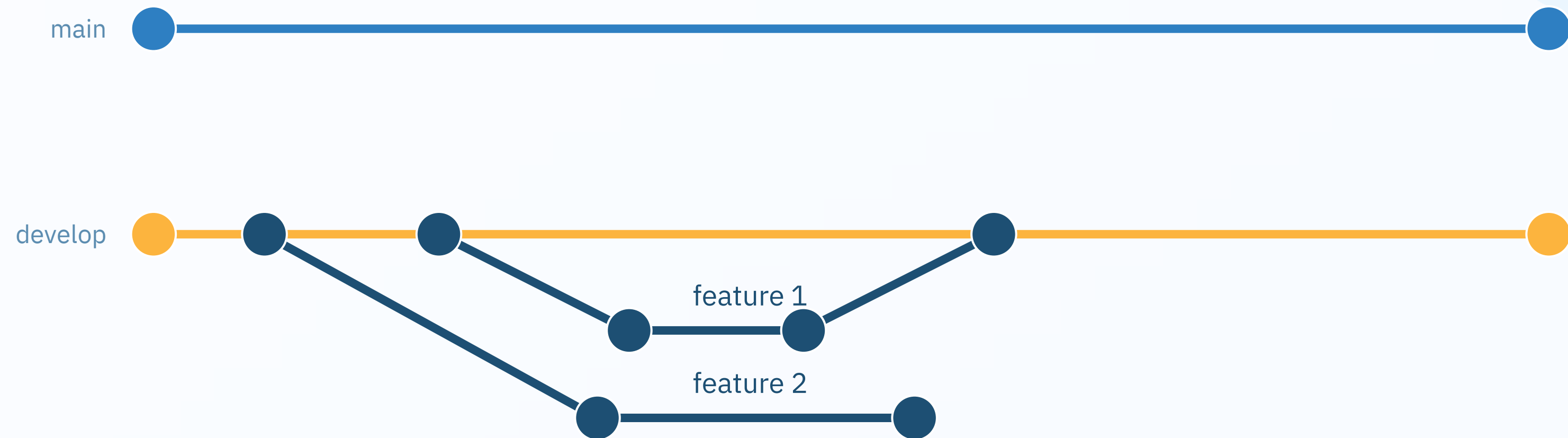
— BRANCHING WORKFLOWS

How GitFlow Works



— BRANCHING WORKFLOWS

How GitFlow Works

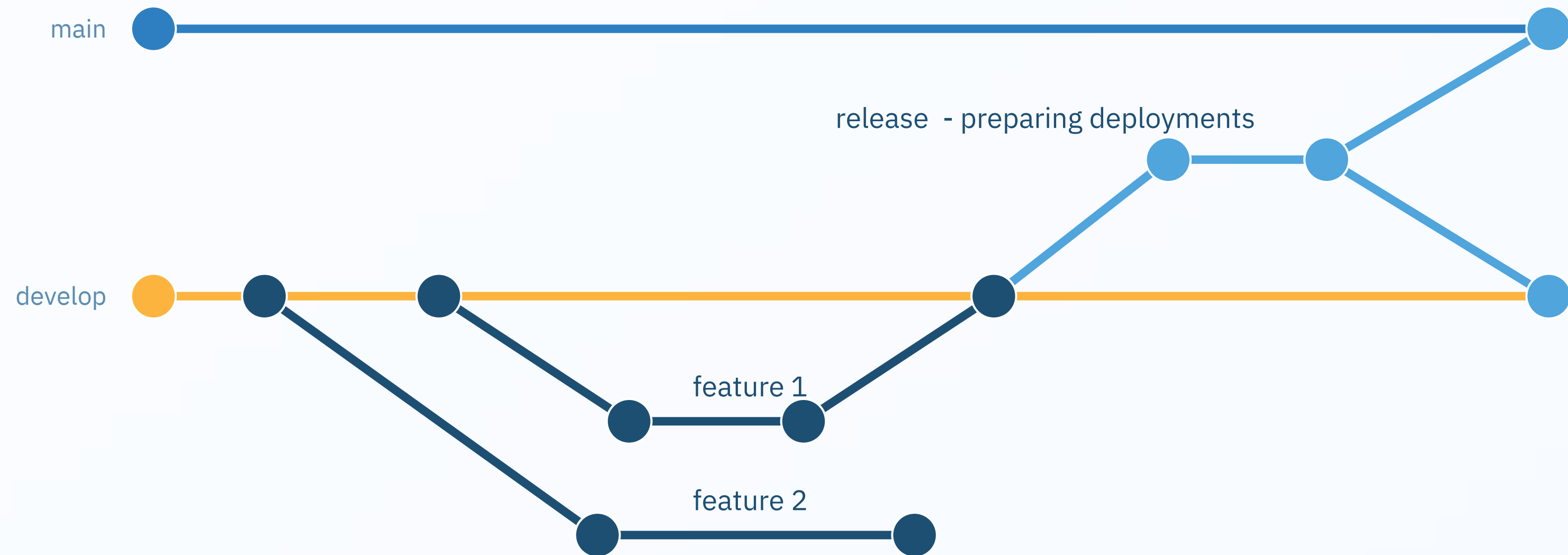


Feature Branch

Used to build a new feature in isolation, then merged back into develop when complete—never directly into main.

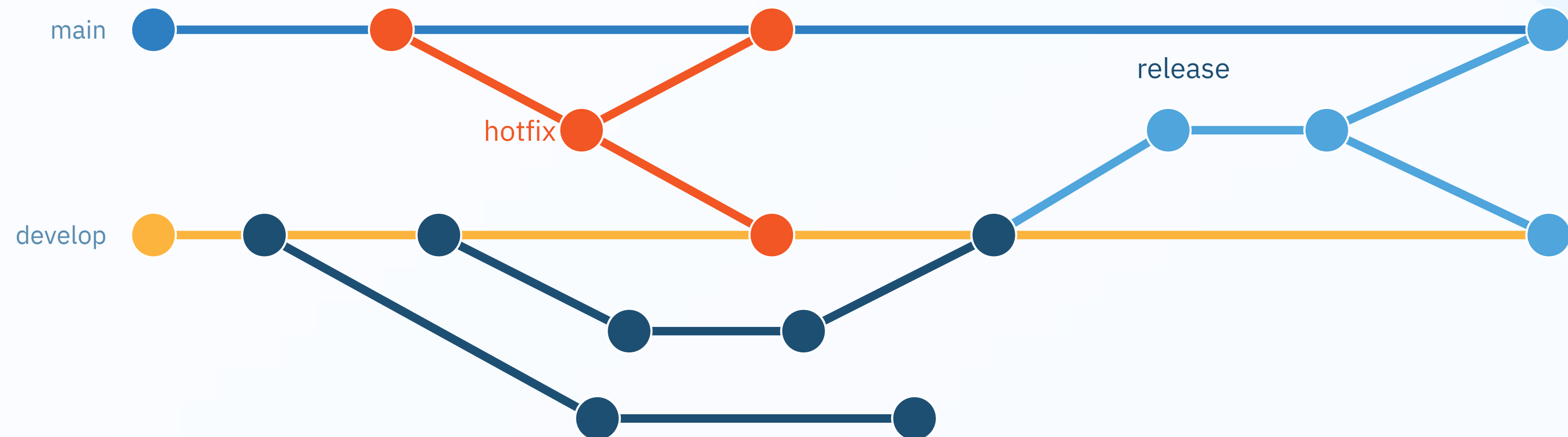
— BRANCHING WORKFLOWS

How GitFlow Works



— BRANCHING WORKFLOWS

How GitFlow Works



Hotfix branch

Created directly from main to quickly fix a production issue. After the fix, it's merged back into main and also into develop to keep future releases aligned.

Where Does GitFlow Shine?



1. Software that ships in versions



2. Multiple versions simultaneously



3. Large teams needing clear structure



4. Industry where changes need approval

GitFlow's Downsides



1. Slows down your development cycle



2. Doesn't play well with continuous delivery



3. Not suitable for web apps that deploy continuously

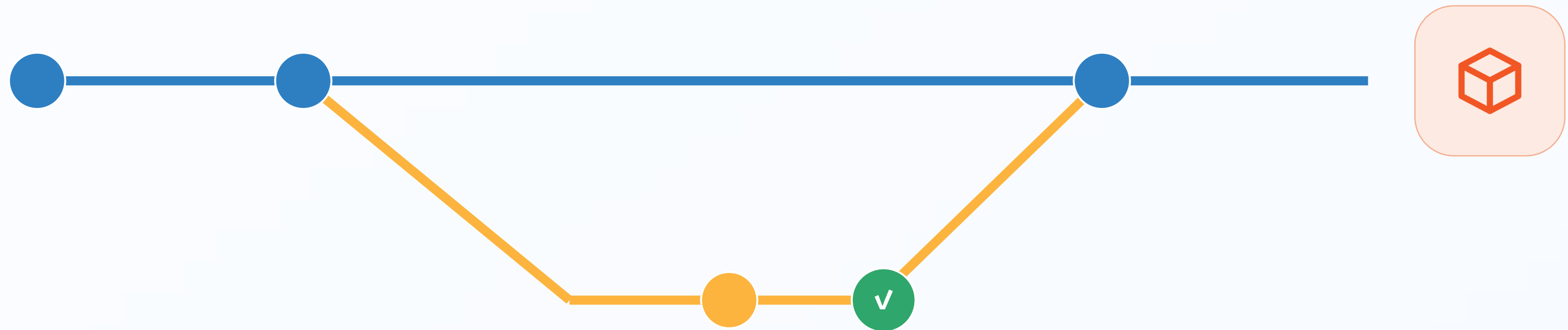
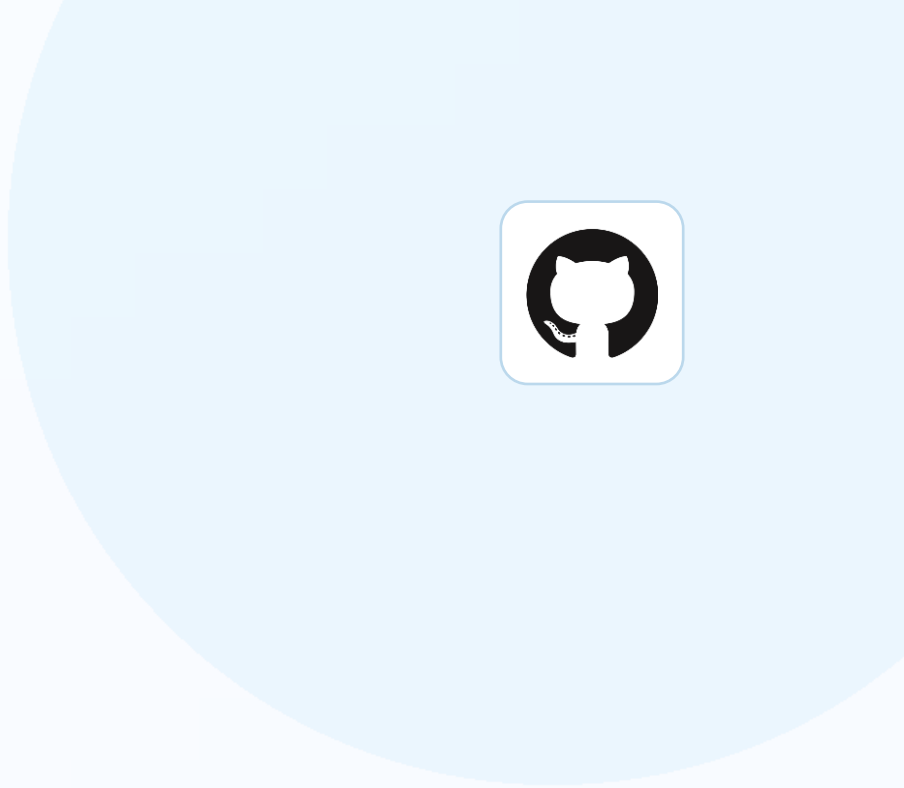
GitHub Flow



The core principle of GitHub Flow is:

Anything in the main branch should always be deployable.

GitHub Flow

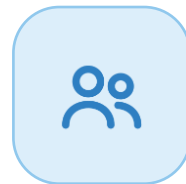




When Does GitHub Flow Shine?



**1. Web apps that
deploy constantly**



**2. Small to medium
sized teams**



**3. Solid automated
testing and CI/CD
pipelines**



**4. Ship features
instead of
managing branches**



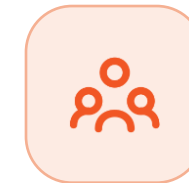
GitHub Flow's Downsides



1. Doesn't handle multiple versions



2. Demands discipline

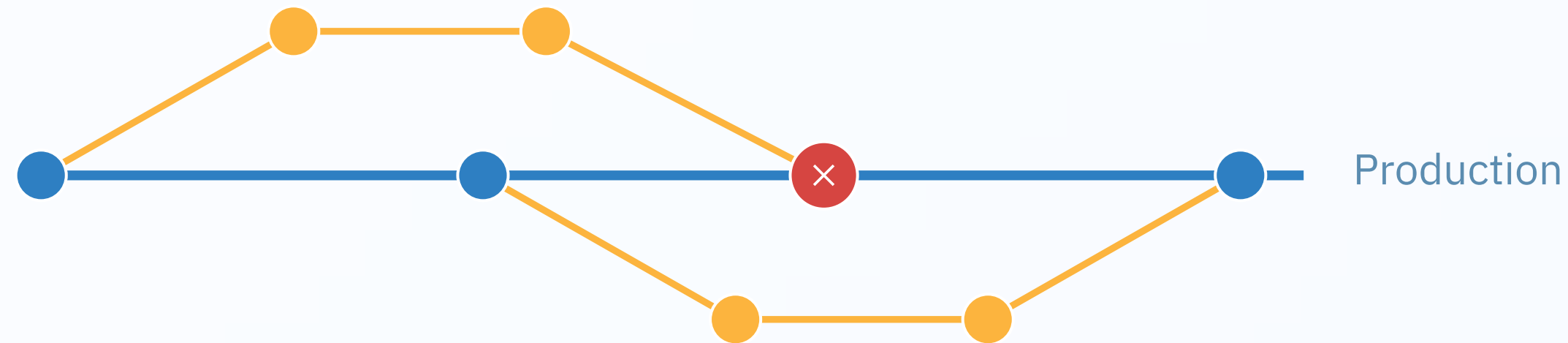


3. Struggles when multiple teams are working on the same codebase



— BRANCHING WORKFLOWS

The Truth About GitHub Flow



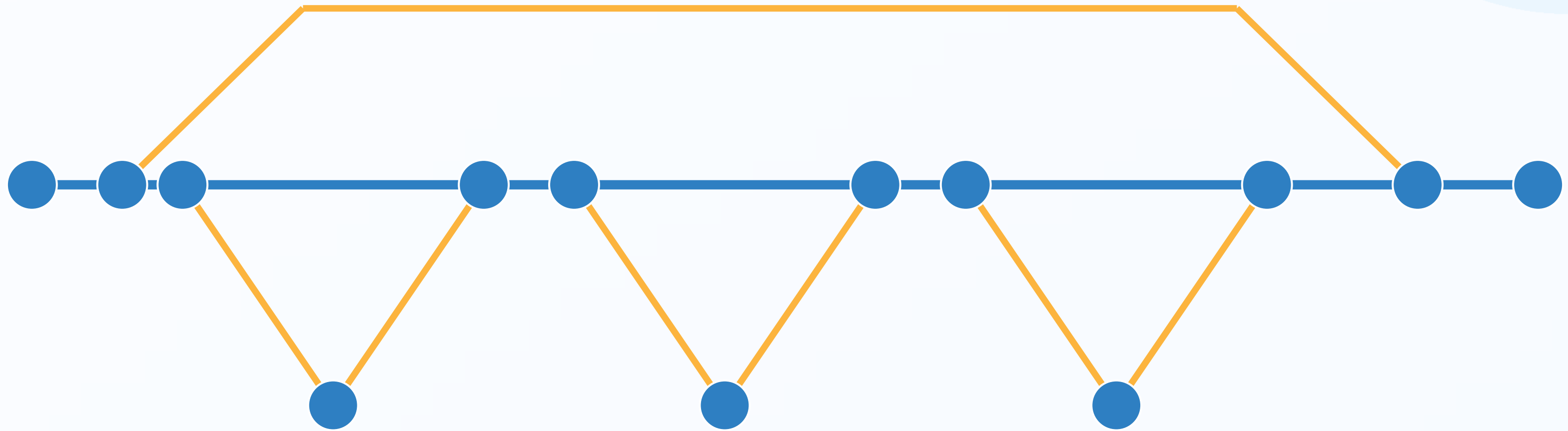
GitHub Flow

- ✓ Team has good habits
- ✓ They are in sync
- ✗ Pull request = Optional suggestion
- ✗ Testing = Homework they forgot to do

} Regret using
Github Flow



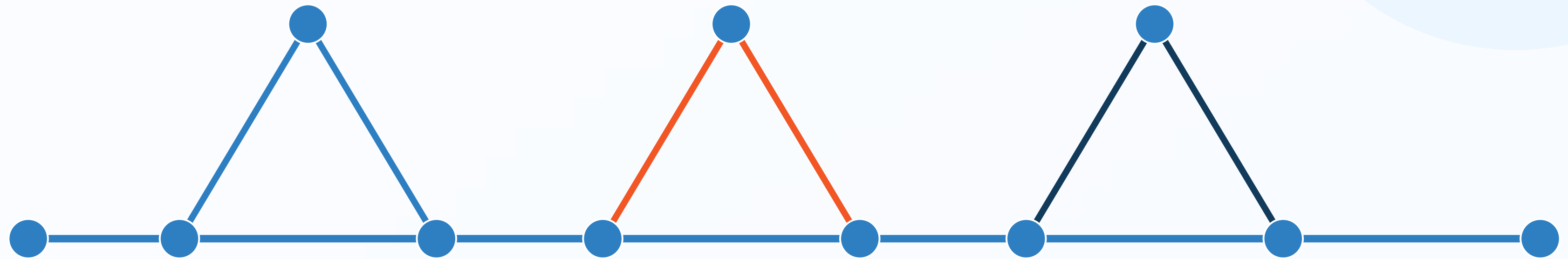
Trunk-Based Development



Integrate continuously

— BRANCHING WORKFLOWS

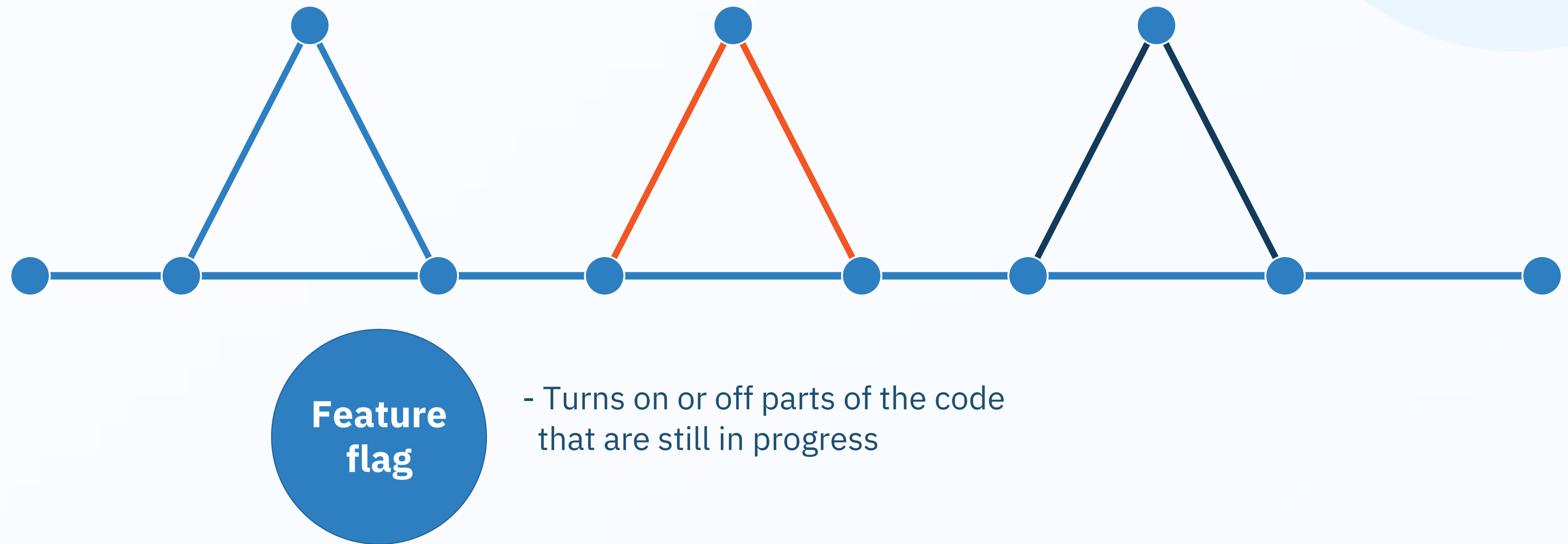
The Real Scenario



But that means that the work is in progress, so they are committing half done features into the main.. How does that make sense?

— BRANCHING WORKFLOWS

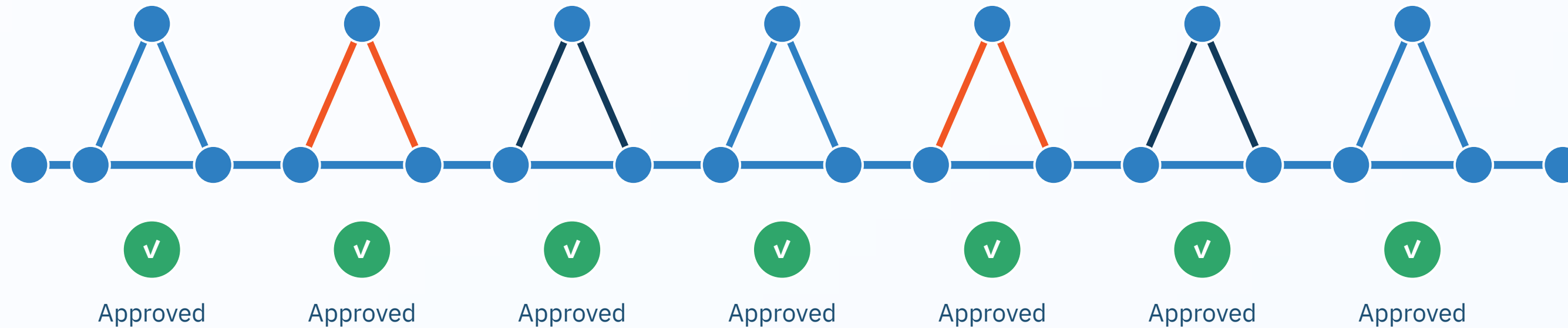
The Real Scenario



— BRANCHING WORKFLOWS

The Real Scenario

All week



✓ Zero merge conflicts



Live

When does TBD Shine?



**1. Your team
consists mainly of
experienced
developers**



**2. Bulletproof
automated testing**



**3. Rapid integration
and faster
deployment**



**4. Shipping SaaS
products with one
version in production**

TBD's Challenges



1. Doesn't handle multiple versions



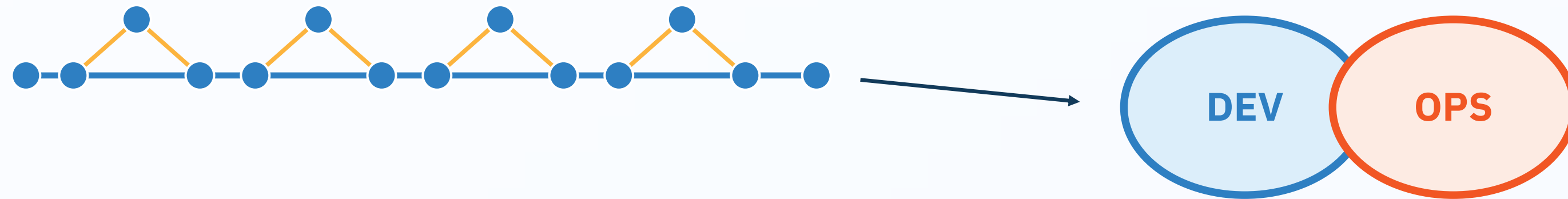
2. Test coverage is weak



3. It requires cultural shift

— BRANCHING WORKFLOWS

The Truth About TBD



It doesn't work without:



Strong CI/CD Pipeline



Automated Tests

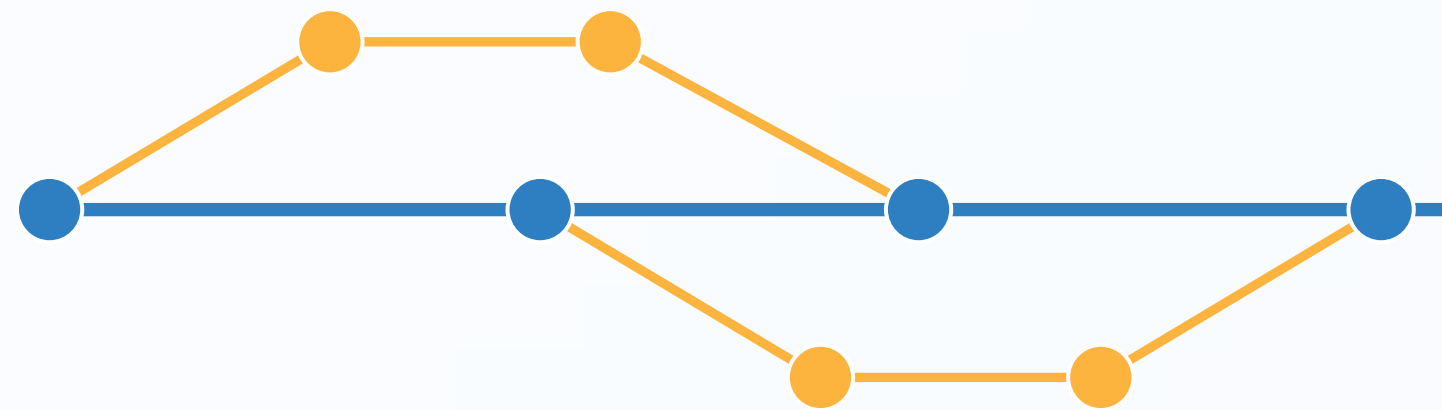


Culture of Quality

— BRANCHING WORKFLOWS

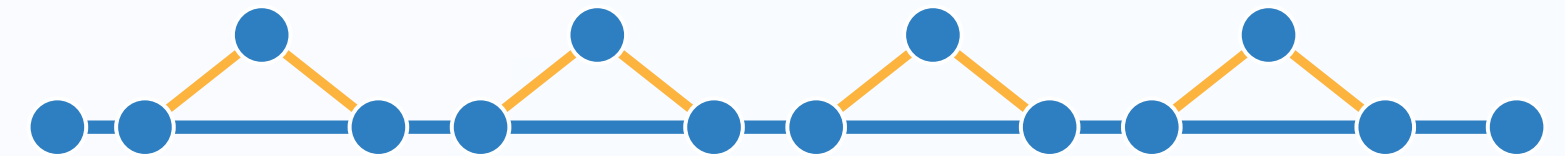
Real World Reflection

Most teams don't follow a standard workflow



GitHub Flow

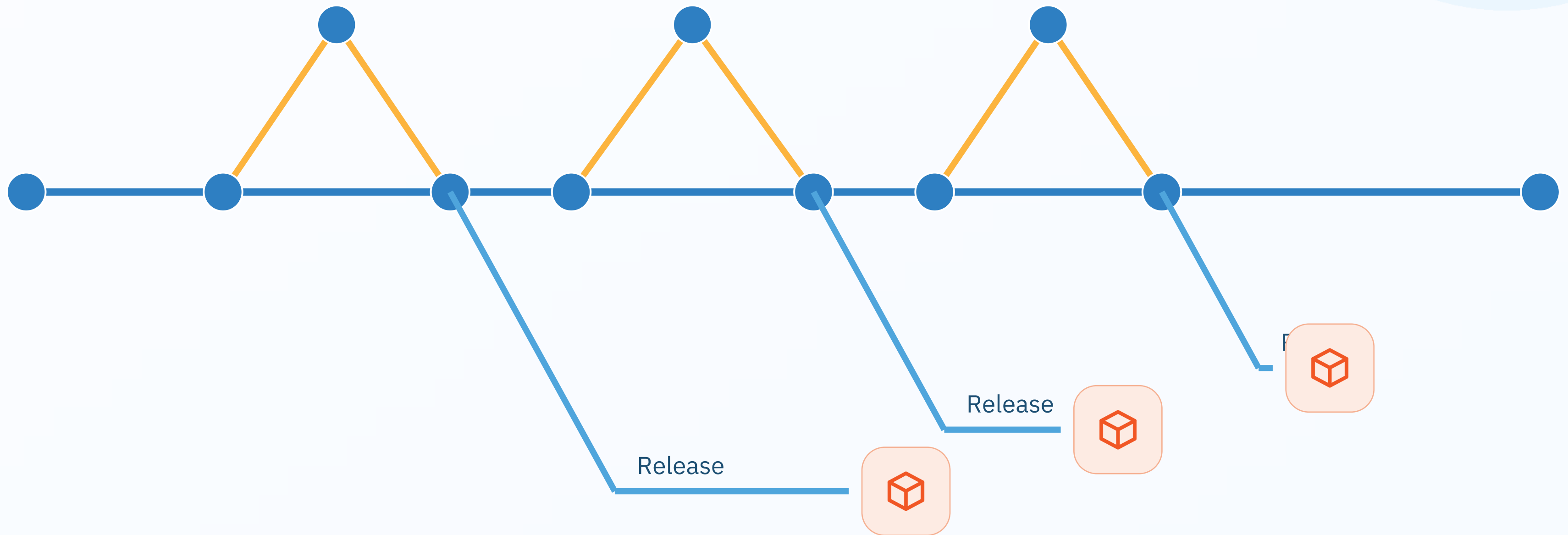
+



Trunk-Based Development

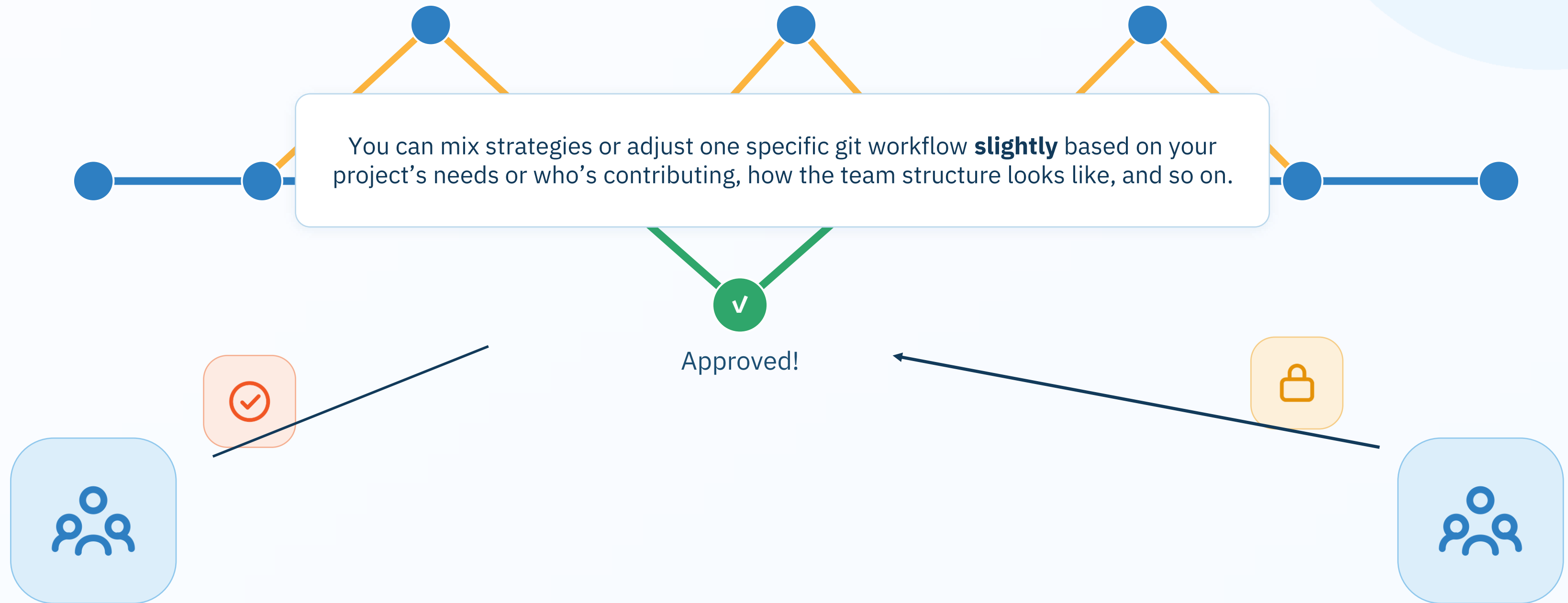
Real World Reflection

TBD workflow adjusted slightly



— BRANCHING WORKFLOWS

Real World Reflection



— BRANCHING WORKFLOWS

Developing in Main



 PART 02

Merge conflicts and merge strategies



How and When Conflicts Occur

When Integrating Commits from Different Sources

`git merge`

`git rebase`

⚡ **Merge Conflicts** ⚡

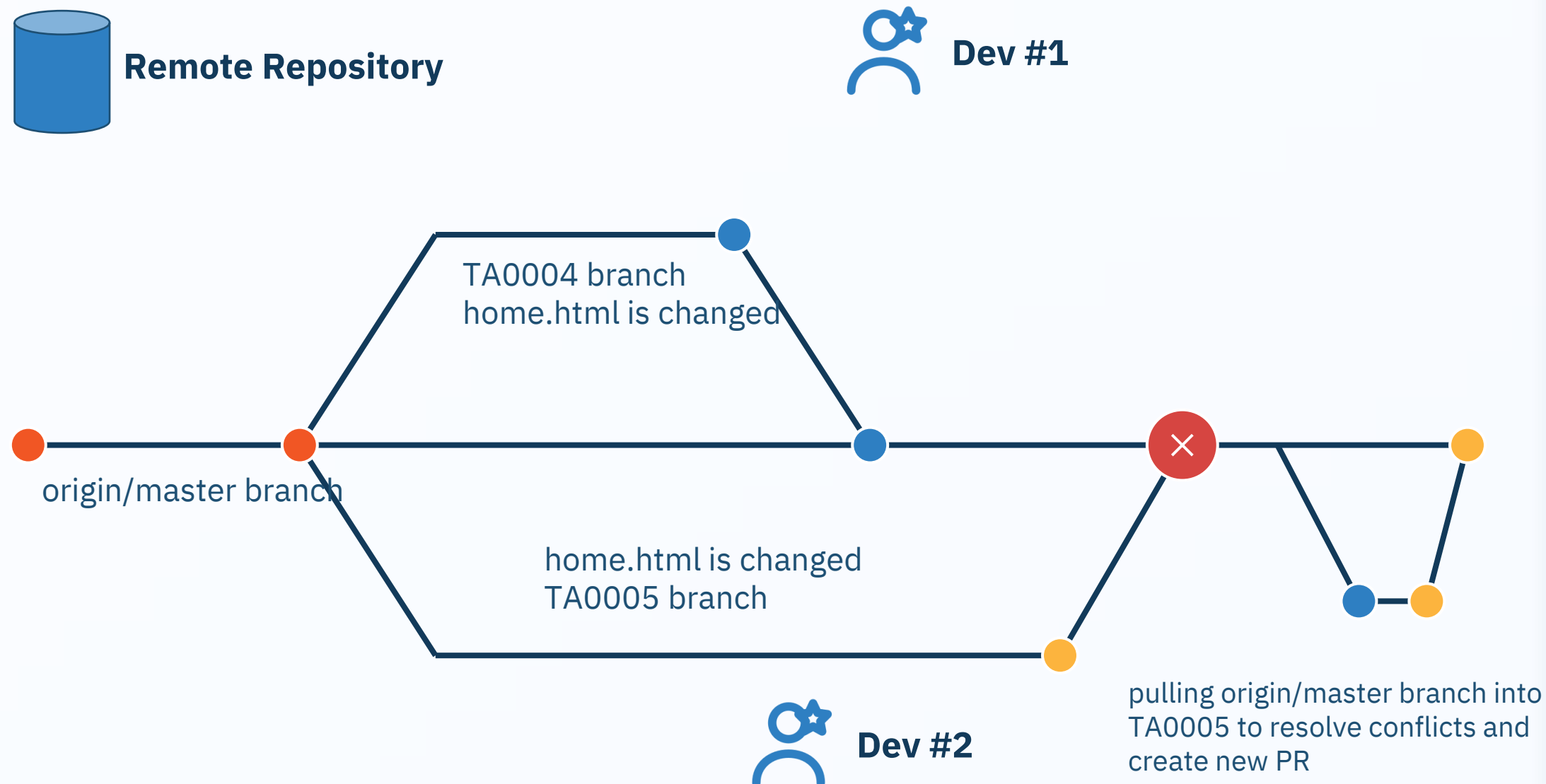
`git pull`

`git cherry-pick`

`git stash apply`

— MERGE CONFLICTS AND MERGE STRATEGIES

When merge conflict can appear?

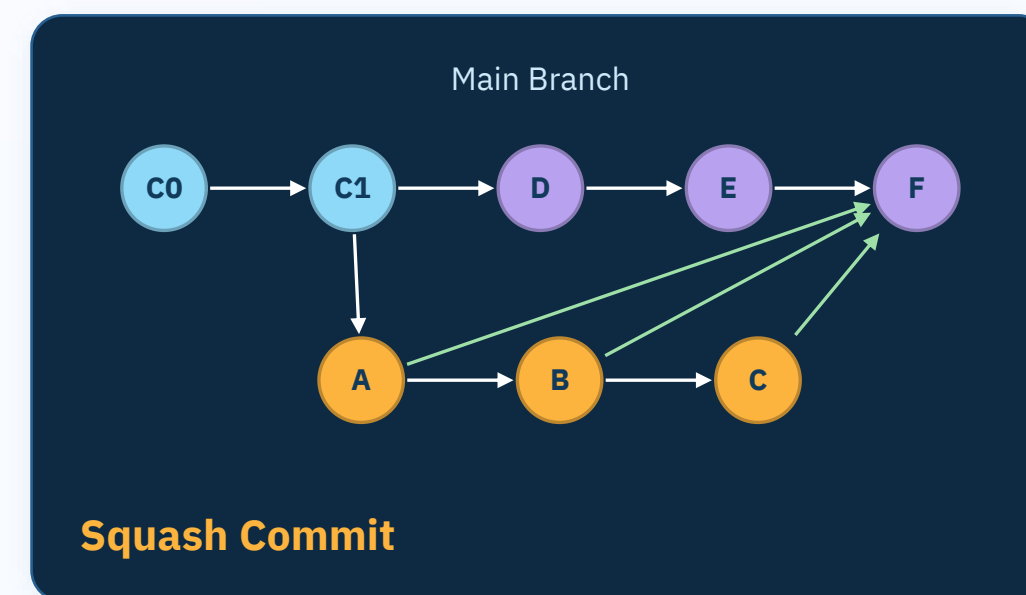
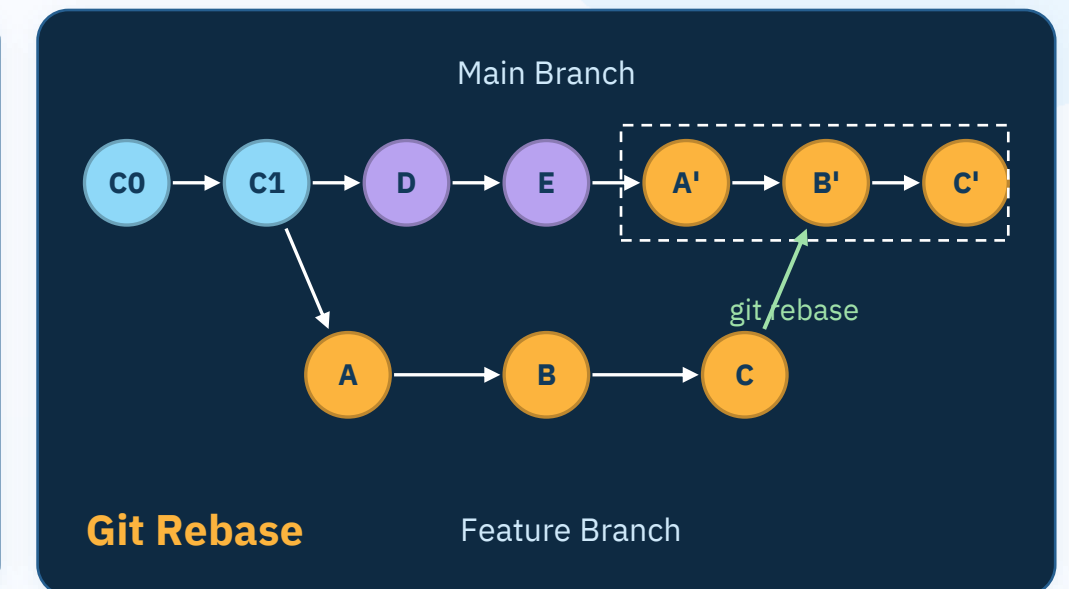
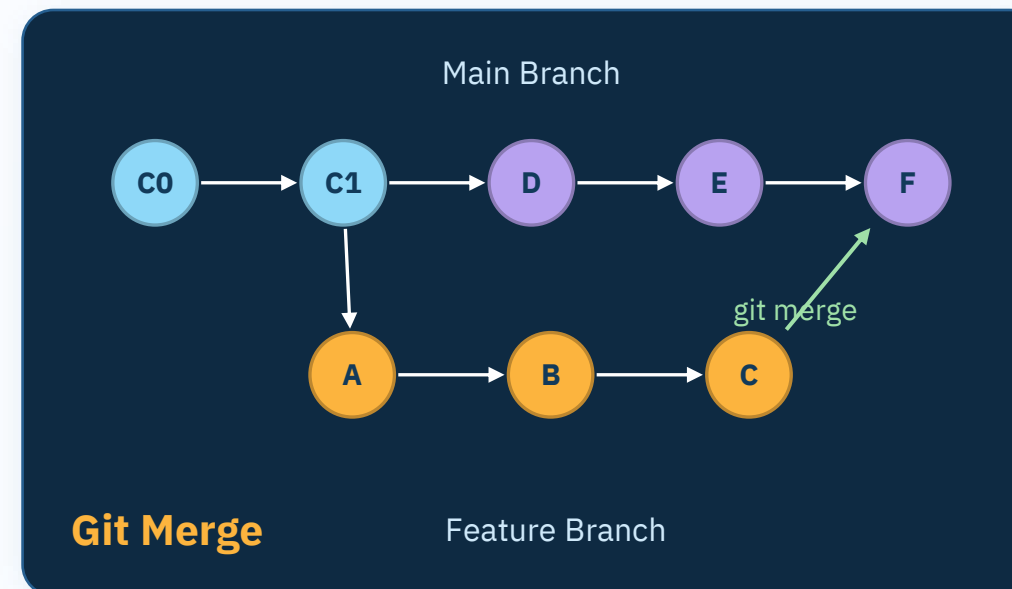
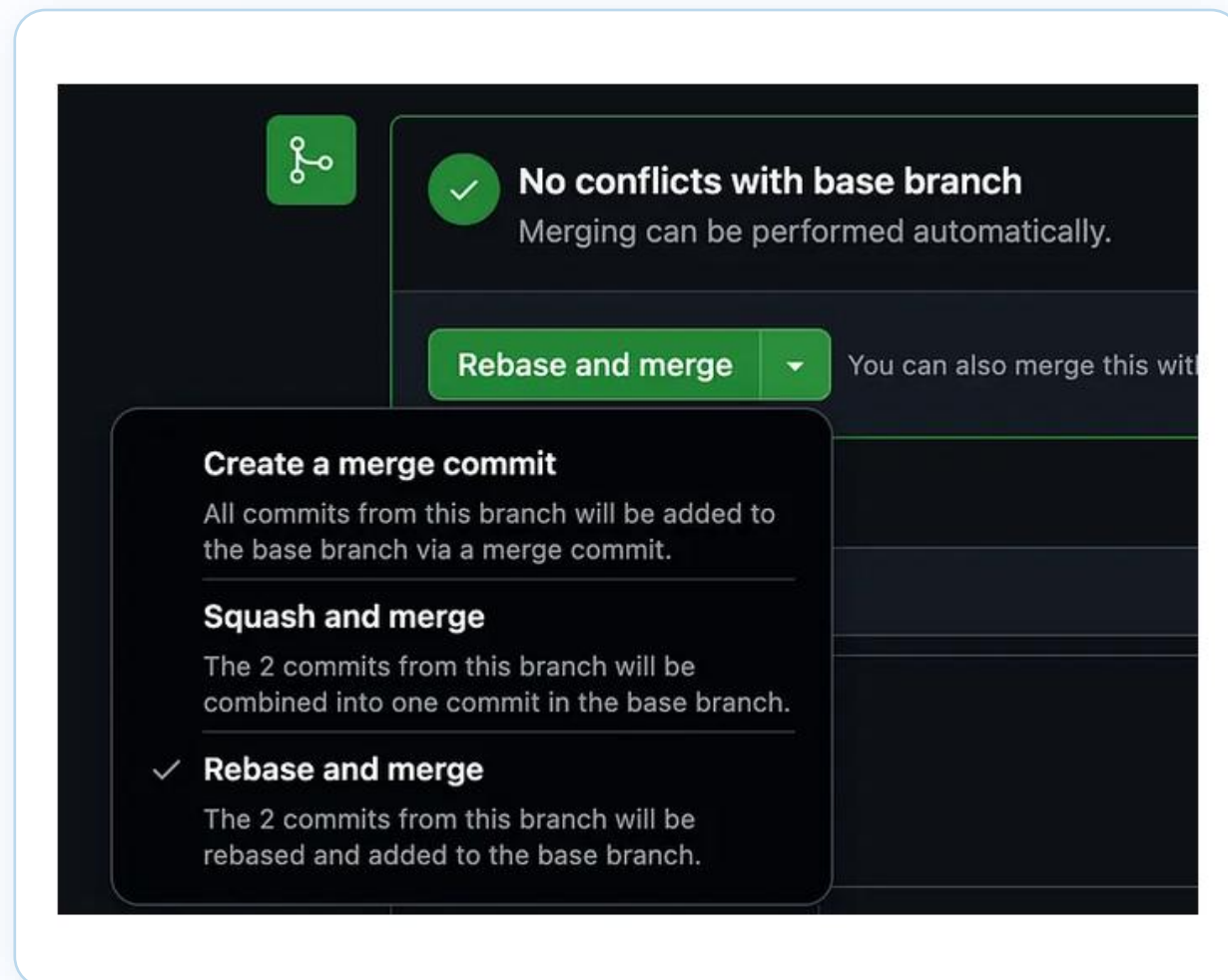


SCENARIO

- 1) Developer #1 creates branch and change 'home.html' file
- 2) Developer #2 creates branch and change 'home.html' file
- 3) Developer #1 creates PR to origin master
- 4) Developer #2 creates PR to origin master
- 5) PR of Developer #1 was merged, and another PR has merge conflicts
- 6) Developer #2 updates local repository to resolve merge conflicts and update pull request

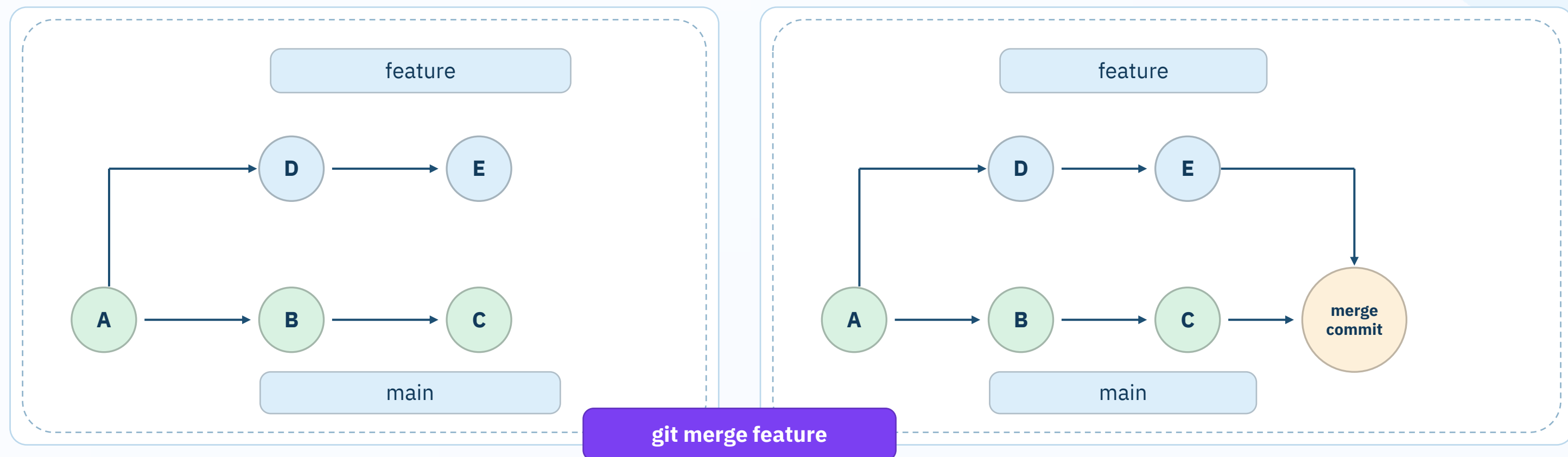
MERGE CONFLICTS AND MERGE STRATEGIES

Git merging strategy



— MERGE CONFLICTS AND MERGE STRATEGIES

Git Merge: Create a merge commit



<https://medium.com/@davide.rubinetti97/git-rebase-vs-merge-vs-squash-different-strategies-explained-7df068f37578>

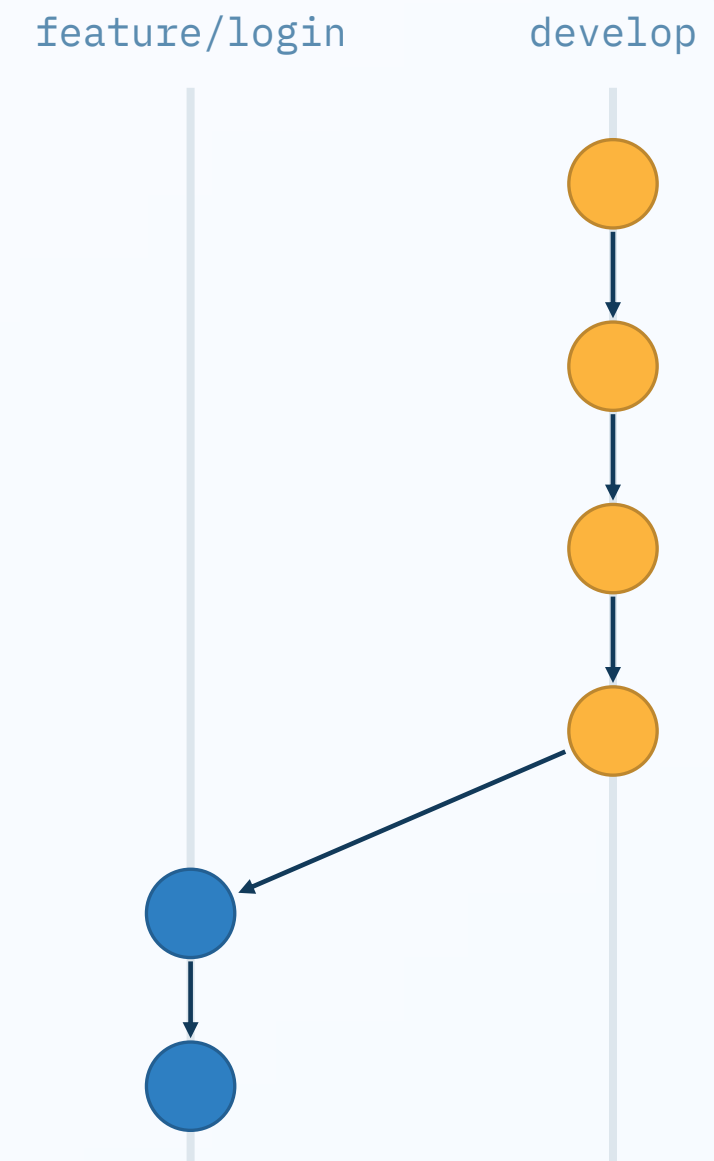
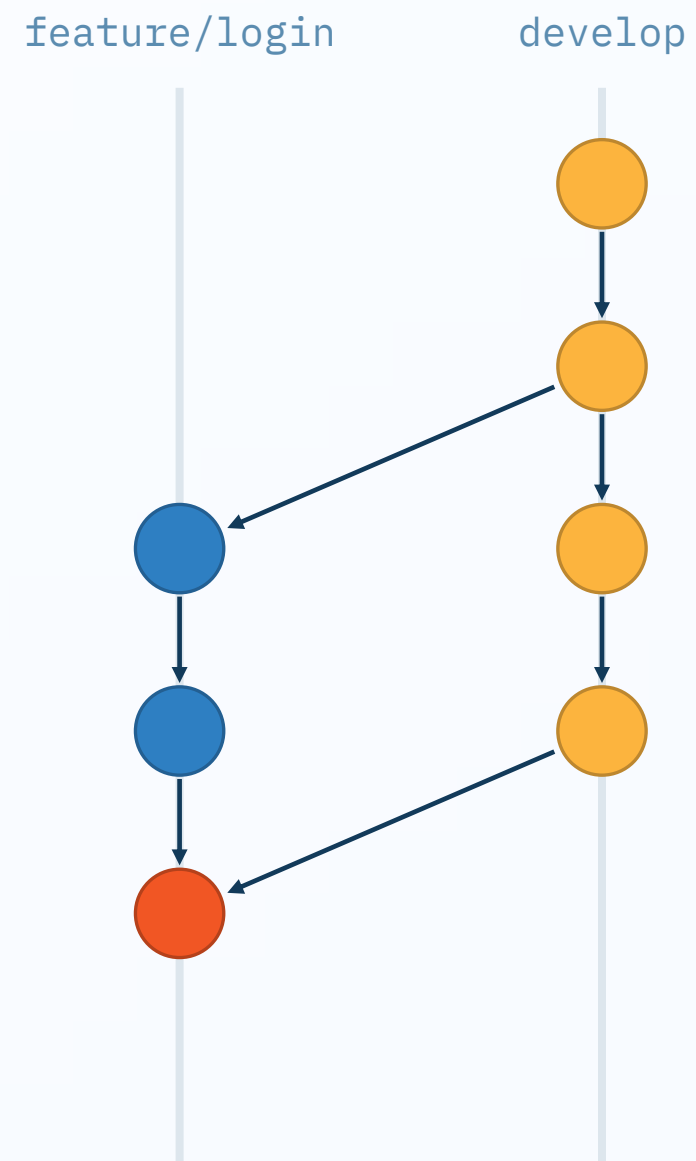
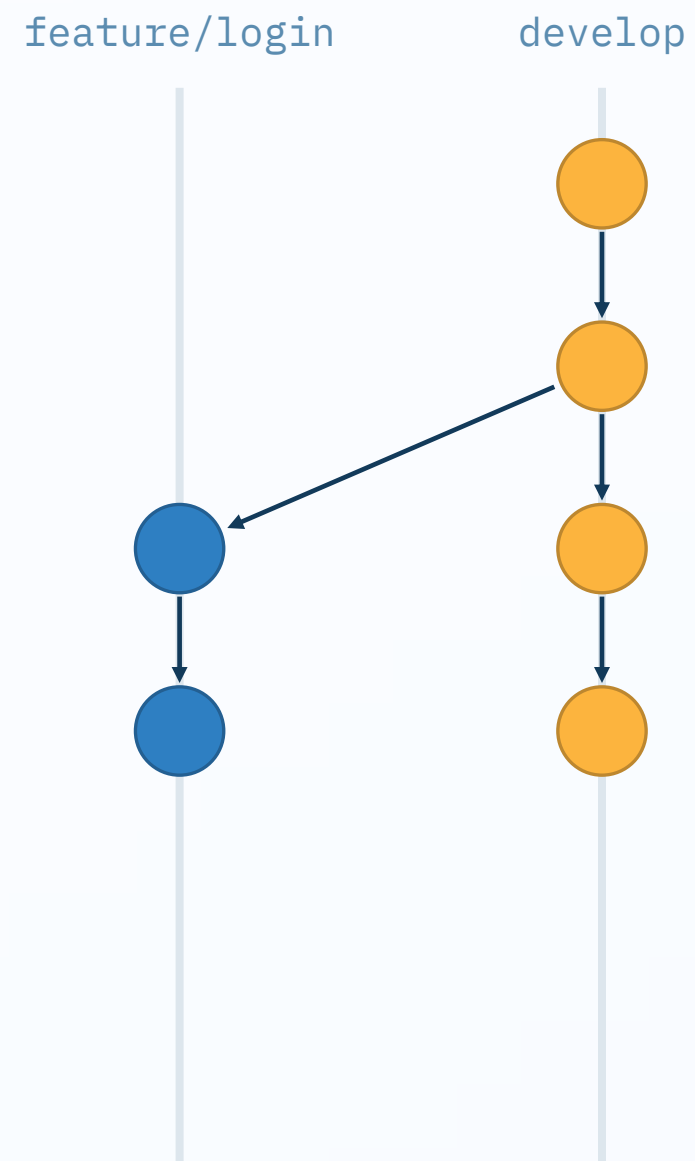
MERGE CONFLICTS AND MERGE STRATEGIES

Git rebase



`$ git merge develop`

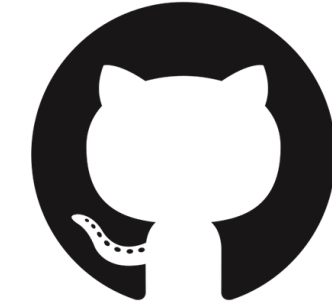
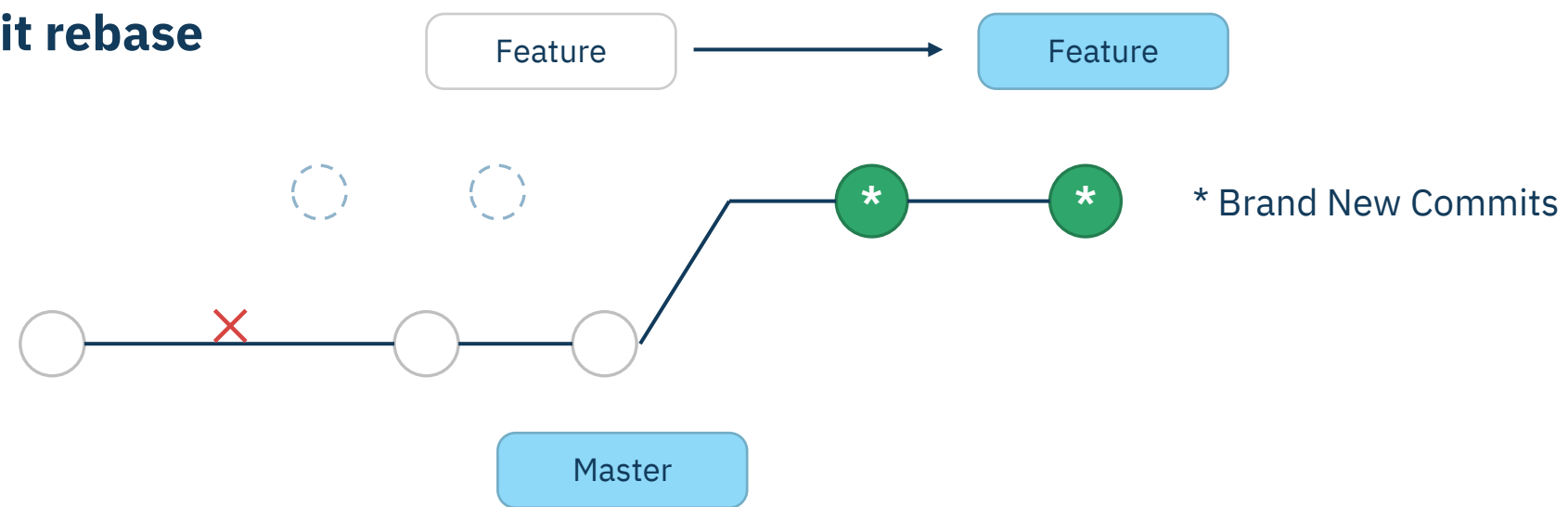
`$ git rebase develop`



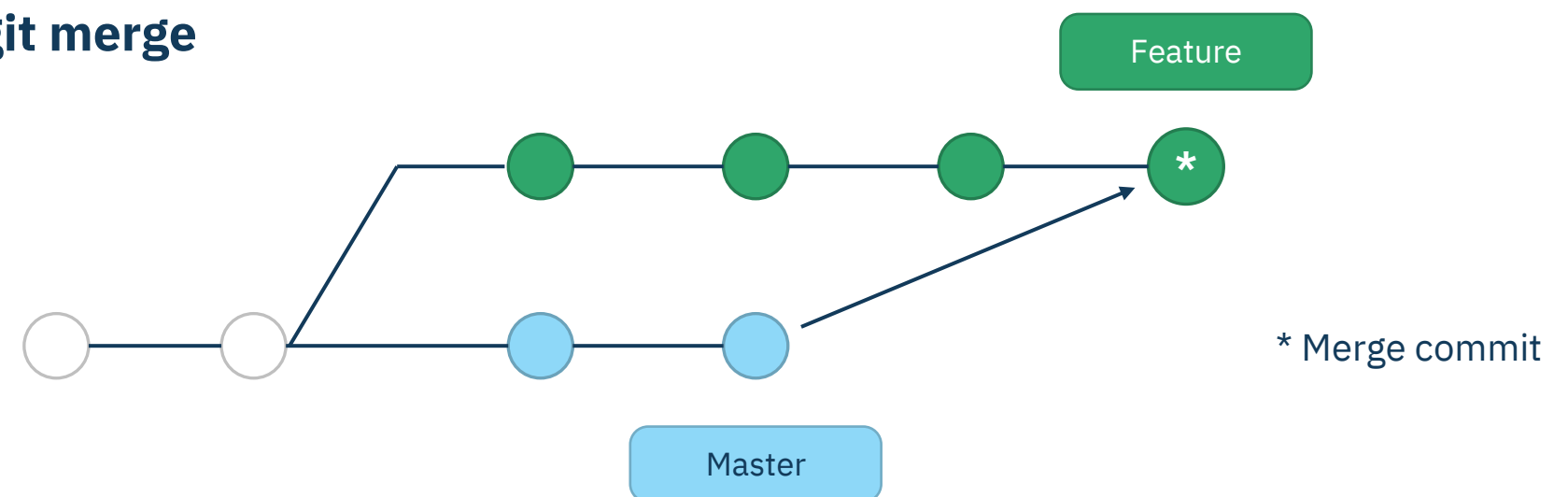
— MERGE CONFLICTS AND MERGE STRATEGIES

What to use? Merge or Rebase?

git rebase

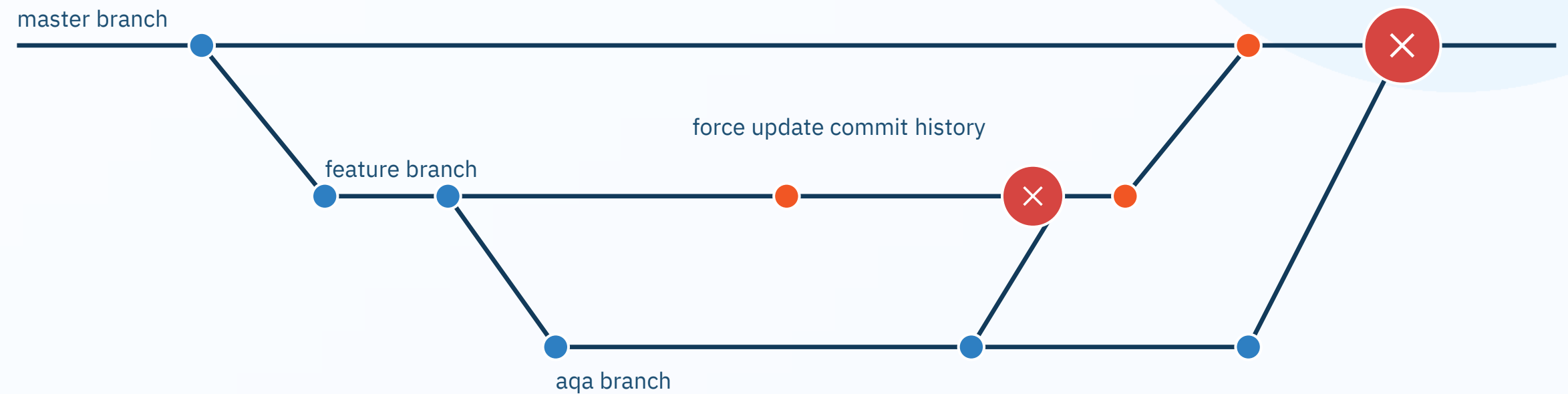


git merge

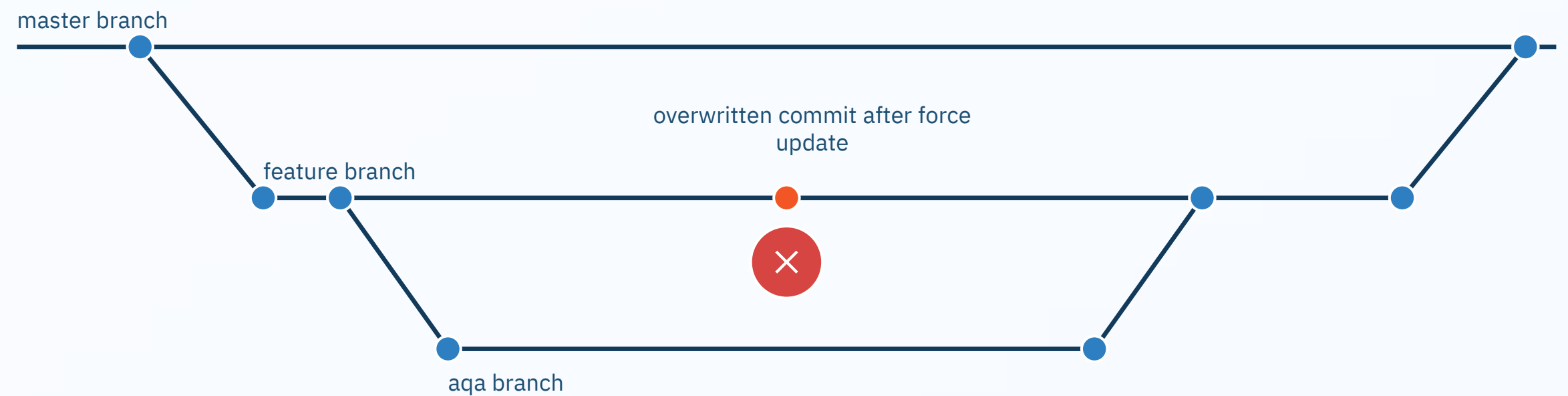


Force update risks

1) Change history for branch which is used by other developers



2) Overwrite other changes



FOUR RULES OF HAPPY WOTK WITH GIT

by Andrii Piatakha



- 1 Always create branches from the master
- 2 Force update and change commit history only on your branches
- 3 Use `--force-with-lease` instead of `-f`
- 4 Always rebase on the origin master branch before creating a pull request



Three commit on the same issues

```
commit 0ad216bcc6a7104098009afecdd6ebb797c17c54 (HEAD -> TA0005_home_carousel)
```

```
Author: Andrii Piatakha <andrii.piatakha@example.com>
```

```
Date: Thu Oct 29 19:18:36 2020 +0200
```

```
one more css class for home carousel slider is added
```

```
commit 52b4c7b3ef8a522e122fe76bc63c7c894a11f506
```

```
Author: Andrii Piatakha <andrii.piatakha@example.com>
```

```
Date: Thu Oct 29 19:17:49 2020 +0200
```

```
css class for scrolling carousel on the home page is added
```

```
commit 297df6aa733bd474c2562d53961489428b1da672
```

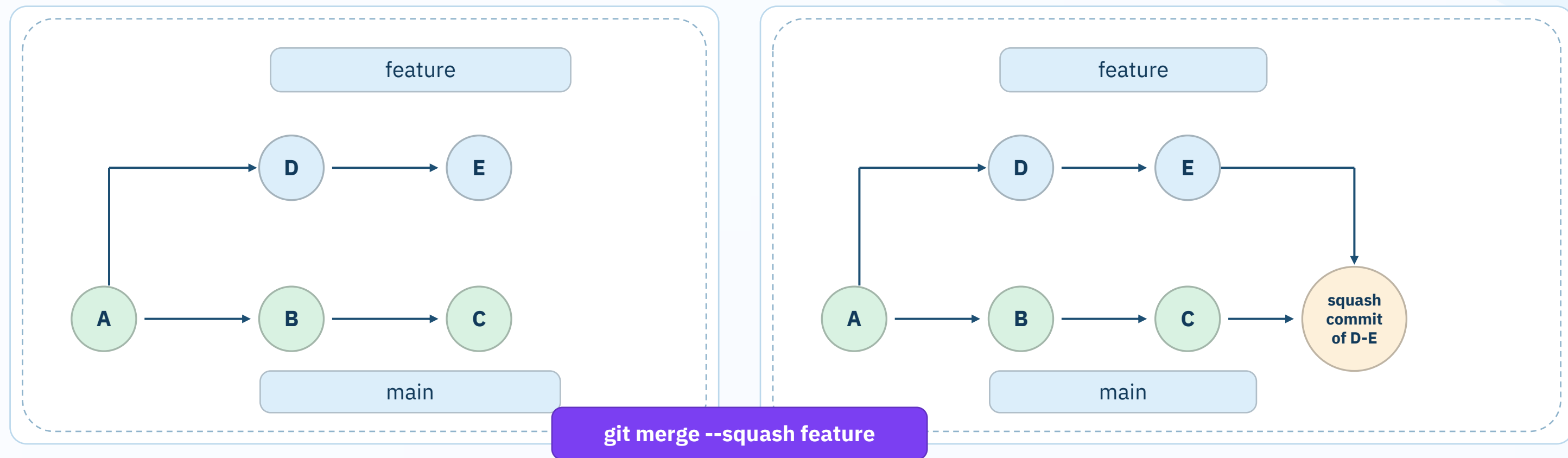
```
Author: Andrii Piatakha <andrii.piatakha@example.com>
```

```
Date: Mon Oct 26 23:27:26 2020 +0200
```

```
carousel on the home page
```

— MERGE CONFLICTS AND MERGE STRATEGIES

Squash and merge





Use interactive rebase

Git rebase -i

```
$ git rebase -i HEAD~3  
hint: Waiting for your editor to close the file...
```

Squash



Squash



```
pick 297df6a carousel on the home page  
s 52b4c7b css class for scrolling carousel on the home page is added  
s 0ad216b one more css class for home carousel slider is added
```

```
# Rebase e50bada..0ad216b onto e50bada (3 commands)
```

git-rebase-todo



Use interactive rebase

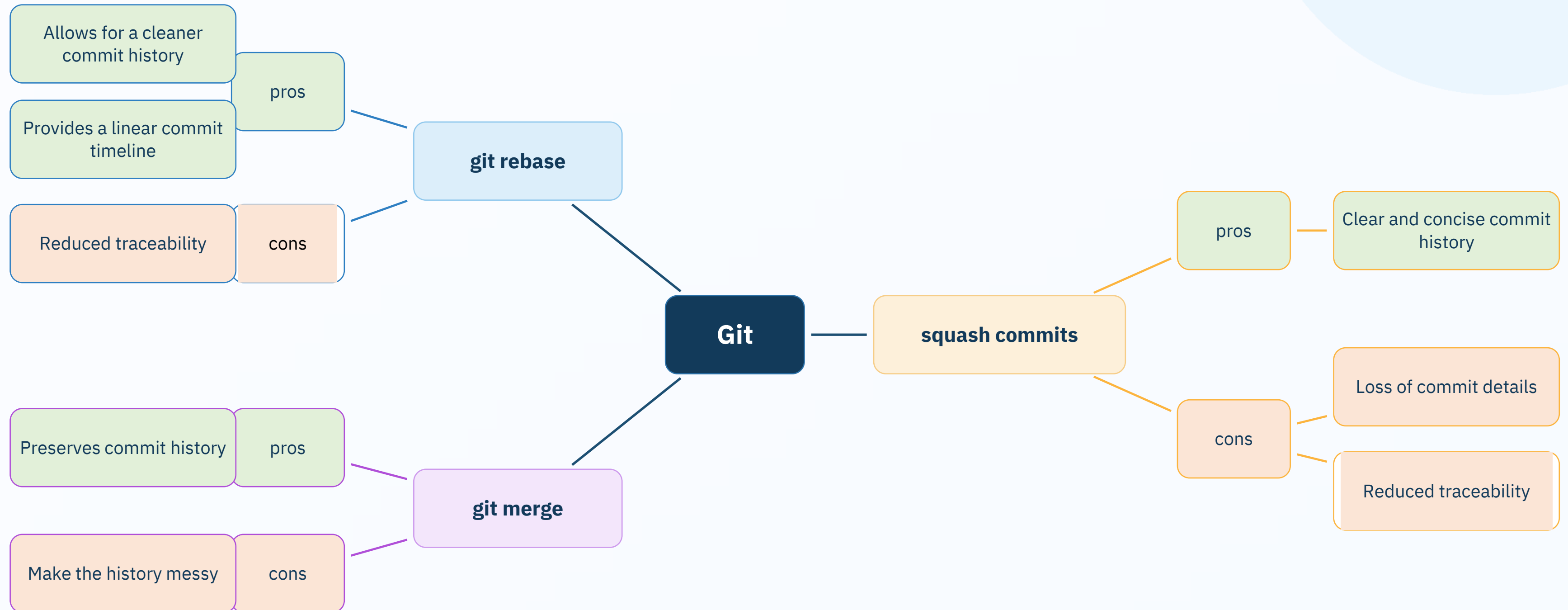
git-rebase-todo

```
#
# Commands:
# p, pick <commit> = use commit
# r, reword <commit> = use commit, but edit the commit message
# e, edit <commit> = use commit, but stop for amending
# s, squash <commit> = use commit, but meld into previous commit
# f, fixup <commit> = like "squash", but discard this commit's log message
# x, exec <command> = run command (the rest of the line) using shell
# b, break = stop here (continue rebase later with 'git rebase --continue')
# d, drop <commit> = remove commit
# l, label <label> = label current HEAD with a name
# t, reset <label> = reset HEAD to a label
# m, merge [-C <commit> | -c <commit>] <label> [# <oneline>]
# .      create a merge commit using the original merge commit's
# .      message (or the oneline, if no original merge commit was
# .      specified). Use -c <commit> to reword the commit message.
#
# These lines can be re-ordered; they are executed from top to bottom.
#
# If you remove a line here THAT COMMIT WILL BE LOST.
#
# However, if you remove everything, the rebase will be aborted.
```

— MERGE CONFLICTS AND MERGE STRATEGIES



Git rebase, git merge and squash commits



 PART 03

The perfect commit

— THE PERFECT COMMIT

The Perfect Commit Message



- 1 Subject = *concise summary of what happened*
- 2 Body = *more detailed explanation*
 - ◆ What is now different than before?
 - ◆ What's the reason for the change?
 - ◆ Is there anything to watch out for / anything particularly remarkable?

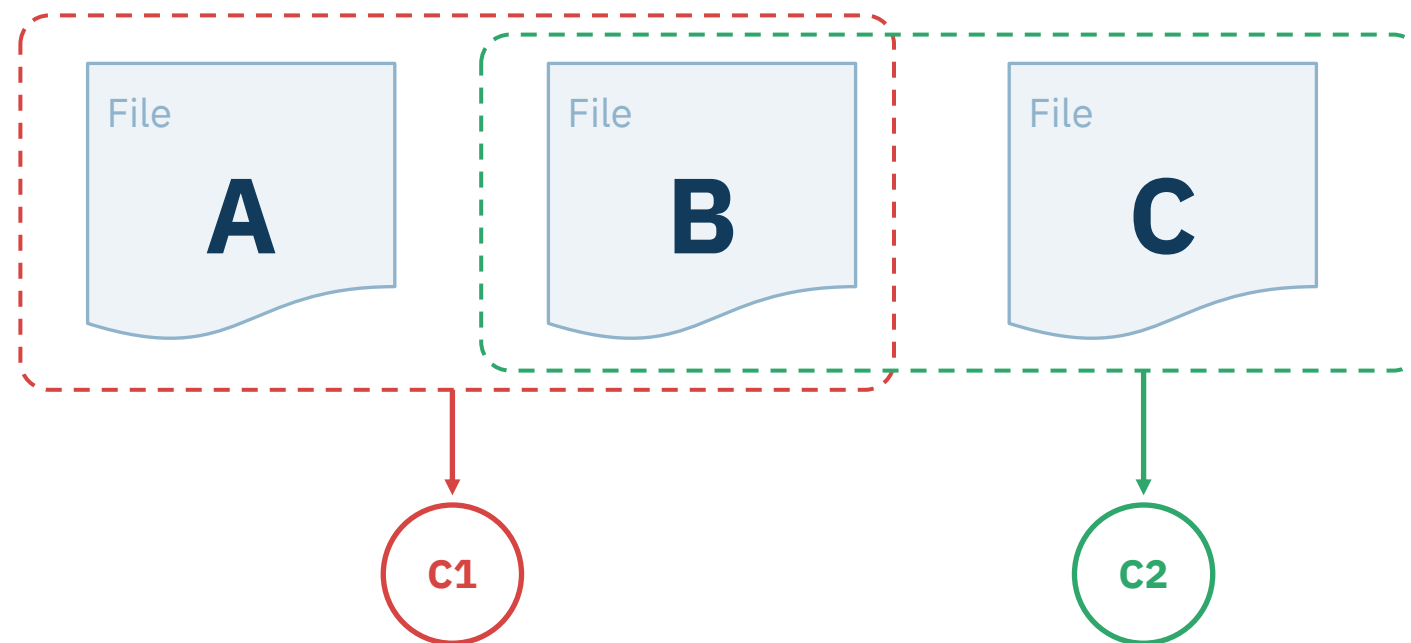
— THE PERFECT COMMIT

The “Perfect commit”



- 1 Add the right changes!
- 2 Compose a good commit message!

The Perfect Commit



Should only contain the right change

```
perfect-commit$ git add -p index.html
diff --git a/index.html b/index.html
index 9648eb0..f242484 100644
--- a/index.html
+++ b/index.html
@@ -13,7 +13,9 @@
     <div id="navigation">
       <ul>
         <li><a href="index.html">Home</a></li>
-        <li><a href="about.html">About</a></li>
+        <li><a href="about.html">About Us</a></li>
+        <li><a href="product.html">Product</a></li>
+        <li><a href="support.html">Support</a></li>
         <li><a href="imprint.html">Imprint</a></li>
       </ul>
     </div>
(1/2) Stage this hunk [y,n,q,a,d,j,J,g,/,e,]?
```

— THE PERFECT COMMIT

Common .gitignore Patterns

Here are some common patterns used in .gitignore files:



```
# Node.js Project

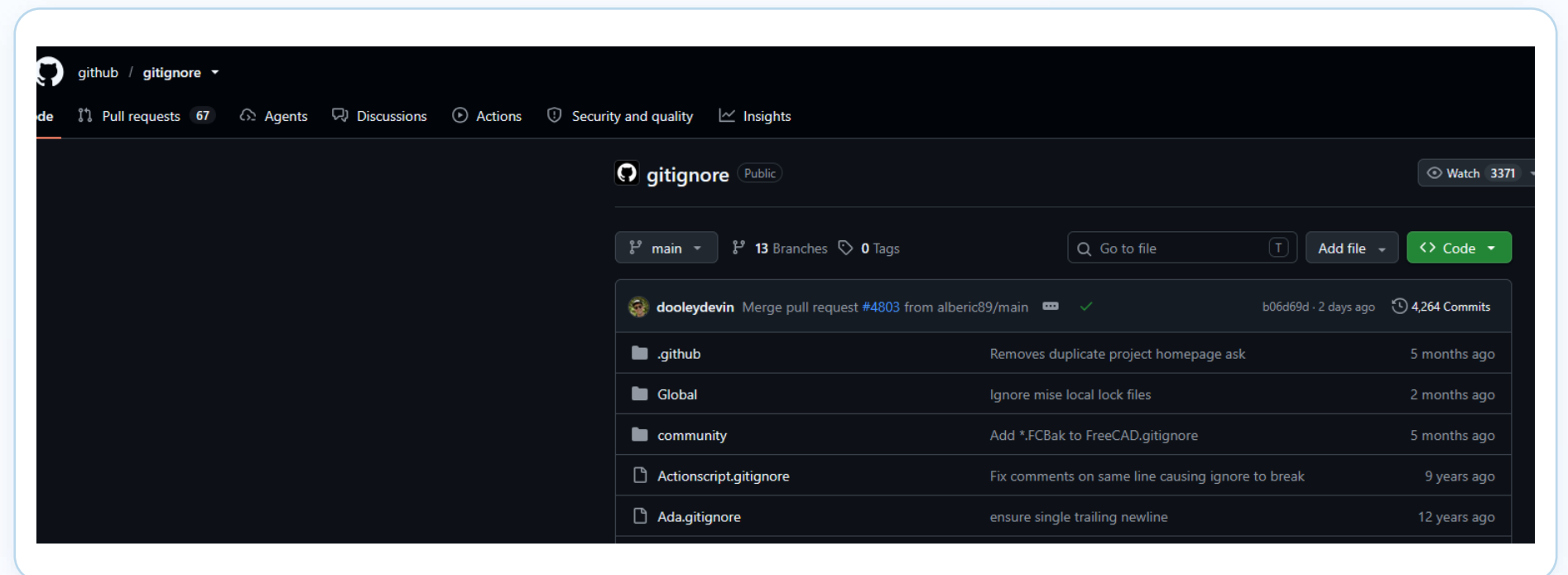
# Ignore node_modules directory
node_modules/

# Ignore logs
npm-debug.log*

#Python Project

# Ignore Python bytecode
__pycache__/
*.py[cod]

# Ignore virtual environments
venv/
```

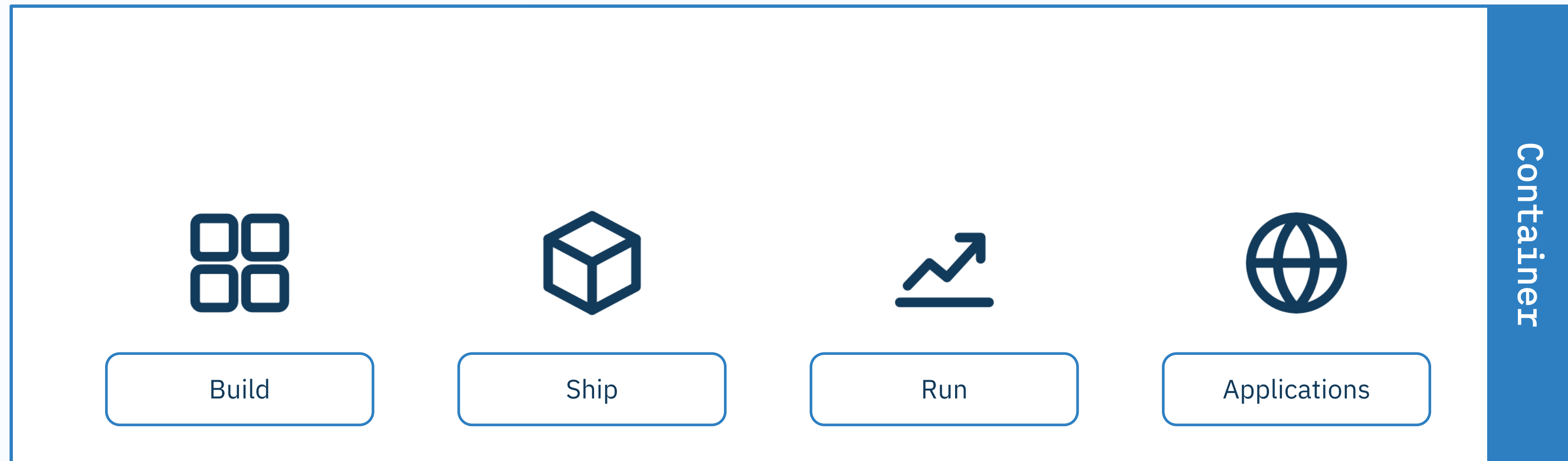


 PART 04

Containers and Docker

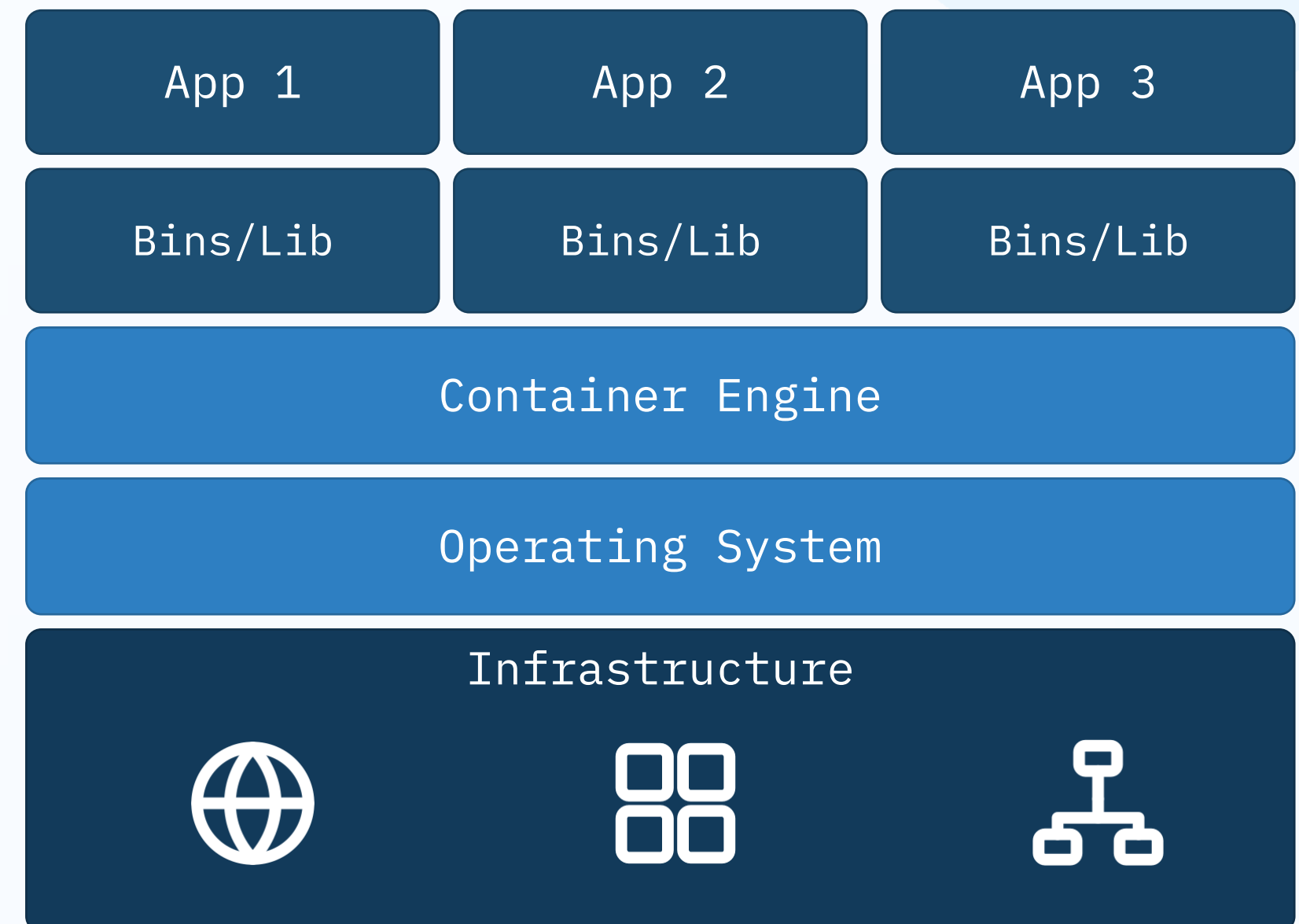
A container - defined

- ◆ A container is a standard unit of software that encapsulates everything that programmers need to build, ship, and run applications

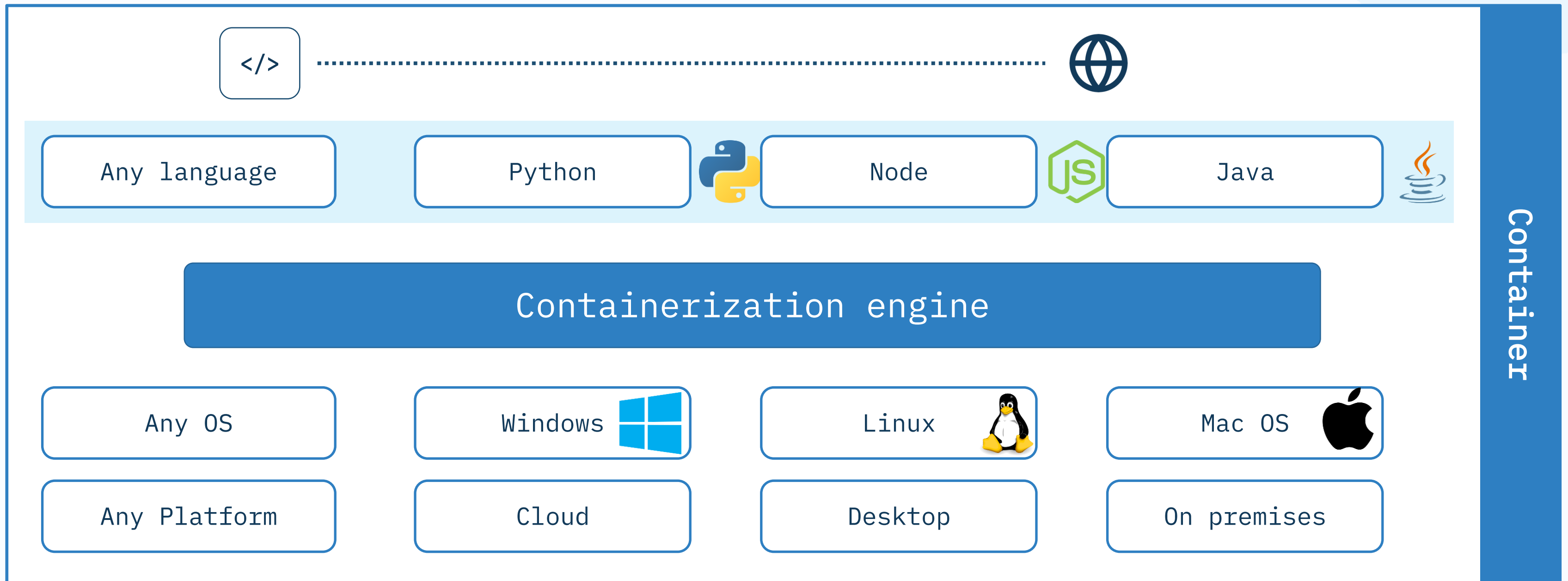


Container characteristics

- ◆ The container engine virtualizes the operating system
- ◆ A container is light-weight, fast, isolated, portable and secure
- ◆ Require less memory space
- ◆ Binaries, libraries within container enable apps to run
- ◆ One machine can host multiple containers



Containers offer easy portability



Container benefits

Containers enable organizations to:

- ◆ Quickly create applications using automation
- ◆ Lower deployment time and costs
- ◆ Improve resource utilization (CPU, memory)
- ◆ Port across different environments
- ◆ Support next-gen applications (microservices)

Container challenges

- ◆ Security impacted if operating system affected
- ◆ Difficult to manage thousands of containers
- ◆ Complex to migrate legacy projects to container technology
- ◆ Difficult to right-size containers for specific scenarios

Container vendors



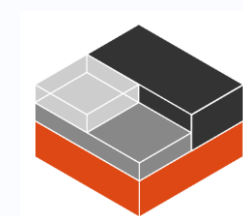
Docker

- ◆ Robust and most popular container platform today



Podman

- ◆ Daemon-less architecture providing more security than Docker containers



LXC

- ◆ Preferred for data-intensive apps and ops



Vagrant

- ◆ Offers highest levels of isolation on the running physical machine

Docker defined



Available since 2013, Docker is an open platform, or engine, where programmers can:



Develop



Ship



Run



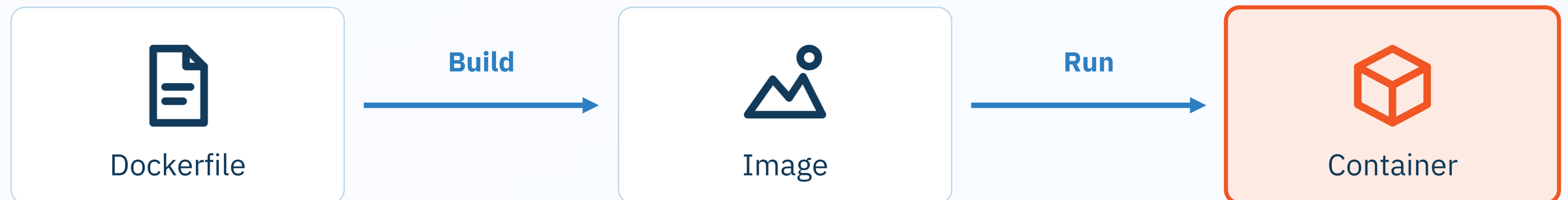
Containers

Docker container creation process



Steps to create and run containers:

- 1 Create a Dockerfile
- 2 Use the Dockerfile to create a container image
- 3 Use the container image to create a running container





Docker build command

Create a container image using the build command:

```
docker build -t my-app:v1 .
```

tag

version

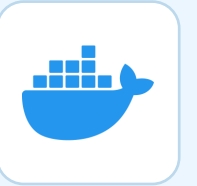
command

repository

Current directory

Output:

```
Sending build context to Docker daemon
.  
.  
Successfully built <image id>  
Successfully tagged my-app:v1
```



Docker image verification



To verify the creation of the image, run the docker images command:

```
$ docker images
```

Output:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
my-app	v1	b8b15c59b352	2 minutes ago	78.2MB



Docker run command



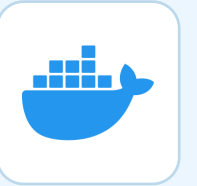
Create the container using the run command:

```
$ docker run my-app:v1
```

Hello, world!

To verify the details of the container created, run the `docker ps -a` command:

```
$ docker ps -a
```



Docker commands

Docker command	Purpose	Example
build	Creates container images from a Dockerfile	<code>docker build -t my-app:v1</code>
images	Lists all images, repositories, tags, and sizes	<code>docker images</code>
run	Creates a container from an image	<code>docker run -p 8080:80 nginx</code>
push	Stores images in a configured registry	<code>docker push my-app:v1</code>
pull	Retrieves images from a configured registry	<code>docker pull nginx</code>

Docker objects



Dockerfile

Image

Container

Network

Storage volumes

Plugins

Add-ons

Dockerfile



- ◆ A Dockerfile is a text file that contains instructions needed to create an image
- ◆ The Dockerfile is created using any editor from the console or terminal
- ◆ a few instructions that Docker provides:
 - FROM
 - Defines base image
 - RUN
 - Executes arbitrary commands
 - CMD
 - Defines default command for container execution



Dockerfile example

```
FROM python:3.13
WORKDIR /usr/local/app

# Install the application dependencies
COPY requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt

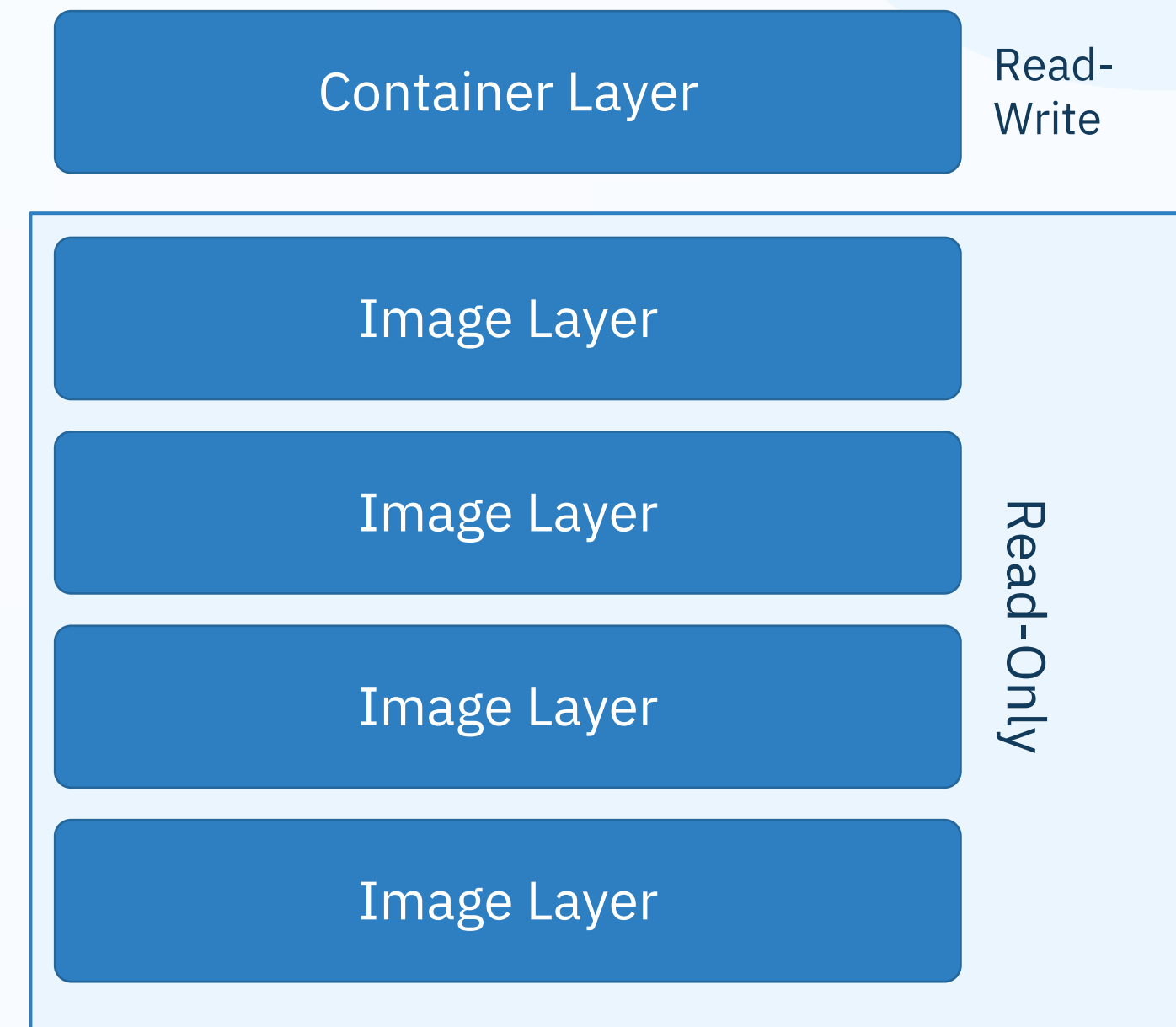
# Copy in the source code
COPY src ./src
EXPOSE 8080

# Setup an app user so the container doesn't run as the root user
RUN useradd app
USER app

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8080"]
```

Docker image

- ◆ Read-only template with instructions for creating a Docker container
- ◆ Built using instructions in a Dockerfile; a new read-only image layer is created for each instruction
- ◆ A writeable layer is added when an image is run as a container
- ◆ Layers can be shared between images, which saves disk space and network bandwidth



Docker containers

A Docker container

- ◆ Is a runnable instance of an image
- ◆ Can be created, stopped, started, or deleted using the Docker API or CLI
- ◆ Can connect to multiple networks, attach storage, or create a new image based on its current state
- ◆ Is well isolated from other containers and its host machine





Docker networks, storage, and plugins

Networks	Networks are used for the isolated container communication
Storage	Docker uses volumes and bind mounts to persist data even after a container stops
Plugins	Storage plugins provide the ability to connect to external storage platforms



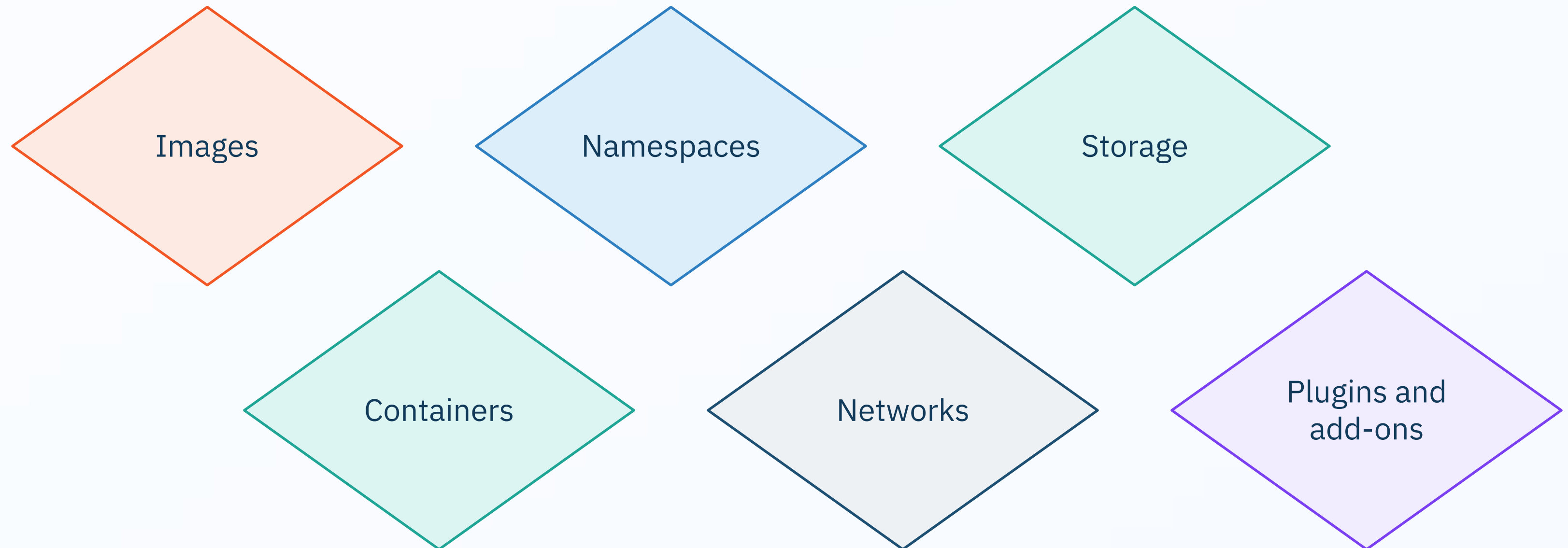
Docker process overview

- 1 Using either the Docker command line interface (CLI) or REST APIs, the Docker client sends instructions or commands to the Docker host server.
- 2 The Docker host server, referred to as the host, includes the Docker daemon known as dockerd.
- 3 The daemon listens for Docker API requests or commands such as “docker run” and processes those commands.
- 4 The daemon builds, runs, and distributes containers to the registry
- 5 The registry stores the images.

Docker host



The Docker host also includes and manages:

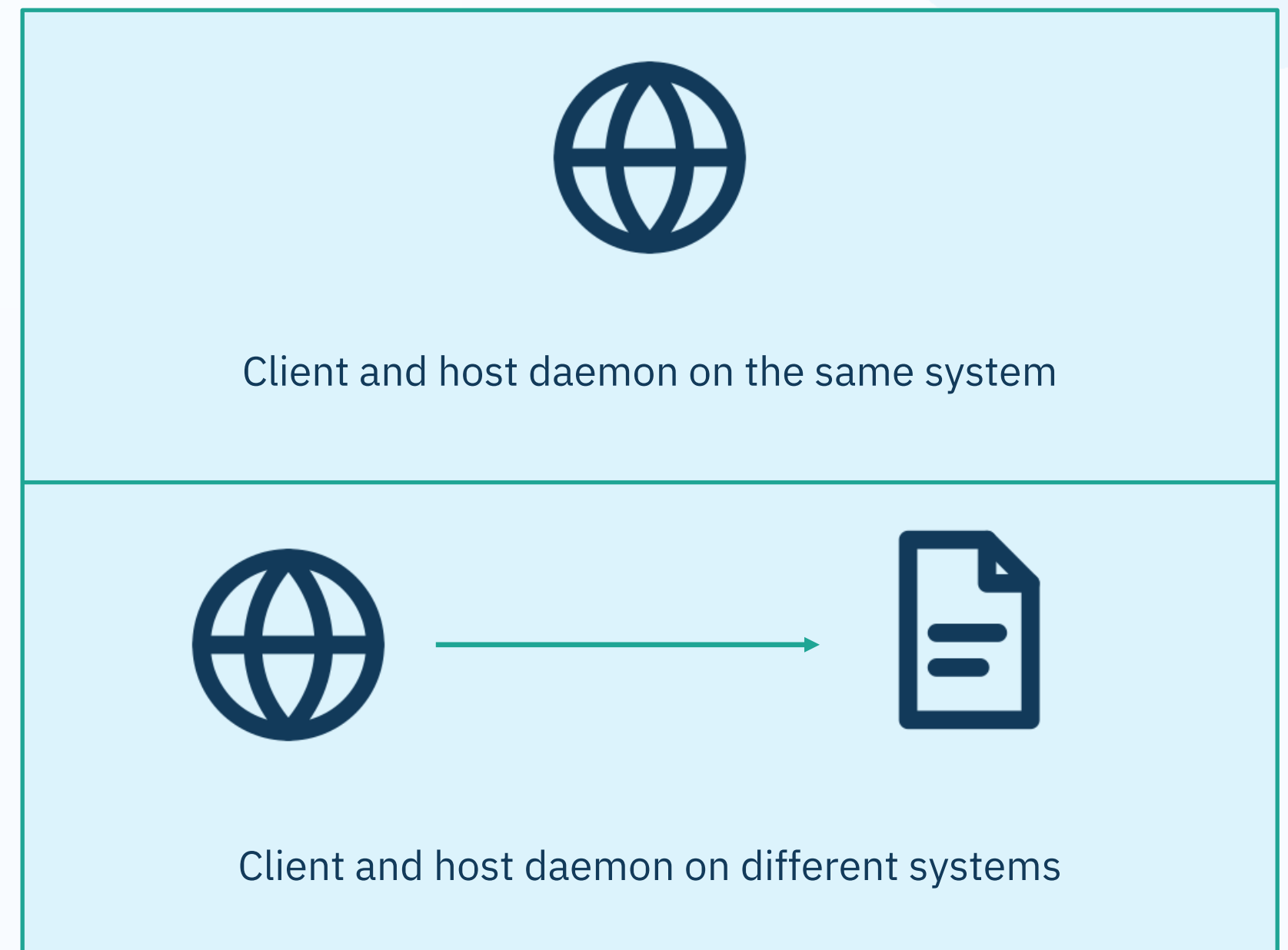


— CONTAINERS AND DOCKER

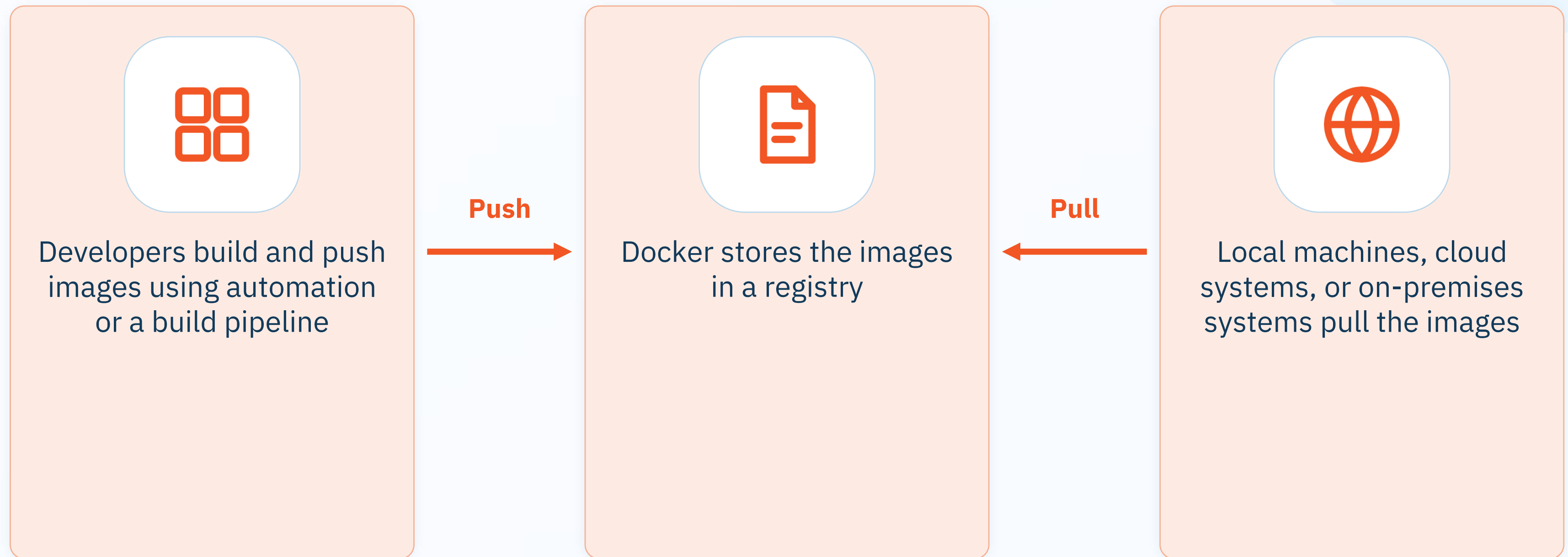
Docker communications



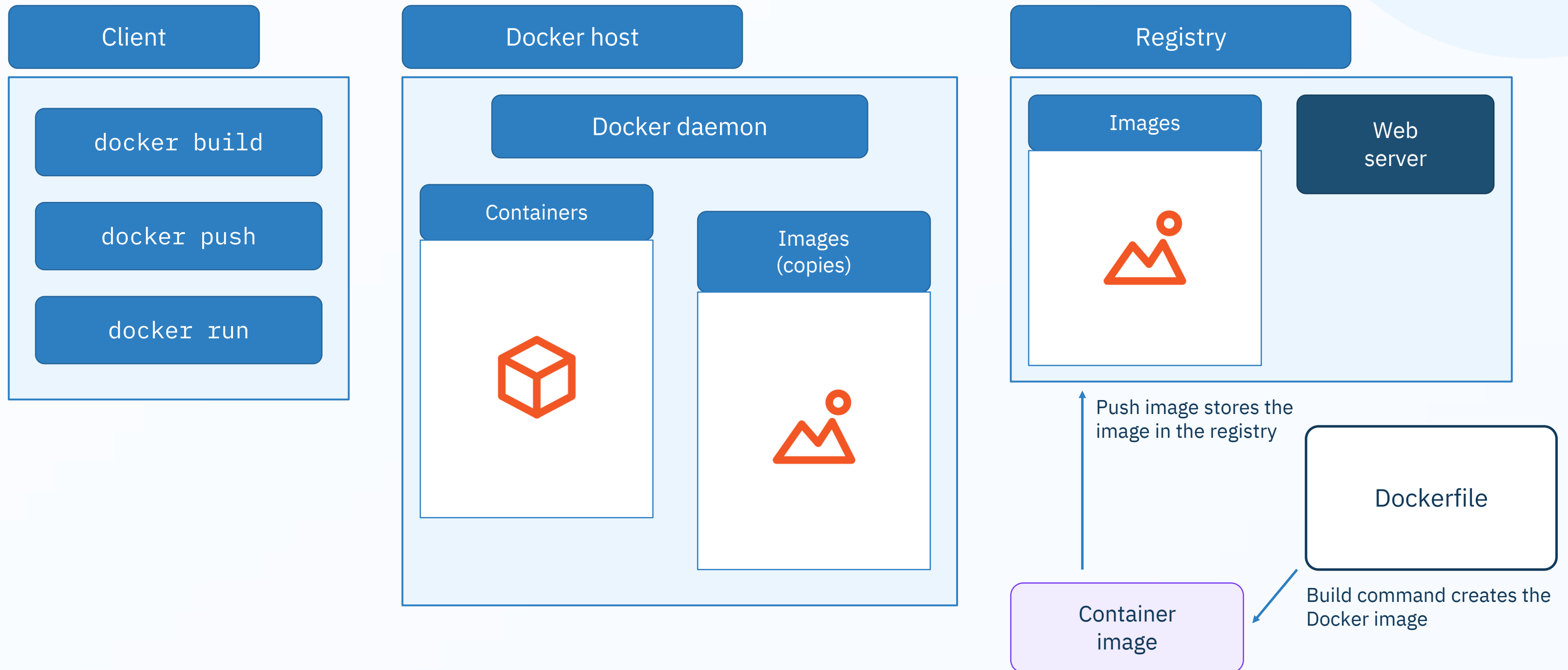
- ◆ The Docker client can communicate with both local and remote hosts.
- ◆ The Docker client and the host daemon can run:
 - On the same system
 - On different systems
- ◆ Docker daemons can also communicate with other daemons to manage Docker services.



Registry access



Containerization process

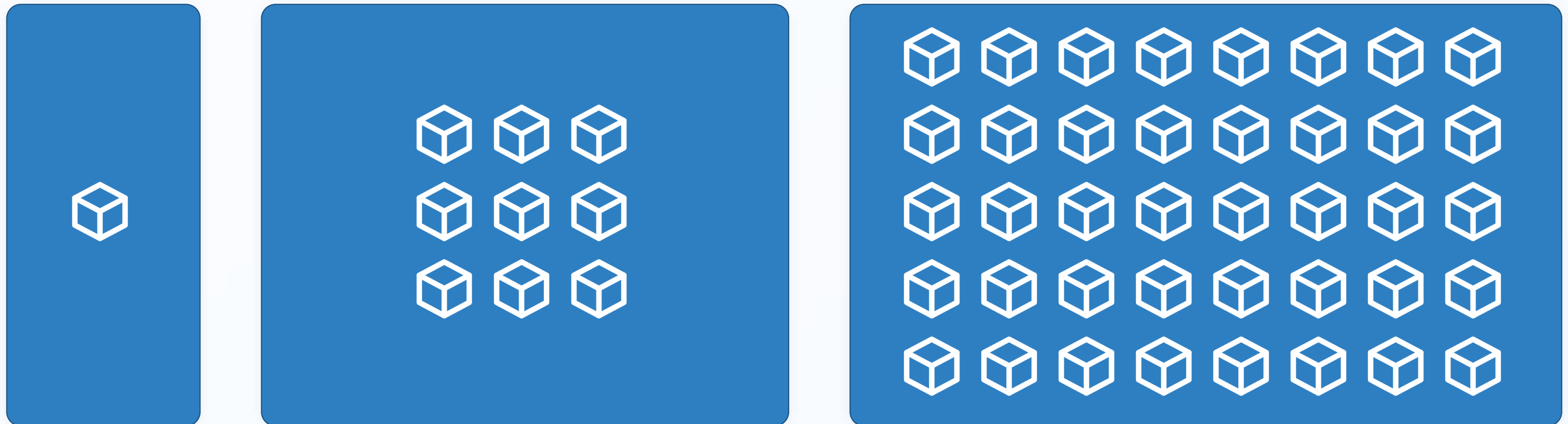


 PART 05

Container orchestration and Kubernetes

Container management challenge

How to manage these containers, especially in different VM or servers



Container orchestration

Container orchestration is a process that automates the container lifecycle of containerized applications.

Container Lifecycle:

- ◆ Deployment
- ◆ Management
- ◆ Scaling
- ◆ Networking
- ◆ Availability

How does Container orchestration work?

Configuration files

YAML or JSON files configure containers to:

- ◆ Find resources
- ◆ Establish a network
- ◆ Store logs

Deployment scheduling

- ◆ Automatically schedules new container deployment
- ◆ Finds the right host based on predefined settings or restrictions

Manages container lifecycle

Configuration file specs inform container decisions:

- ◆ System parameters
- ◆ File parameters

Scaling and productivity

Automation is used to:

- ◆ Support scaling
- ◆ Maintain productivity

Container orchestration tools

Marathon

- ◆ Open-source cluster manager
- ◆ Scales container infrastructure by automating management and task monitoring



Nomad

- ◆ Free, open-source cluster management and scheduling tool
- ◆ Supports various app types on all major operating systems



Docker Swarm

- ◆ Open-source container orchestration platform
- ◆ Automates deployment of containerized apps

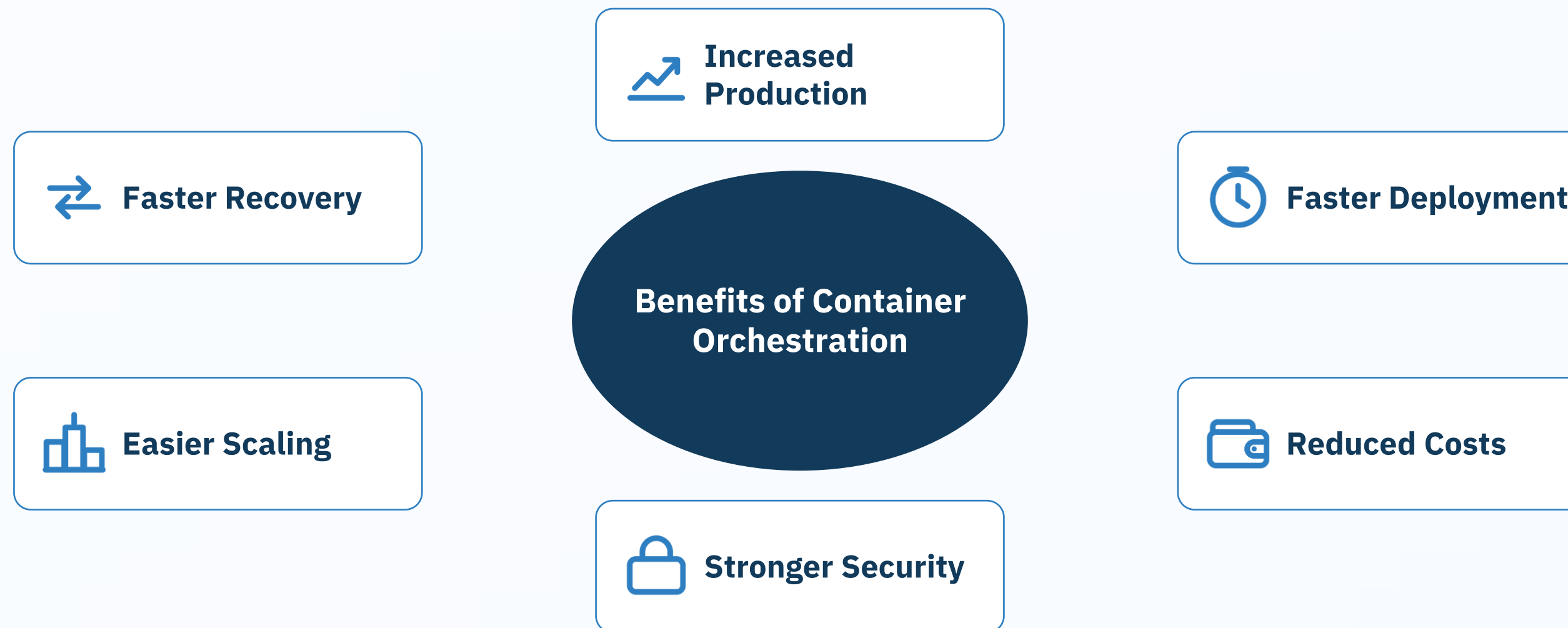


Kubernetes

- ◆ Standard for open-source container orchestration platforms
- ◆ Robust feature set, broadly supported

Container orchestration benefits

Container orchestration helps to meet business goals and increase profitability by using automation.



What is Kubernetes?

“

Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.”

Source: [Kubernetes.io](https://kubernetes.io)



What is Kubernetes?

- ◆ An open-source containerization orchestration platform
- ◆ Easily portable across clouds and on-premises
- ◆ Includes a growing ecosystem of projects, products and partners
- ◆ Facilitates declarative management





What Kubernetes is not

Is not a traditional, all-inclusive platform as a service (PaaS)

Is not rigid or opinionated but a flexible model that supports a diverse variety of workloads and containerized applications

Does not provide integration/continuous delivery (CI/CD) pipelines to deploy source code or build applications

Does not prescribe logging, monitoring, or alerting solutions

Does not provide built-in middleware, databases, or other services



Kubernetes concepts

Object/ concept	What you need to know
Pods and Workloads	Pods are the smallest deployable compute object in Kubernetes and the higher-level abstractions to run workloads
Services	A service exposes application running on a set of Pods
Storage	Kubernetes supports both persistent and temporary storage for Pods
Configuration	Resources that Kubernetes provides for configuring Pods
Security	Security for cloud-native workloads enforces security for Pod and API access



Kubernetes concepts

Object/ concept	What you need to know
Policies	Create policies for groups of resources helps ensure that Pods match to the Nodes so that the kubelet can find them and run the Pod
Schedule, Eviction	Runs and proactively terminates one or more Pods on resource-starved Nodes
Preemption	Terminates lower priority Pods so that higher priority Pods can run on Nodes
Administration	Management details necessary to administer a Kubernetes cluster



Kubernetes capabilities

Automated rollouts and rollbacks

- ◆ Progressively rolls out changes to application or configuration
- ◆ Monitors application health and ensures instances are running
- ◆ Rolls back changes

Storage orchestration

- ◆ Automatically mounts your chosen storage system whether from local storage, network storage, or public cloud

Horizontal scaling

- ◆ Scales loads automatically based on metrics or via commands



Kubernetes capabilities

Automated bin packing

- ◆ Increases resource utilization and cost savings using a mix of critical and best-effort workloads
- ◆ Performs container auto-placement based on resource requirements and conditions without sacrificing high availability (HA)

Secret and configuration management

- ◆ Stores and manages sensitive information (credentials, keys, or tokens) securely
- ◆ Deploys and updates secrets and configuration without rebuilding images

IPv4/IPv6 dual-stack

- ◆ Assigns both IPv4 and IPv6 addresses to Pods and Services



Kubernetes capabilities

Batch execution

- ◆ Manages batch and continuous integration workloads, and replaces failed containers, if configured

Self-healing

- ◆ Restarts, replaces, reschedules, and kills failing or unresponsive containers
- ◆ Exposes containers to clients only if healthy and running



Kubernetes capabilities

Service discovery and load balancing

- ◆ Discovers Pods using their own IP addresses or a single DNS name
- ◆ Load-balances traffic across Pods for better performance and high availability

Designed for extensibility

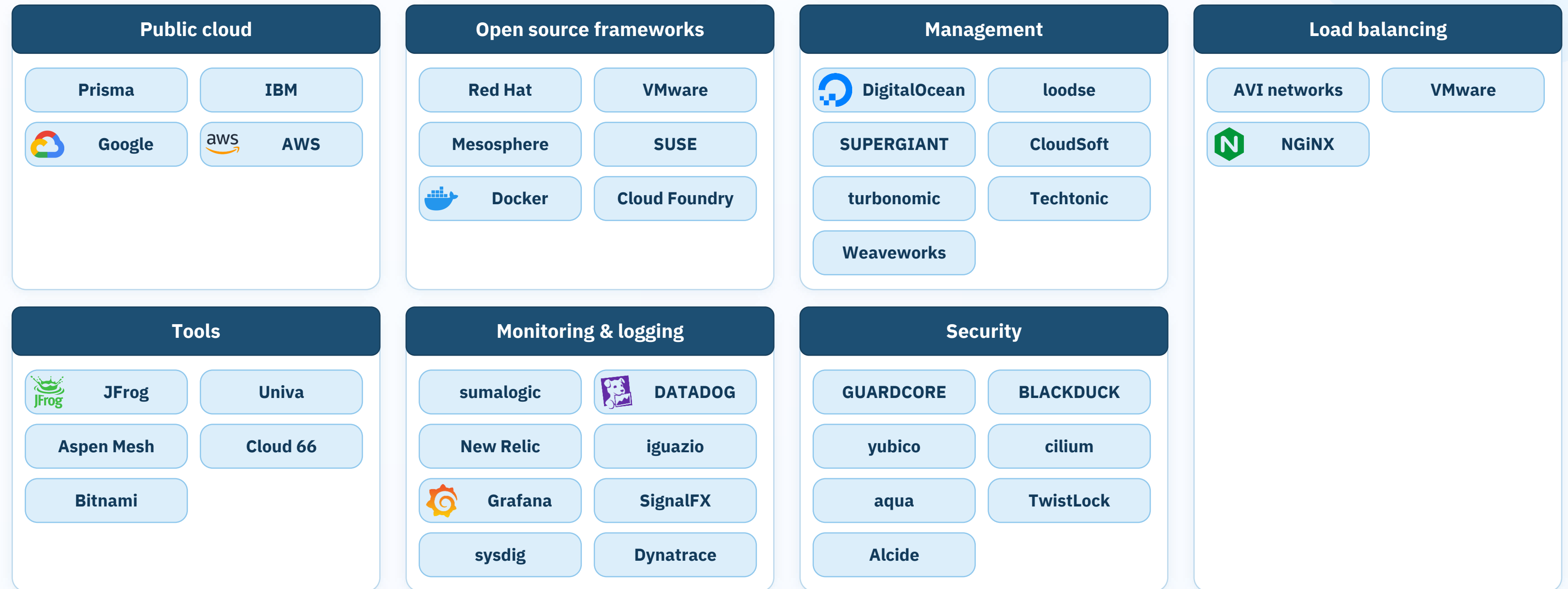
- ◆ Easily extensible by adding or providing additional features to your Kubernetes cluster without any source code modifications

Kubernetes ecosystem



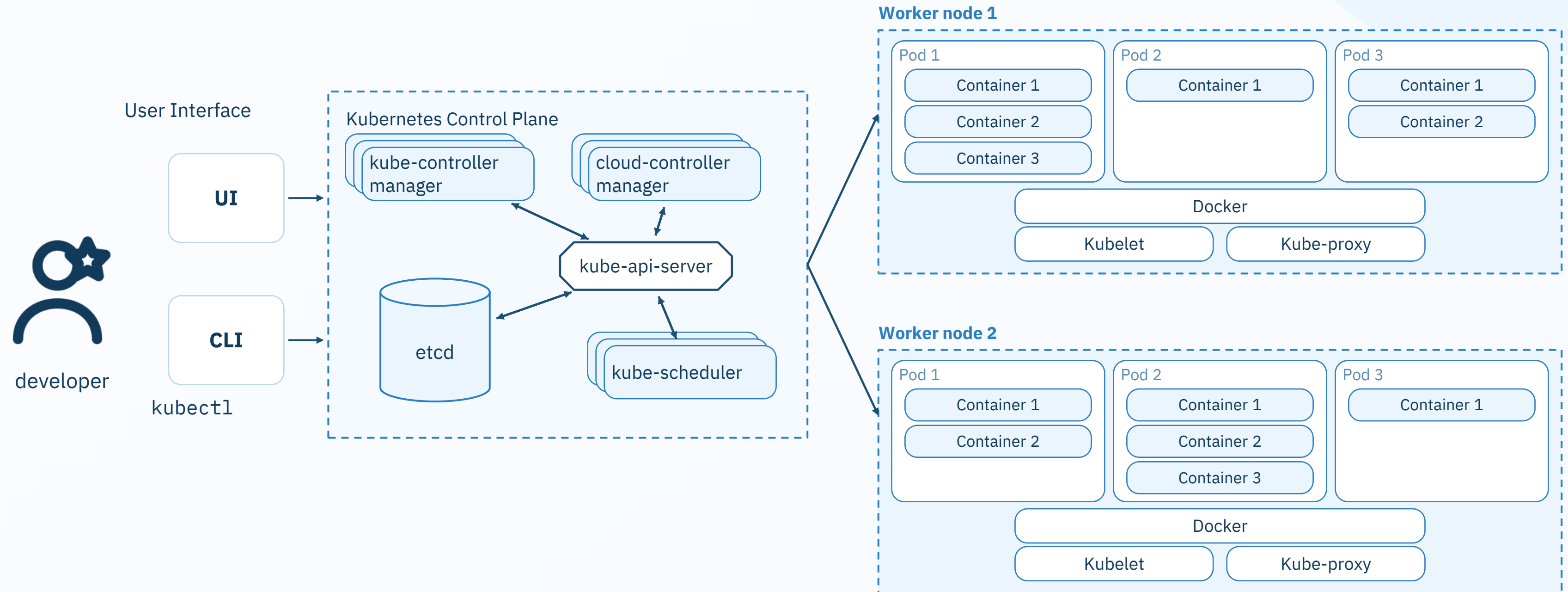
- ◆ Contains services, support, and tools that are widely available
- ◆ Provides additional Kubernetes services
 - Building container images
 - Storing images in a container registry
 - Application logging and monitoring
 - Continuous improvement and continuous delivery CI/CD

Kubernetes ecosystem





Kubernetes worker nodes



 PART 06

Infrastructure as Code

What is Infrastructure as Code?

- ◆ Describes your infrastructure in textual format for easy system configuration
- ◆ Textual code that you hand to an IaC tool to provision your infrastructure elements

Often use YAML format



Why use IaC?

- ◆ Reduces manual system and software configurations
- ◆ CMSs made this possible
- ◆ IaC makes it easy to rapidly provision the same platform repeatedly



Declarative vs. imperative approach

- ◆ In a declarative approach, you specify the desired state
- ◆ The tool determines how to get there



Terraform, Puppet, SaltStack, CloudFormation, and to some extent, Ansible

- ◆ In an imperative approach, you specify the order of execution
- ◆ The tool executes commands in that order



Tools like Chef are imperative and to some extent, Ansible can be as well.

Inventory files and IaC tools



It's also important to note that Ansible is idempotent. It will not affect a server if it is already in the desired state. This allows you to be able to reapply a playbook and not worry about it installing things twice. Ansible will take care not to do that if it isn't needed

Benefits of IaC

- ◆ Faster time to production
- ◆ Faster, more efficient development
- ◆ Protection against staff churn
- ◆ Lower costs and improved use of developers' time



Top IaC tools



Terraform



Ansible



Chef



Puppet



SaltStack

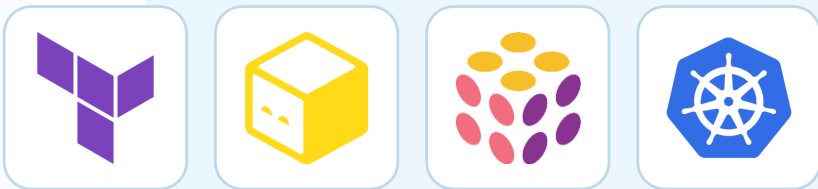
You can use Terraform as a base tool in combination with Ansible where Terraform provisions the base infrastructure and Ansible configures the software on top of it. It is a very popular tool for cloud provisioning. Ansible is an open source tool that automates IT tools such as intra-service orchestration, application deployment, cloud provisioning, and more. It uses familiar YAML files to describe the state that you want to achieve. Ansible is simple to set up because it doesn't require any client-side agents or bespoke security infrastructure, and it works simply delivering modules to the clients.

Glossary



Term	Definition	Where the term is introduced
Ansible	An open-source IaC tool that automates various IT tasks such as, configuration management, application deployment, and cloud provisioning.	Infrastructure as Code (IaC) Video
Circle CI	A CI/CD platform that enables the implementation of DevOps practices and supports Continuous Delivery.	Platform and Tools Video
Configuration Management Systems (CMSs)	Software tools or frameworks that enable organizations to manage and control the configuration of their systems and infrastructure in a consistent and efficient manner.	Infrastructure as Code (IaC) Video
Jenkins	A popular CI/CD software installed on a server to facilitate centralized build processes.	Platform and Tools Video
Travis CI	A hosted continuous integration service used to build and test software projects hosted on GitHub, Bitbucket, GitLab, Perforce, Apache Subversion and Assembla.	Platform and Tools Video
YAML	YAML stands for Yet Another Markup Language is a simple and expressive way to represent data structures and configurations in a plain-text format that is easy for both humans and machines to read and write.	Infrastructure as Code (IaC) Video

IAC vs Kubernetes



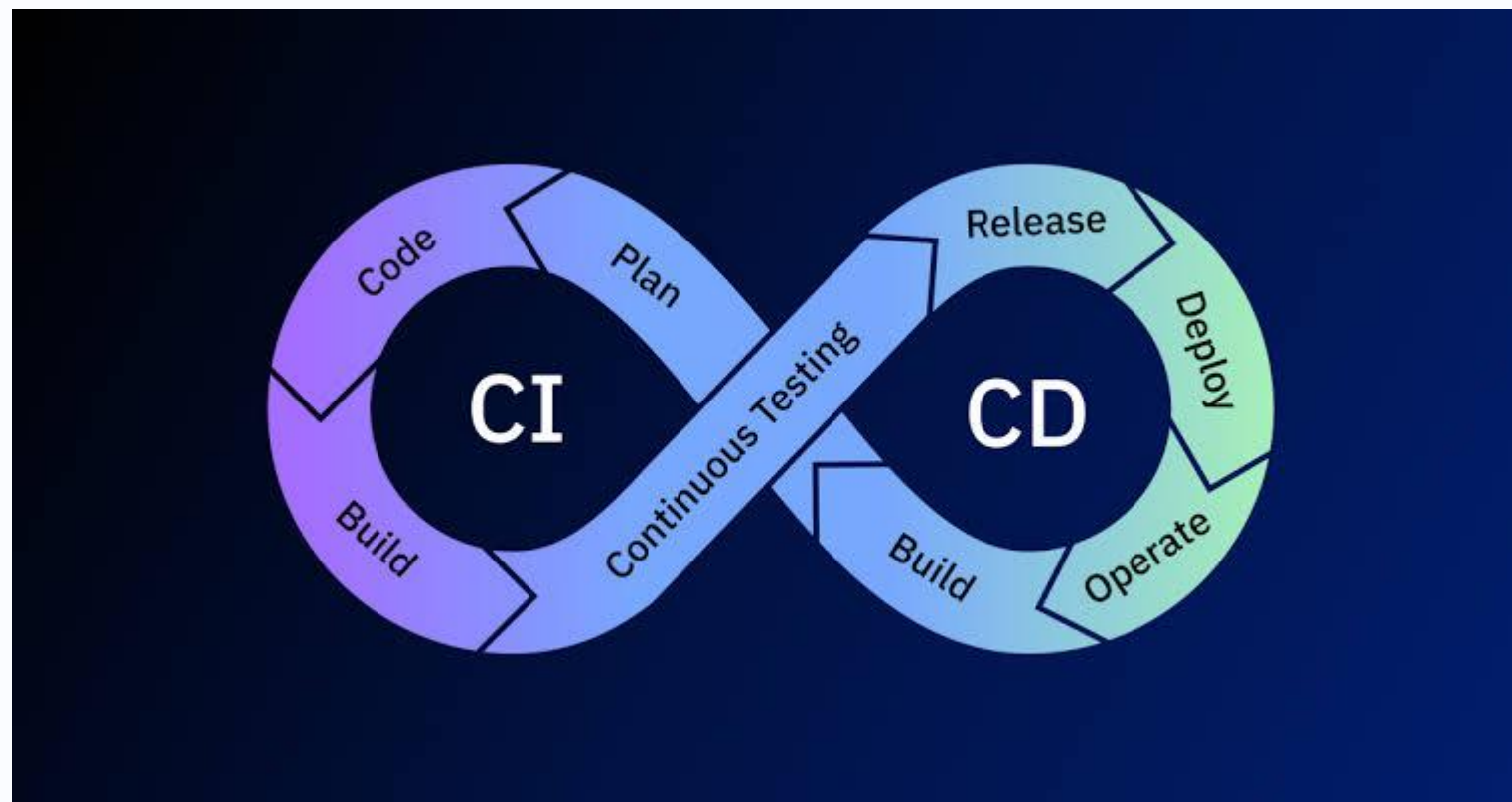
Direct Comparison

Feature	IaC Tools (e.g., Terraform, OpenTofu, Pulumi)	Kubernetes (K8s)
Primary Role	Provisioning & Setup	Container Orchestration
What it Manages	Virtual machines, VPC networks, databases, storage buckets, and the Kubernetes cluster itself.	Containers, application scaling, internal networking, and microservices.
Execution Model	Plan & Apply: You run a command to make changes, and the tool stops running once the task is complete.	Continuous Control Loop: It constantly monitors the cluster 24/7 to ensure the active state matches your desired state.
State Management	Relies on a local or remote state file to map your code to real-world resources.	Uses an internal etcd database to maintain the real-time status of the cluster.

 PART 07

CI/CD

Difference between CI and CD



<https://www.ibm.com/think/topics/ci-cd>.

- ◆ CI/CD is not one process—it's two distinct processes
- ◆ CI is continuously integrating code back to main/master/trunk branch
- ◆ CD is deploying that integrated code somewhere

What is Continuous Integration?

- ◆ Automation process that allows developers to integrate their work into a repository
- ◆ Helps to enable collaborative development across teams
- ◆ Helps to identify integration bugs sooner
- ◆ Teams can work in small chunks and regularly integrate new code into the main branch



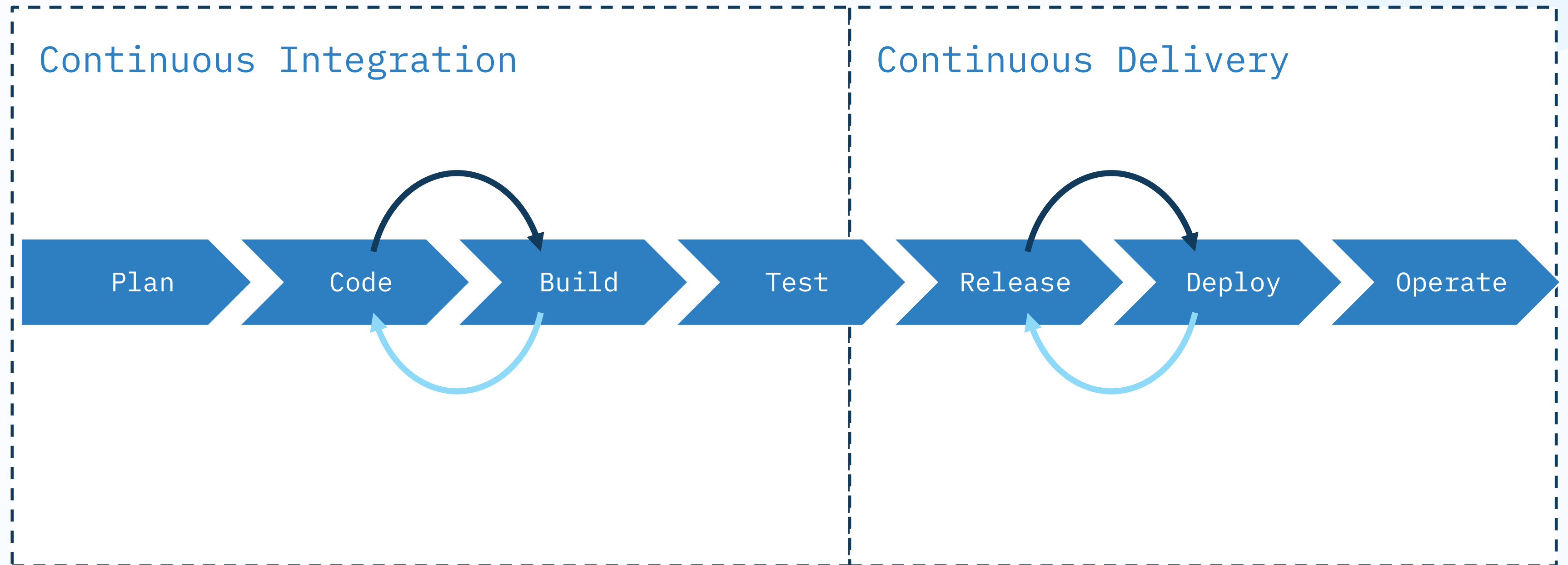
What is Continuous Delivery?

- ◆ Continuous Delivery comes after Continuous Integration
- ◆ Prepares the code for release
- ◆ Automates the steps that are needed to deploy a build

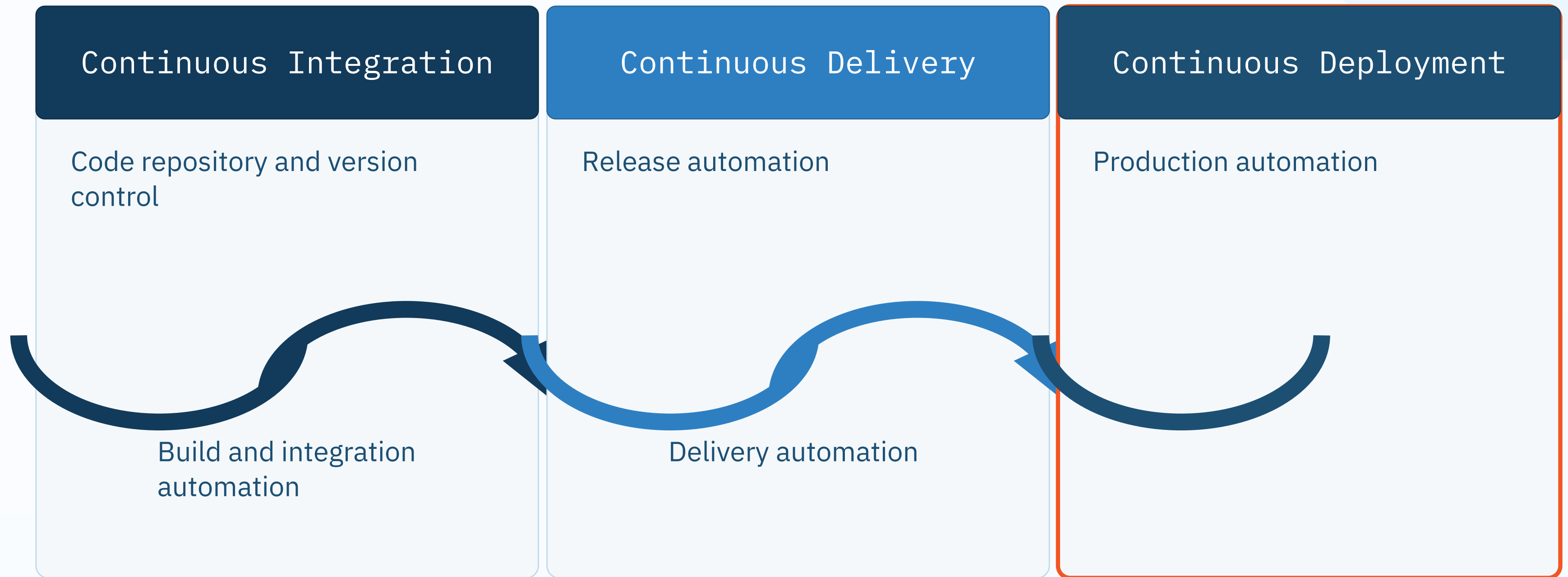


— CI/CD

CI/CD



Continuous Deployment



— CI/CD

Benefits of CI/CD

- ◆ Faster reaction times to changes
- ◆ Reduced code integration risk
- ◆ Higher code quality
- ◆ Code in version control works
- ◆ Less deployment time



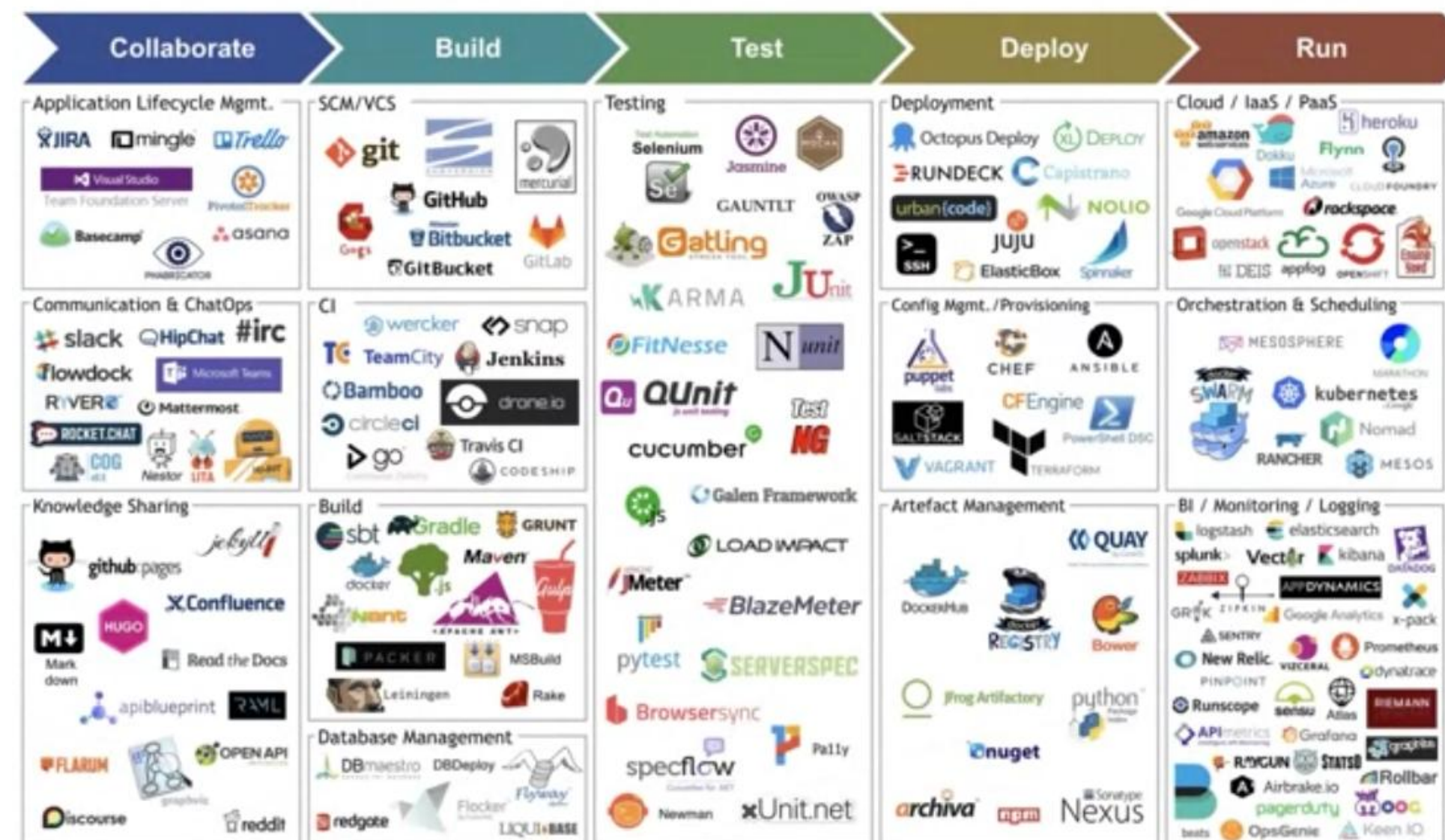
Pipelines can use many tools

	SCM	BUILD	CI	REPO	ENV	
LOB A Team 1	 GitLab	 Travis CI	 Travis CI	Nexus	 Google Cloud	IBM Cloud
LOB A Team 2	 GitHub	 Jenkins	 Jenkins	 JFrog Artifactory	 Amazon Web Services	
LOB C	 Bitbucket	Hudson	 Jenkins	Nexus	 OpenStack	 Microsoft Azure
LOB D	 Subversion	CruiseControl	CruiseControl	 JFrog Artifactory		

— CI/CD

Pipeline tools

There are lots of pipeline tools!



Overview of common CI/CD tools



Jenkins

CI/CD software installed on a server where the central build takes place



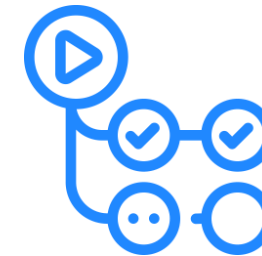
CircleCI

CI/CD platform that can be used to implement DevOps practices



Travis CI

Hosted CI service helps developers build and test software projects hosted on GitHub and Bitbucket



GitHub Actions

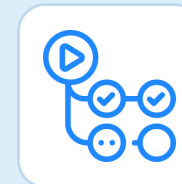
CI/CD platform to automate GitHub workflows

— CI/CD

Automated CI

- ◆ Uses webhooks to monitor events
- ◆ Ensures your code is doing what you want
- ◆ Streamlines development

Tools:



GitHub Actions



CircleCI



Jenkins CI



Travis CI

Faster code-change reaction time

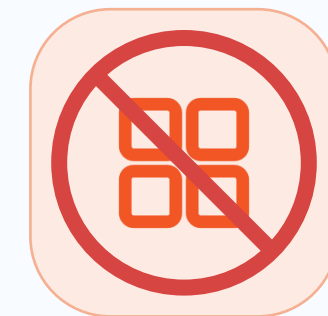
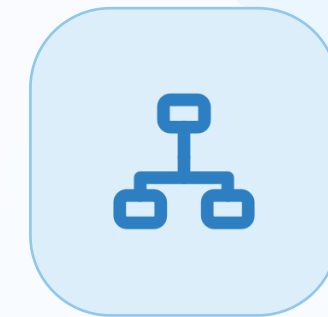
- 1 You make a change to your code
- 2 Code gets tested
- 3 Change gets built
- 4 Deliver solution quickly into customer's hands



— CI/CD

Reduced code integration risk

- ◆ Integrating smaller and smaller things
- ◆ No need to integrate 100,000 lines of code into your code-base
- ◆ Only need to integrate 10-50 lines of code



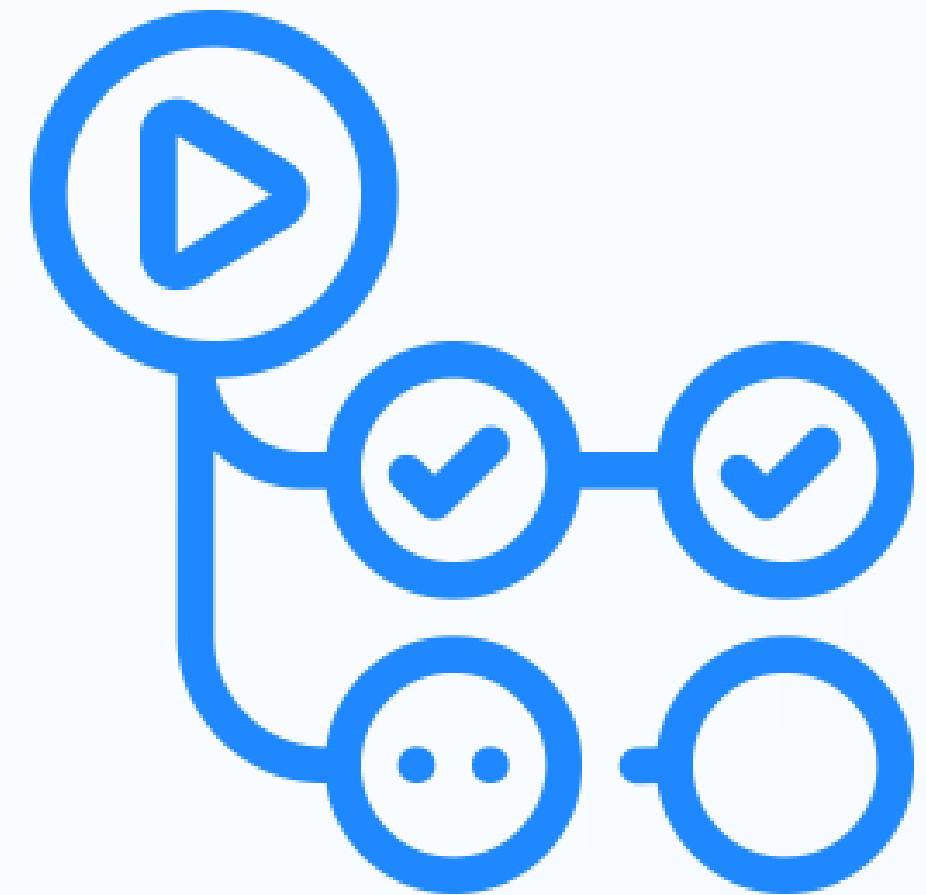
 PART 08

GitHub Actions

— GITHUB ACTIONS

GitHub Actions overview

- ◆ GitHub Actions is a CI/CD tool available on every repository
- ◆ Integrated into GitHub as a service
- ◆ It allows you to treat your CI pipeline as code
- ◆ It uses a `.github/workflows/` folder to store workflow definitions as `.yaml` files



— GITHUB ACTIONS

Marketplace



- ◆ GitHub Actions Marketplace hosts actions for workflows
- ◆ Large variety of actions available
- ◆ Jumpstarts your workflows

Actions

An entirely new way to automate your development workflow.

215 results for "python" filtered by Actions x

Actions



Setup Python

By actions

Set up a specific version of Python and add the command-line tools to the PATH

☆ 814 stars



python-yamale

By eXpire163

A schema and validator for YAML



Coveralls Python

By AndreMiras

Python coverage reports via coveralls.io

☆ 34 stars



Python Linter

By sunnysid3up

Ensure Python code quality. Supports pylint, black, mypy and isort

☆ 7 stars



Python Cov

By ororo

publish python coverage report to a PR & enforce coverage on new & modified files

☆ 5 stars

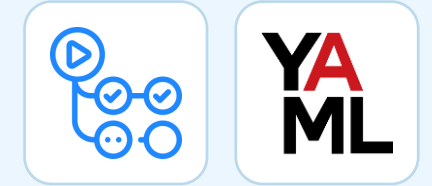


python-stdeb

By lpenz

Build a python 3 debian package using stdeb

GitHub Actions setup



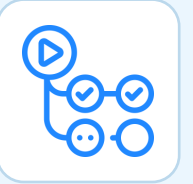
- ◆ No website needed for setup
- ◆ All you need is a workflows folder and a .yaml file
- ◆ No other configuration is needed
- ◆ Starter code is available

```
name: Python application

on:
  push:
  pull_request:
    branches: [ "master" ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python 3.10
        uses: actions/setup-python@v3
        with:
          python-version: "3.10"
      - name: Install dependencies
        run: |
          pip install -r requirements.txt
      - name: Test with PyUnit
        run: |
          unittest -m discover
```

Basic concepts



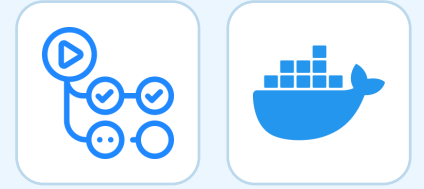
- ◆ Workflow is the basic concept in GitHub Actions
- ◆ Workflow is a series of automated procedures
- ◆ Every repository can have any number of workflows
- ◆ Each workflow has the following components:



— GITHUB ACTIONS

How does GitHub Actions work?

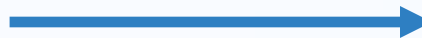
- ◆ You create a folder called `.github/workflows/` in the root of your project
- ◆ Place a `.yaml` file in that folder
- ◆ Depending on how you configure GitHub Actions
 - Whenever you push to master, a build is executed
 - Whenever you issue a pull request, tests are run
- ◆ Job runs in an isolated environment (VM or Docker container)



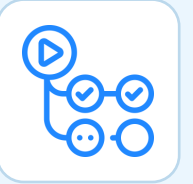
Initial setup



```
.github
├── workflows
│   ├── ci.yaml
│   ├── cd.yaml
│   └── publish.yaml
```



Place workflow .yaml files



Workflow composition

EVENT

Something that activates the execution of a workflow

JOB

Made up of steps that are performed on the same runner

RUNNER

A server that performs jobs on a specific platform or OS

STEPS

Tasks comprising one or more shell commands or actions

ACTIONS

Procedures that can be executed within a step

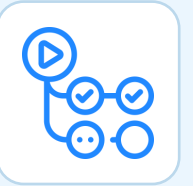
Event

Something that activates the execution of a workflow

branch_protection_rule
check_run
check_suite
create
delete
deployment
deployment_status
discussion
discussion_comment
fork
gollum
issue_comment
issues

label
milestone
page_build
project
project_card
project_column
public
pull_request
pull_request_comment
pull_request_review
pull_request_review_comment
pull_request_target

push
registry_package
release
repository_dispatch
schedule
status
watch
workflow_call
workflow_dispatch
workflow_run



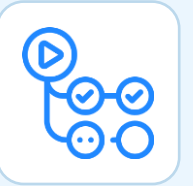
Event

Something that activates the execution of a workflow

branch_protection_rule
check_run
check_suite
create
delete
deployment
deployment_status
discussion
discussion_comment
fork
gollum
issue_comment
issues

label
milestone
page_build
project
project_card
project_column
public
pull_request
pull_request_comment
pull_request_review
pull_request_review_comment
pull_request_target

push
registry_package
release
repository_dispatch
schedule
status
watch
workflow_call
workflow_dispatch
workflow_run



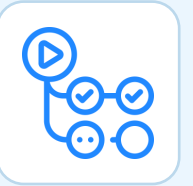
Event

Something that activates the execution of a workflow

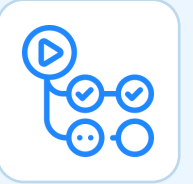
branch_protection_rule
check_run
check_suite
create
delete
deployment
deployment_status
discussion
discussion_comment
fork
gollum
issue_comment
issues

label
milestone
page_build
project
project_card
project_column
public
pull_request
pull_request_comment
pull_request_review
pull_request_review_comment
pull_request_target

push
registry_package
release
repository_dispatch
schedule
status
watch
workflow_call
workflow_dispatch
workflow_run



Event examples



```
on:
  pull_request:
    types: [opened, reopened]
    branches:
      - master
```

- ◆ Runs whenever a pull request is opened or reopened against the master branch

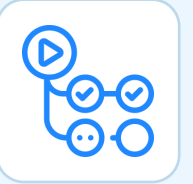
```
on:
  push:
    branches:
      - main
```

- ◆ Runs whenever a push is made to the main branch including from merging a pull request

```
on:
  release:
    types: [publish]
```

- ◆ Runs whenever a release is published

Jobs



◆ Each workflow can have multiple jobs

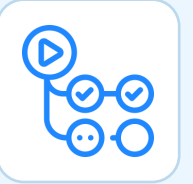
- Jobs are named whatever you want
- Jobs run in parallel unless dependencies specified using 'needs' keyword

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:

  publish:
    runs-on: ubuntu-latest
    needs: build
    steps:
```

The 'publish' job will only run after the 'build' job that it depends on has been completed successfully

Jobs



◆ Each workflow can have multiple jobs

- Jobs are named whatever you want
- Jobs run in parallel unless dependencies specified using 'needs' keyword

◆ Each job contains:

- Runner
- Services (optional)
- Steps

```
jobs:
  build:
    runs-on: ubuntu-latest
    steps:

  publish:
    runs-on: ubuntu-latest
    needs: build
    steps:
```

Runners



Environment	YAML Label
Ubuntu 22.04 beta	ubuntu-22.04
Ubuntu 20.04	ubuntu-latest or ubuntu-20.04
Ubuntu 18.04	ubuntu-18.04
macOS 12	macos-12
macOS 11	macos-latest or macos-11
macOS 10.15	macos-10.15
Windows Server 2022	windows-latest or windows-2022
Windows Server 2019	windows-2019
Windows Server 2016	windows-2016

Runner examples



```
jobs:
  build:
    runs-on: ubuntu-latest
```

- ◆ This job runs on the latest version of Ubuntu

```
jobs:
  build:
    runs-on: ubuntu-22.04
```

- ◆ This job runs on Ubuntu 22.04

```
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
```

- ◆ This job runs inside a Python 3.9-slim Docker container (Debian 11) on top of the latest Ubuntu distribution

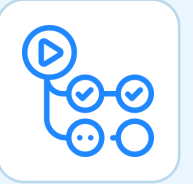
Services



- ◆ Services are defined as Docker containers
 - Databases, message queues, and more
- ◆ The name you use becomes the DNS name used to access it
- ◆ The service in this example is named postgres

```
# Required services
services:
  # Name used to access the service
  postgres:
    image: postgres
    env:
      POSTGRES_PASSWORD: pgs3cr3t
      POSTGRES_DB: testdb
    ports:
      - 5432:5432
  # Set health checks
  options: >-
    --health-cmd pg_isready
    --health-interval 10s
    --health-timeout 5s
    --health-retries 5
```

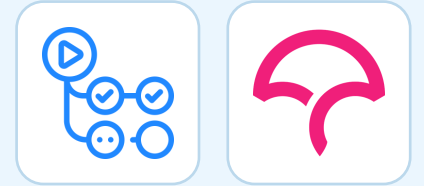
Steps



- ◆ Steps can have an optional name (name:) displayed in the report
- ◆ Steps can have an optional ID (id:) to refer to other steps
- ◆ Steps have either an action (uses:) or shell commands (run:)
- ◆ Steps can have environment variables (env:)

```
steps:  
  - name: Run unit tests with nose  
    id: testing  
    run: nosetests  
    env:  
      DATABASE_URI: "redis://redis:6379"  
      FLASK_APP: service:app
```

Actions



- ◆ Actions require the (uses:) keyword
- ◆ Actions can have arguments by specifying (with:) keyword followed by name-value pairs

```
steps:  
  - name: Upload code coverage  
    uses: codecov/codecov-action@v3  
    with:  
      version: "v0.1.13"
```

Actions Marketplace



The screenshot shows the GitHub Actions Marketplace interface. At the top, there's a navigation bar with the GitHub logo, a search bar, and links for Pull requests, Issues, Marketplace, and Explore. Below the navigation bar, the page is titled 'Marketplace / Search results'. On the left side, there's a sidebar with 'Types' (Apps, Actions) and 'Categories' (API management, Chat, Code quality, Code review, Continuous integration, Dependency management, Deployment, IDEs, Learning). The main content area shows search results for 'python'. A red box highlights the search bar containing 'python' and the 'Sort: Best Match' dropdown. Below this, the 'Actions' section is highlighted with a red box, showing a list of actions: 'Setup Python' (by actions), 'python-yamale' (by eXpire163), 'Coveralls Python' (by AndreMiras), 'Python Linter' (by sunnysid3up), 'python-isort' (by isort), and 'python-stdeb' (by lpenz). Each action card includes a play button icon, the action name, the creator, a brief description, and the number of stars.

Marketplace / Search results

Search or jump to... Pull requests Issues Marketplace Explore

Types

Apps

Actions

Categories

API management

Chat

Code quality

Code review

Continuous integration

Dependency management

Deployment

IDEs

Learning

Search: python

Sort: Best Match

Actions

An entirely new way to automate your development workflow.

182 results for "python" filtered by Actions

Actions

- Setup Python**
By actions
Set up a specific version of Python and add the command-line tools to the PATH
☆ 669 stars
- python-yamale**
By eXpire163
A schema and validator for YAML
- Coveralls Python**
By AndreMiras
Python coverage reports via coveralls.io
☆ 34 stars
- Python Linter**
By sunnysid3up
Ensure Python code quality. Supports pylint, black, mypy and isort
☆ 7 stars
- python-isort**
By isort
Run isort on a Python project
- python-stdeb**
By lpenz
Build a python 3 debian package using

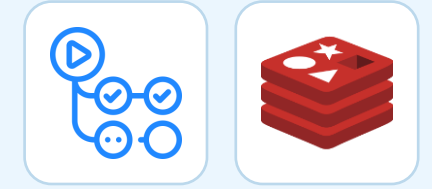
Workflow example



```
name: CI Build
on:
  pull_request:
    branches: [master]
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
    services:
      redis:
        image: redis:6-alpine
```

```
steps:
  - uses: actions/checkout@v2
  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip wheel
      pip install -r requirements.txt
  - name: Run unit tests with nose
    run: nosetests
    env:
      DATABASE_URI: "redis://redis:6379"
  - name: Upload code coverage
    uses: codecov/codecov-action@v2
    with:
      version: "v0.1.13"
```

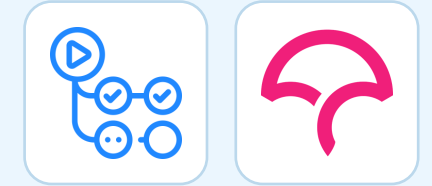
Workflow example – part 1



```
name: CI Build
on:
  pull_request:
    branches: [master]
jobs:
  build:
    runs-on: ubuntu-latest
    container: python:3.9-slim
    services:
      redis:
        image: redis:6-alpine
```

Creates a Redis service

Workflow example – part 2



```
steps:
  - uses: actions/checkout@v2
  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip wheel
      pip install -r requirements.txt
  - name: Run unit tests with nose
    run: nosetests
    env:
      DATABASE_URI: "redis://redis:6379"
  - name: Upload code coverage
    uses: codecov/codecov-action@v2
    with:
      version: "v0.1.13"
```

Uploads code coverage

 PART 09

Code review & code analysis tool



Linters

Linters:

- ◆ Focus on identifying code style issues and potential errors.
- ◆ Enforce coding conventions and flag deviations from best practices.



Flake8 (for Python):

- ◆ Linter for Python code.
- ◆ Checks for adherence to PEP 8 style guide and detects syntax errors and code smells.



ESLint (for JavaScript):

- ◆ Popular linter for JavaScript.
- ◆ Enforces coding standards, detects errors, and highlights potential issues.
- ◆ Offers configurable rules and integrates with build systems and IDEs.

StyleCop (for C#):

- ◆ Linter for C# code.
- ◆ Enforces coding style conventions and identifies violations.
- ◆ Helps maintain consistent coding practices within C# projects.



Code Quality Metrics

Code Quality Metrics:

- ◆ Automated tools measure metrics like cyclomatic complexity, code duplication, and code coverage.
- ◆ Metrics offer insights into maintainability, readability, and testability of code.

Sigrid:

- ◆ Tool for analyzing code quality metrics.
- ◆ Measures factors like cyclomatic complexity and code duplication.
- ◆ Provides insights to improve code maintainability and testability.



SonarCloud:

- ◆ Platform for analyzing code quality metrics.
- ◆ Evaluates factors such as code duplication and test coverage.
- ◆ Helps teams prioritize improvements for enhanced code quality.



Code Climate:

- ◆ Analyzes code quality metrics including duplication, complexity, and test coverage.
- ◆ Offers insights into code maintainability.
- ◆ Assists teams in prioritizing improvements for overall code quality enhancement.



Security Scanners

Security Scanners:

- ◆ Identify vulnerabilities and security weaknesses in code.
- ◆ Focus on issues like injection flaws, XSS vulnerabilities, and insecure configurations.



Snyk:

- ◆ Security scanner tool for identifying vulnerabilities.
- ◆ Helps address security risks before deployment.
- ◆ Provides insights into vulnerable dependencies and insecure configurations.

WhiteSource Bolt:

- ◆ Identifies vulnerabilities and security weaknesses.
- ◆ Assists developers in securing their codebases before release.

Black Duck by Synopsys:

- ◆ Helps mitigate security risks by identifying potential issues.
- ◆ Provides actionable insights for improving code security.



OWASP Dependency-Check:

- ◆ Security scanner tool targeting known vulnerabilities in project dependencies.
- ◆ Scans dependencies for outdated or vulnerable components.
- ◆ Offers actionable insights to mitigate security risks.

Codecov



◆ Codecov:

- Integrates with version control systems like GitHub and GitLab.
- Provides code coverage reports for automated tests.
- Helps track code coverage trends and identify areas needing additional testing.



Custom Rules and Configurations

Custom Rules and Configurations:

- ◆ Code analysis tools offer the flexibility to define custom rules and configurations.
- ◆ Tailors analysis to project-specific requirements and coding standards.



PHP_CodeSniffer (for PHP):

- ◆ Allows customization of coding standards and rules for PHP projects.
- ◆ Ensures adherence to specific coding guidelines and practices.
- ◆ Enables consistent code quality across PHP codebases.



TSLint (for TypeScript):

- ◆ Provides options for defining custom rules and configurations.
- ◆ Ensures TypeScript code adheres to project-specific standards and practices.
- ◆ Facilitates maintaining consistency and quality in TypeScript projects.



Checkstyle (for Java):

- ◆ Customizable static code analysis tool for enforcing Java coding standards.
- ◆ Enables developers to define custom rules and configurations.
- ◆ Helps ensure adherence to best practices and consistency across Java projects.



Integration of Tools into the Development Process



Documentation Tools Integration:

- ◆ Tools like Sphinx, Doxygen, or MkDocs assist in project documentation.
- ◆ Integration with version control ensures versioning and updating alongside code changes.
- ◆ Automated processes generate and publish documentation as part of CI/CD pipelines, keeping it current and accessible.



Alerting and Notification Integration:

- ◆ Systems such as Slack, email, or SMS offer real-time feedback on critical events.
- ◆ Integration with CI/CD tools provides immediate alerts for build failures or vulnerabilities.
- ◆ Developers receive alerts to address issues promptly, preventing escalation.

 PART 10

CD: continuous delivery

— CD: CONTINUOUS DELIVERY

So what is Continuous Delivery?

A series of practices designed to ensure that code can be swiftly and safely deployed to production by delivering every change to a production-like environment



Goals of Continuous Delivery

“

Continuous Delivery is a software development discipline where you build software in such a way that the software can be released to production at any time.”

— Martin Fowler

— CD: CONTINUOUS DELIVERY

Goals of Continuous Delivery

- ◆ Main or master branch must always be deployable
- ◆ Requires checks to ensure no bad code gets into main branch and breaks your product
- ◆ Continuous Integration runs tests on every pull request
- ◆ Ensures things work before merging back to main branch



— CD: CONTINUOUS DELIVERY

Benefits of Continuous Delivery

- ◆ Automate software transportation through SDLC stages
- ◆ Reduce deployment time
- ◆ Reduce costs
- ◆ Scale based on project size
- ◆ Deploy code automatically into each stage of SDLC



— CD: CONTINUOUS DELIVERY

Build in quality

- ◆ Ensure quality built in at every step
- ◆ Continually review your code
- ◆ Ensure everything looks good along the pipeline



— CD: CONTINUOUS DELIVERY

Work in small batches

- ◆ Make your user stories small
- ◆ Continuously integrate changes



— CD: CONTINUOUS DELIVERY

Best practices

- ◆ Make every change releasable
- ◆ Embrace trunk-based development
- ◆ Deliver through an automated pipeline



— CD: CONTINUOUS DELIVERY

More best practices

- ◆ Automate as many processes as possible
- ◆ Eliminate downtime
- ◆ Release at the granularity of test



Overview of CD tools



Jenkins – popular, supports many plugins, but not ideal for CD



Spinnaker – dedicated cloud-agnostic CD tool, native support for load balancers and scaling clusters, but is not a build tool



Concourse – mainly a CI tool with containers in mind, but can run in VMs too



GitLab – implements CI and CD, easy to automate, also an SCM tool, supports major cloud platforms

Overview of CD tools



Travis CI – CI tool that had CD capabilities, but isn't feature-rich



Tekton – can build, test, and deploy to Kubernetes, offers modularity



Go CD – easy pipeline setup, native Docker and Kubernetes support, build pipelines with YAML, JSON, or GUI



Argo CD – well-designed UI, easy to use, integrates with various CI tools

What to look for

- ◆ Feature richness
- ◆ Compatibility
- ◆ Ease of use/setup
- ◆ Maintenance effort



— CD: CONTINUOUS DELIVERY

Tasks in a CD pipeline

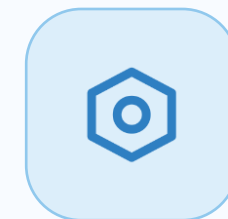
- ◆ Security scanning
- ◆ Vulnerability scanning
- ◆ Secret scanning
- ◆ Static Application Security Testing (SAST)
- ◆ Dynamic Application Security Testing (DAST)
- ◆ Deployment



— CD: CONTINUOUS DELIVERY

Argo CD overview

- ◆ Declarative Continuous Delivery tool
- ◆ Follows GitOps
- ◆ Implemented as Kubernetes controller



Tekton overview

- ◆ A flexible, open source framework
- ◆ Standardizes tooling and processes
- ◆ Made for reusability



 PART 11

Tekton

What is Tekton?

- ◆ Tekton is a flexible, open source framework for creating CI/CD pipelines
- ◆ Allows developers to build, test, and deploy applications
- ◆ Works across both cloud providers and on-premises systems



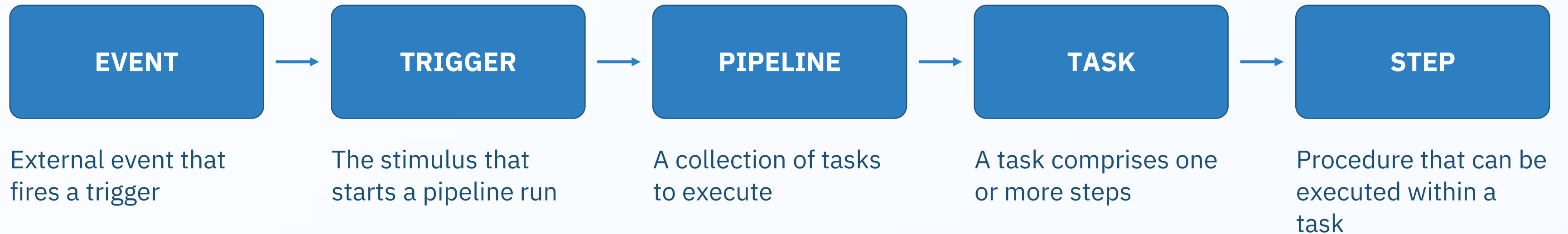
Benefits of Tekton

- ◆ Standardization across vendors, languages, and deployment environments
- ◆ Built-in best practices, cloud native solution
- ◆ Maximum flexibility and customization



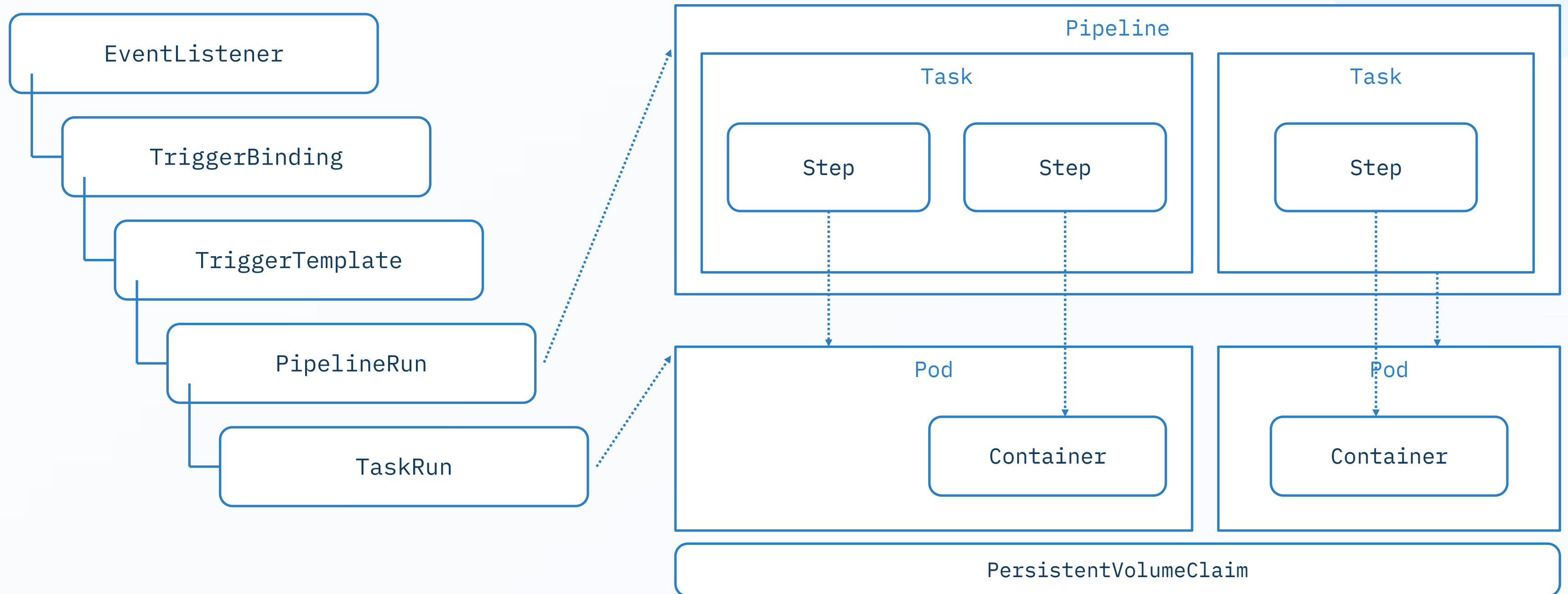


Conceptual building blocks





Physical building blocks



Complete pipeline

- ◆ Check out the code
- ◆ Run quality checks
- ◆ Run unit tests
- ◆ Build container image
- ◆ Deploy to an environment

← Task

← Task

← Task

← Task

← Task

} Pipeline





Checkout task

```
apiVersion: tekton.dev/v1beta1
kind: Task
metadata:
  name: checkout
spec:
  params:
    - name: repo-url
      description: The URL of the git repo to clone
      type: string
  steps:
    - name: checkout
      image: bitnami/git:latest
      command: [git]
      args: ["clone", "$(params.repo-url)"]
```

Pipeline



```
apiVersion: tekton.dev/v1beta1
kind: Pipeline
metadata:
  name: pipeline
spec:
  params:
    - name: repo-url
  tasks:
    - name: clone
      taskRef:
        name: checkout
      params:
        - name: repo-url
          value: "$(params.repo-url)"
```



Run the pipeline

```
$ kubectl apply -f tasks.yaml
task.tekton.dev/checkout created

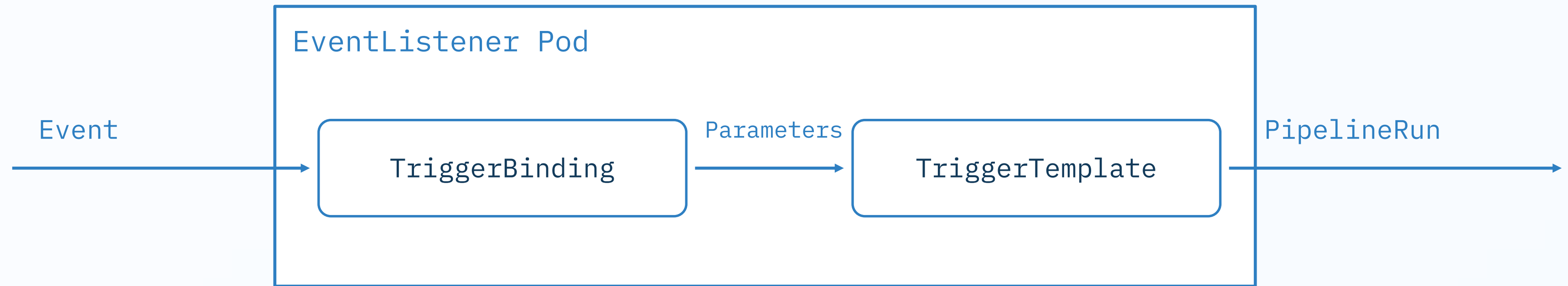
$ kubectl apply -f pipeline.yaml
pipeline.tekton.dev/pipeline created

$ kubectl get pipelines
NAME          AGE
pipeline      1m

$ tkn pipeline start pipeline --showlog \
    -p repo-url="https://github.com/rofrano/tekton-testing.git"
Pipelinerun started: pipeline-run-txg84
Waiting for logs to be available...
```



Tekton triggers flow



EventListener



```
apiVersion: triggers.tekton.dev/v1beta1
kind: EventListener
metadata:
  name: cd-listener
spec:
  serviceAccountName: pipeline
  triggers:
    - binding:
        name: cd-binding
      template:
        name: cd-template
```

TriggerBinding



```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerBinding
metadata:
  name: cd-binding
spec:
  params:
    - name: repository
      value: "$(body.repository.url)"
    - name: branch
      value: "$(body.ref)"
```

TriggerTemplate



```
apiVersion: triggers.tekton.dev/v1beta1
kind: TriggerTemplate
metadata:
  name: cd-template
spec:
  params:
    - name: repository
      description: The URL of the Git repo
      default: ""
    - name: branch
      description: The branch to process
      default: "master"
  resourcetemplates:
    . . .
```

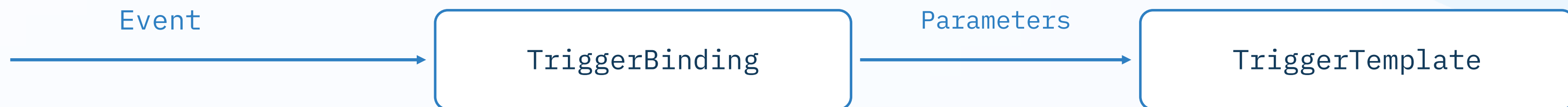
TriggerTemplate



```
. . .
resourcetemplates:
- apiVersion: tekton.dev/v1beta1
  kind: PipelineRun
  metadata:
    generateName: cd-pipeline-run-
  spec:
    serviceAccountName: pipeline
    pipelineRef:
      name: cd-pipeline
    params:
      - name: repo-url
        value: $(tt.params.repository)
      - name: branch
        value: $(tt.params.branch)
```



Parameter flow



```

'event': {
  'repository': {
    'url': 'github.com'
  }
  'ref': 'ade5f34eb78'
}
  
```

```

params:
  - name: repository
    value: "$(body.repository.url)"
  - name: branch
    value: "$(body.ref)"
  
```

```

params:
  - name: repo-url
    value: "$(tt.params.repository)"
  - name: branch
    value: "$(tt.params.branch)"
  
```

— TEKTON

Tekton catalog

- ◆ Tekton catalog or Tekton Hub
- ◆ Repository of Tekton tasks
- ◆ Contributed by the community
- ◆ Reusable parts to build a CI/CD pipeline
- ◆ Located at hub.tekton.dev



Categories of tasks

- ◆ Automation
- ◆ Build tools
- ◆ CLI
- ◆ Cloud
- ◆ Code quality
- ◆ Continuous Integration
- ◆ Deployment
- ◆ Developer tools
- ◆ Git
- ◆ Image build
- ◆ Integration & delivery
- ◆ Kubernetes
- ◆ Messaging
- ◆ Monitoring
- ◆ Networking
- ◆ OpenShift
- ◆ Publishing
- ◆ Security
- ◆ Storage
- ◆ Testing

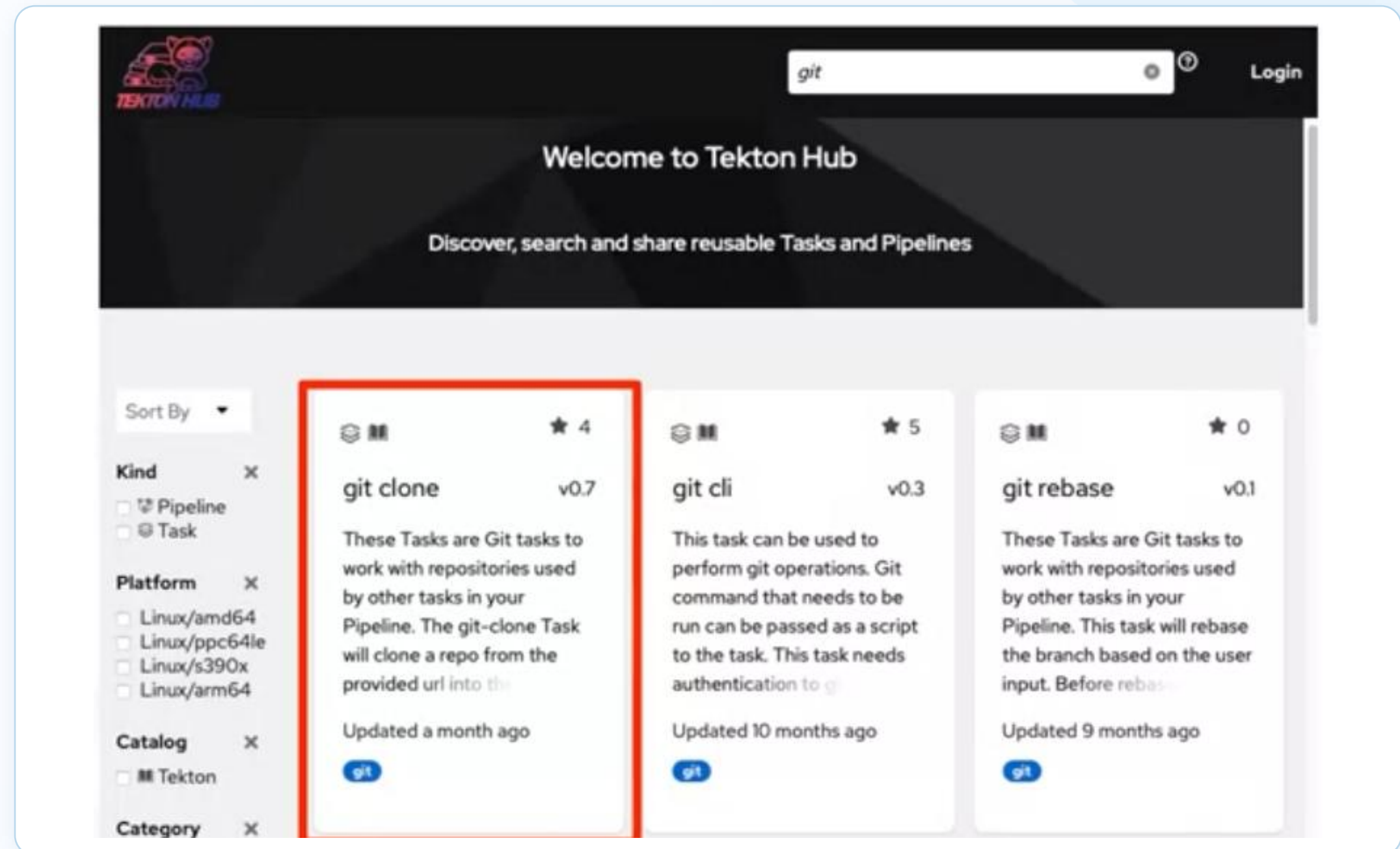


— TEKTON

Search for "git"



- ◆ hub.tekton.dev
- ◆ Search: "git"
- ◆ Find: git clone





Example git-clone instructions

Description

YAML

git-clone

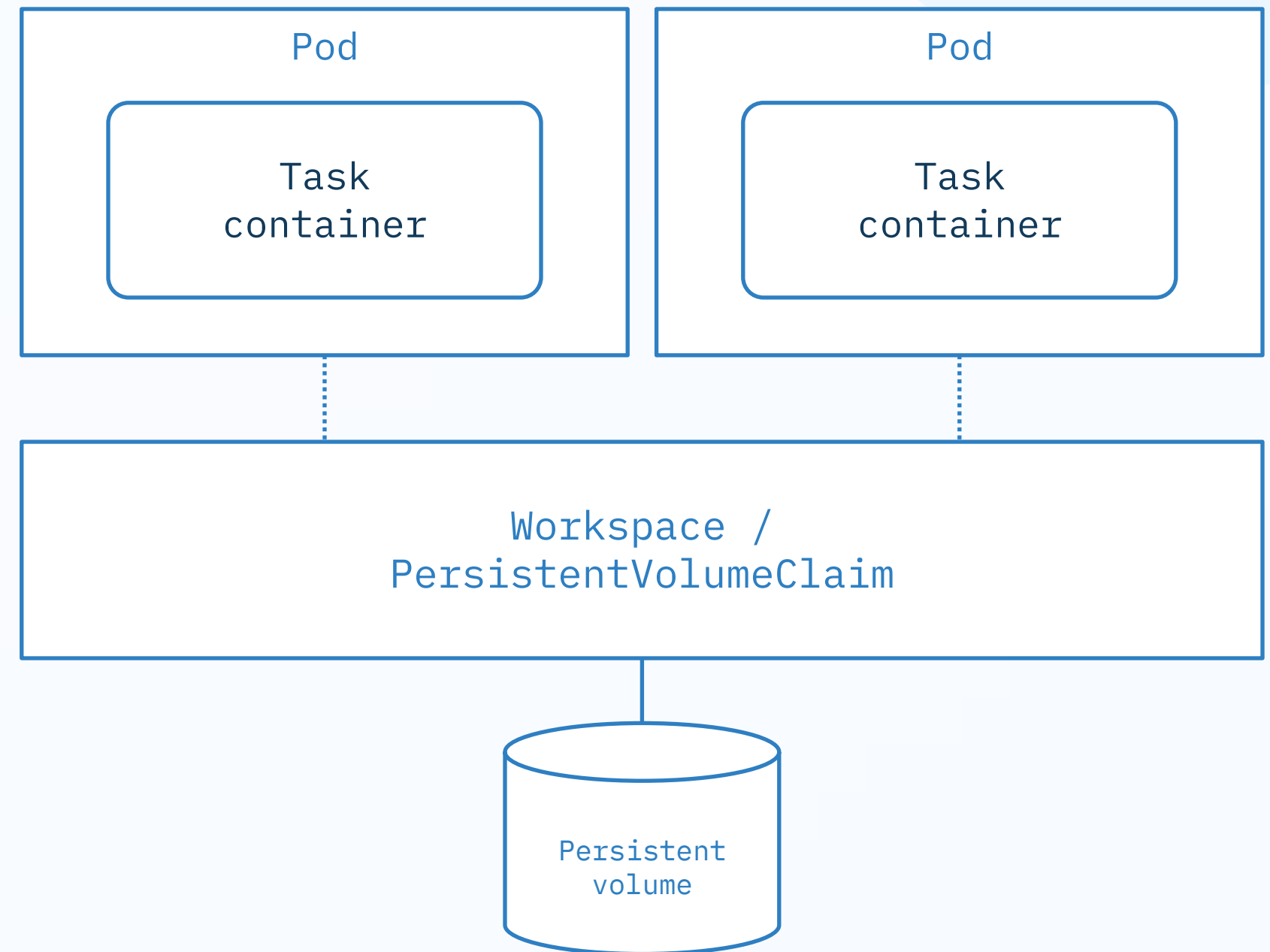
Please Note: this Task is only compatible with Tekton Pipelines versions 0.29.0 and greater!

This **Task** has two required inputs:

1. The URL of a git repo to clone provided with the **url** param.
2. A Workspace called **output**.

What is a workspace?

- ◆ A shared volume that all tasks have access to
- ◆ Required to share build artifacts
- ◆ Declared in the PipelineRun
- ◆ Implemented as a PersistentVolumeClaim





Defining a workspace

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pipelinerun-pvc
spec:
  StorageClassName: default
resources:
  requests:
    storage: 1Gi
volumeMount: FileSystem
accessModes:
  - ReadWriteOnce
```

```
apiVersion: tekton.dev/v1beta1
kind: PipelineRun
metadata:
  generateName: pipeline-run-
spec:
  pipelineRef:
    name: cd-pipeline
  workspaces:
    - name: pipeline-workspace
      persistentVolumeClaim:
        claimName: pipelinerun-pvc
  params:
    - name: repo-url
      value: "$(tt.params.repo)"
```

Example git-clone



```
spec:
  workspaces:
    - name: pipeline-workspace
  params:
    - name: repo-url
  tasks:
    - name: clone
      taskRef:
        name: git-clone
      workspaces:
        - name: output
          workspace: pipeline-workspace
      params:
        - name: url
          value: "$(params.repo-url)"
```

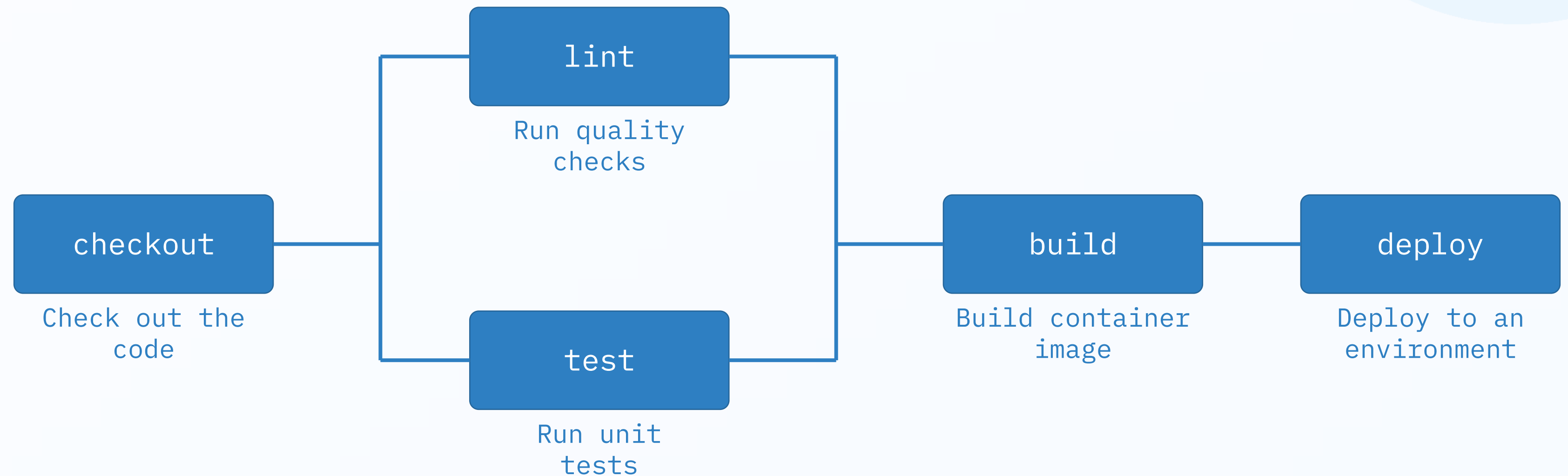
Description

YAML

git-clone

Please Note: this Task is only compatible with Tekton Pipelines

CD pipeline



Questions?

Next Week –Exam review by AJ Suthep

INSTRUCTOR

Dr. Santawat

DEPARTMENT

Computer Engineering, KMUTT

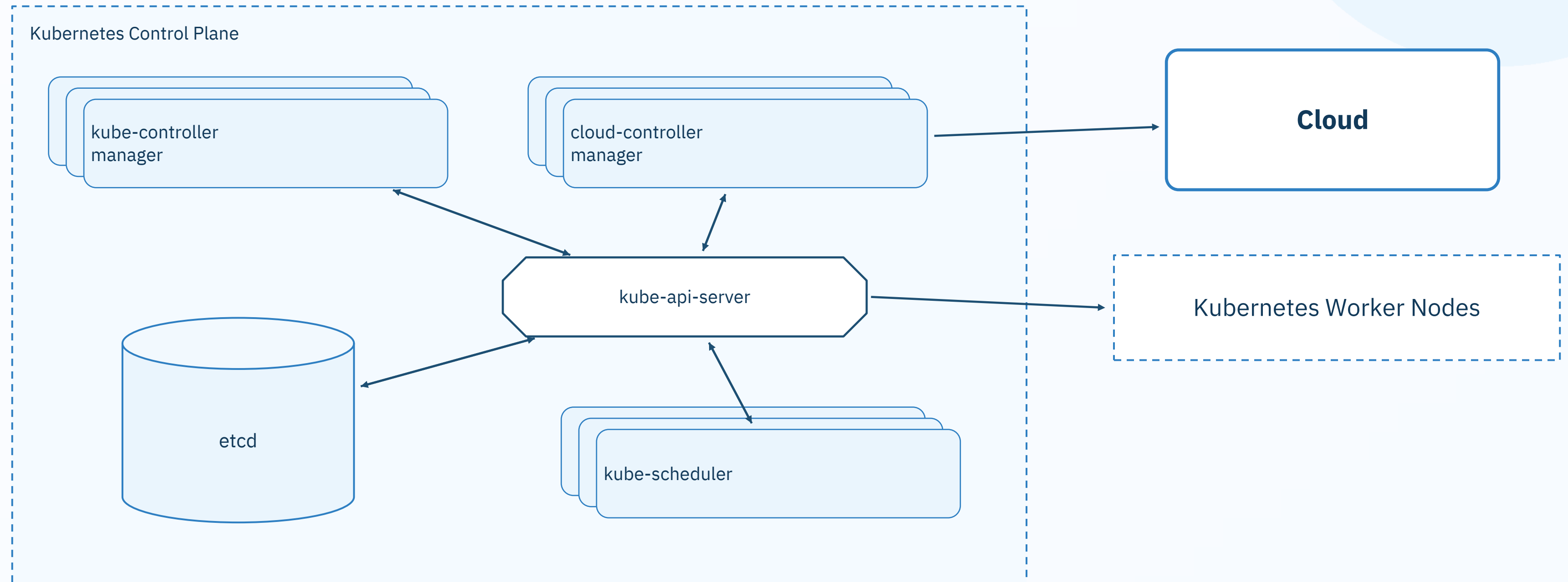


 Extra

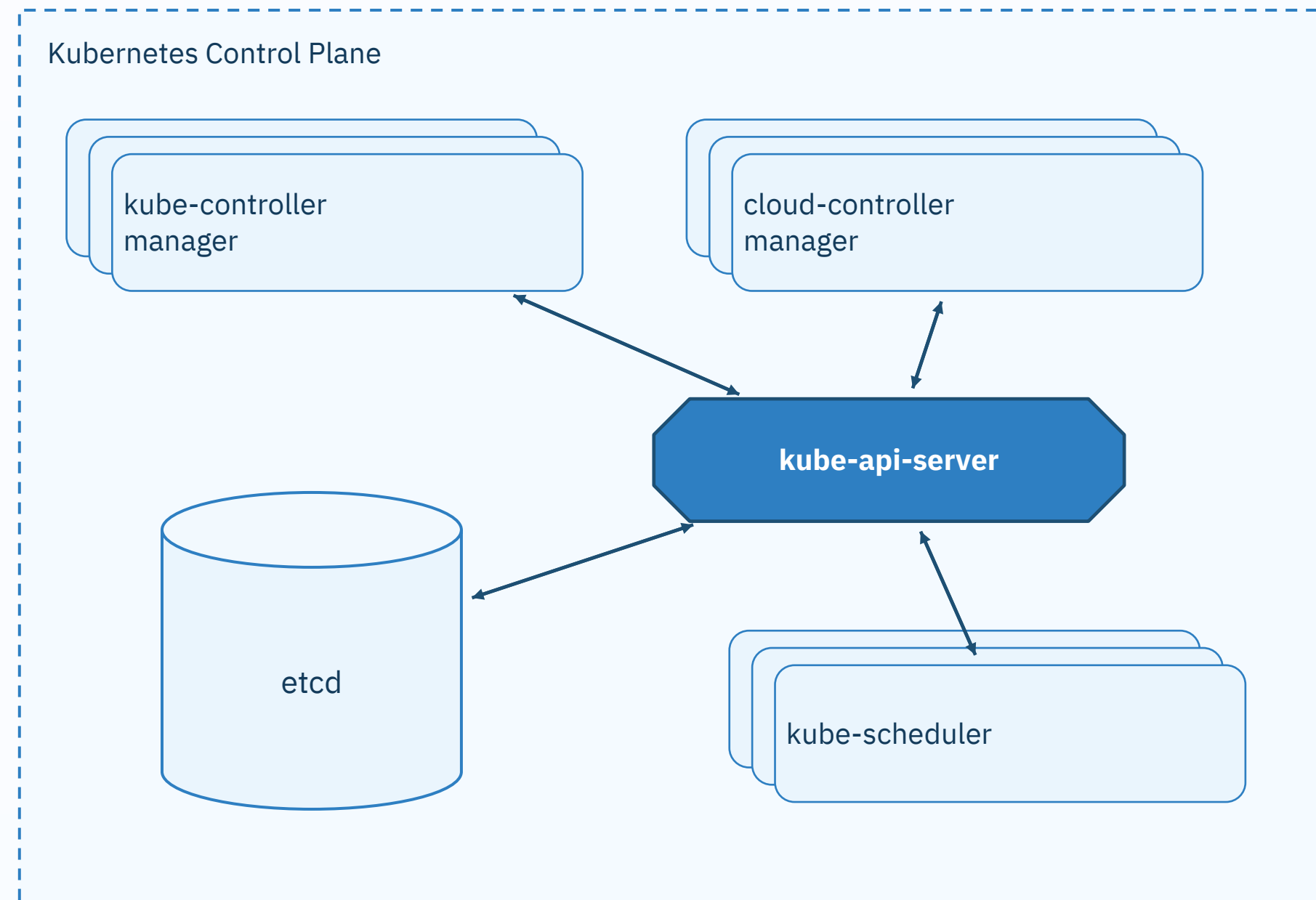
Kubernetes Details



Kubernetes control plane

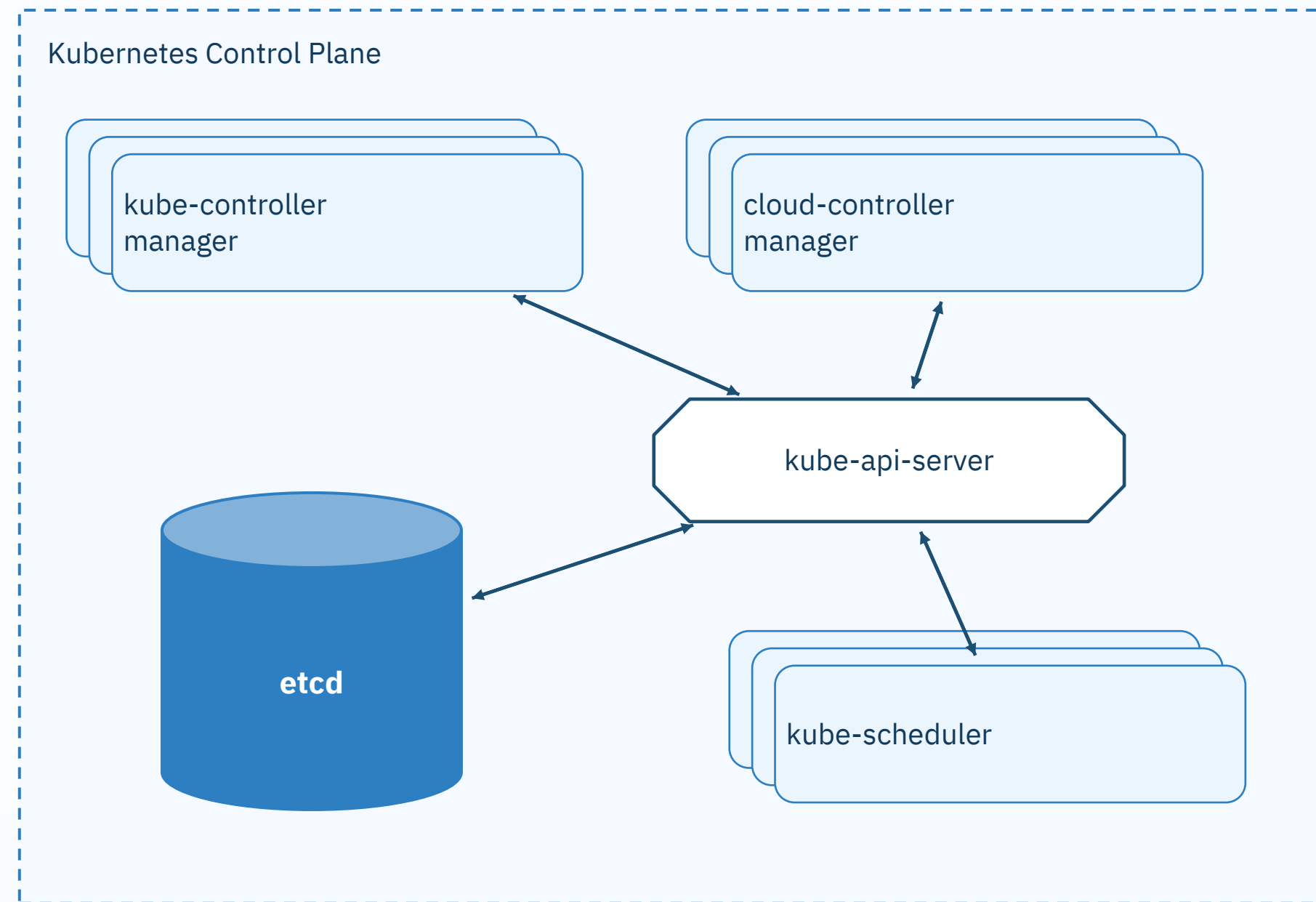
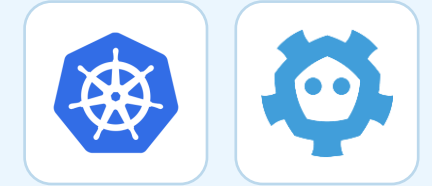


Kubernetes API server



- ◆ Exposes the Kubernetes API
- ◆ Front-end for the Kubernetes control plane
- ◆ All communication in the cluster utilizes this API
- ◆ Designed to scale horizontally and balance traffic between them

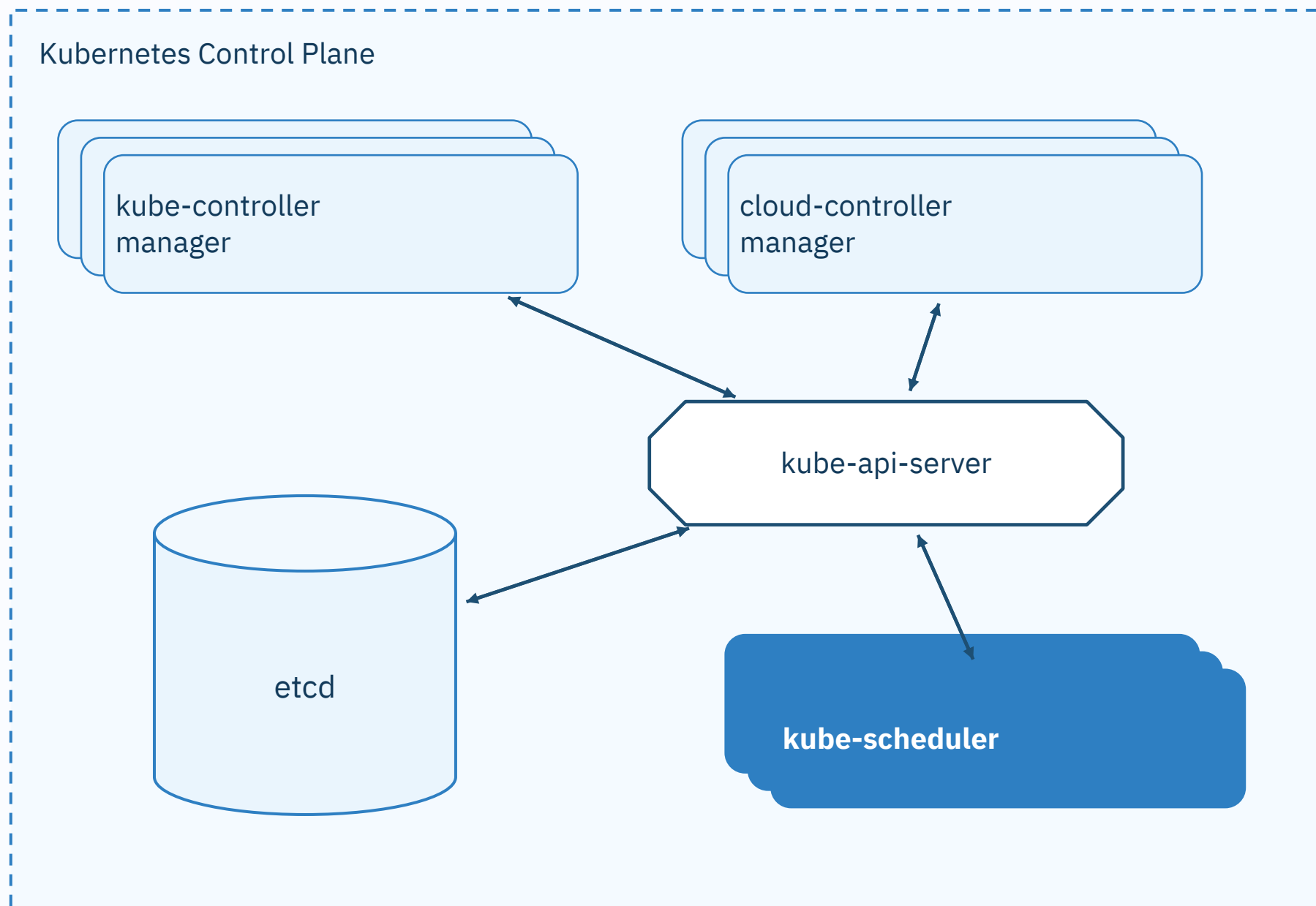
etcd



- ◆ Highly available, distributed key-value store that contains all cluster data
- ◆ Stores deployment configuration data, the desired state, and meta data in a way that can be accessed in a common location



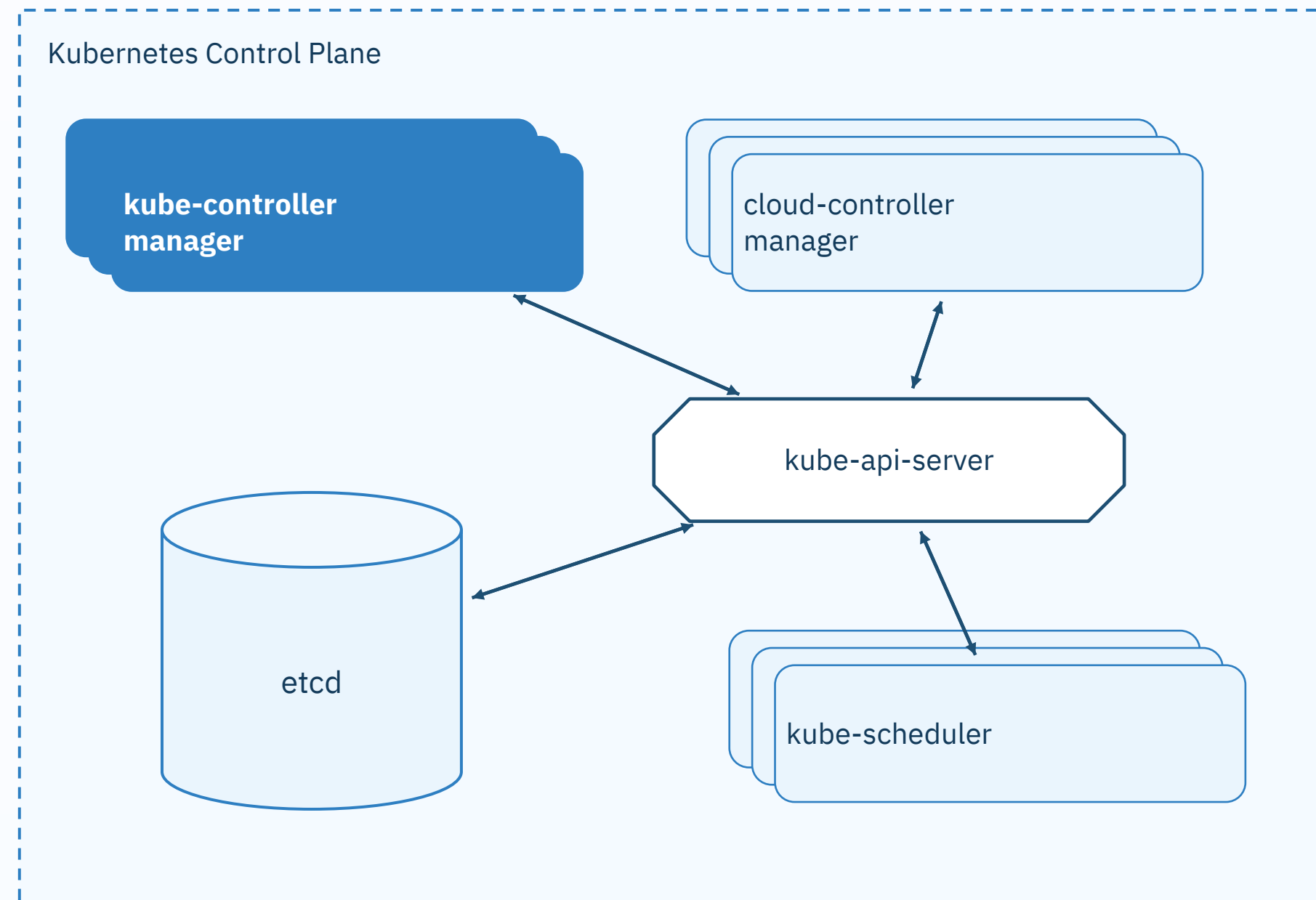
Kubernetes scheduler



- ◆ Assigns newly created Pods to nodes
- ◆ Selects optimal node according to Kubernetes scheduling principles, configuration options, and available resources



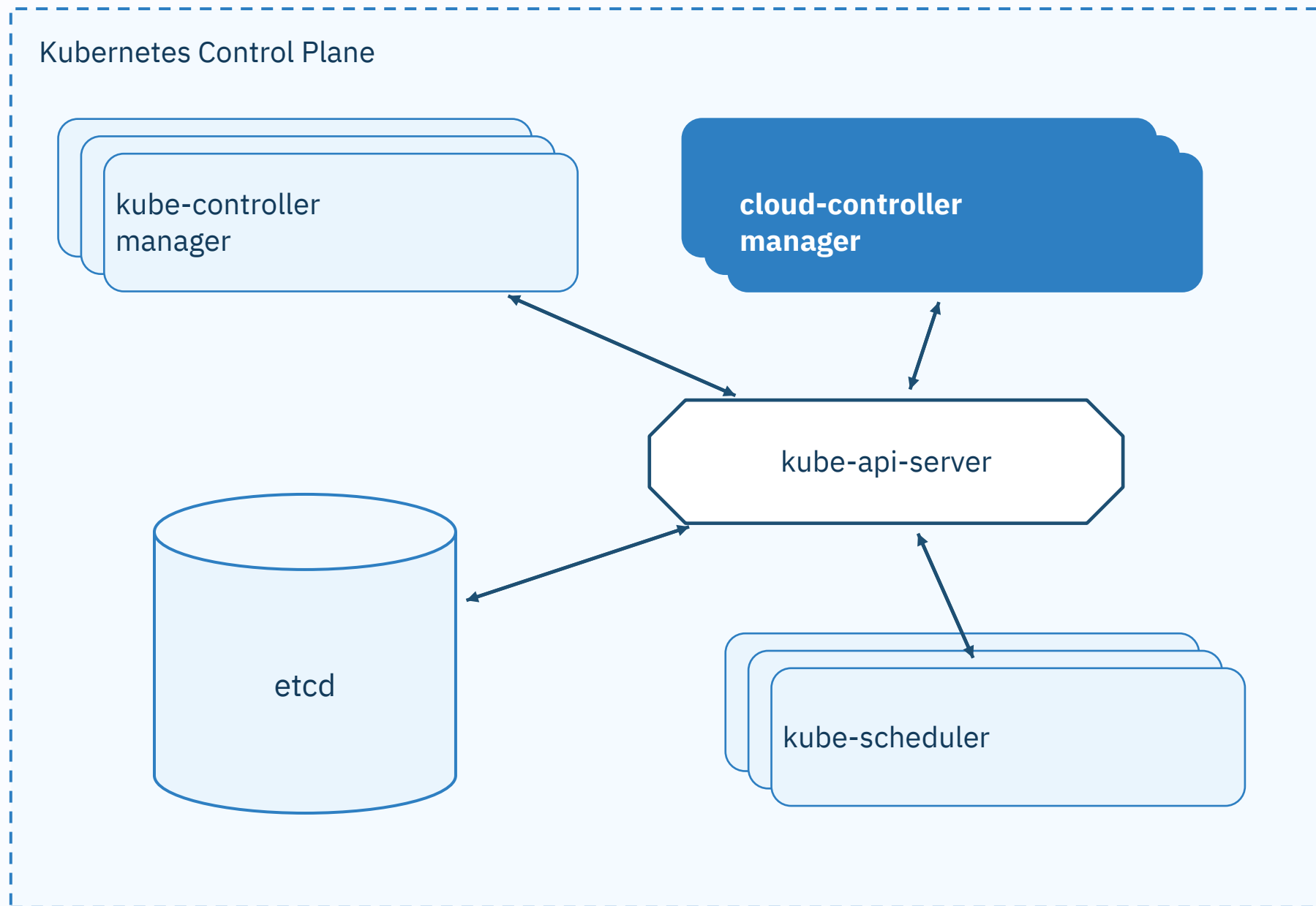
Kubernetes controller manager



- ◆ Runs controller processes that monitor cluster state
- ◆ Runs controller processes that ensure the actual state matches the desired state



Cloud controller manager



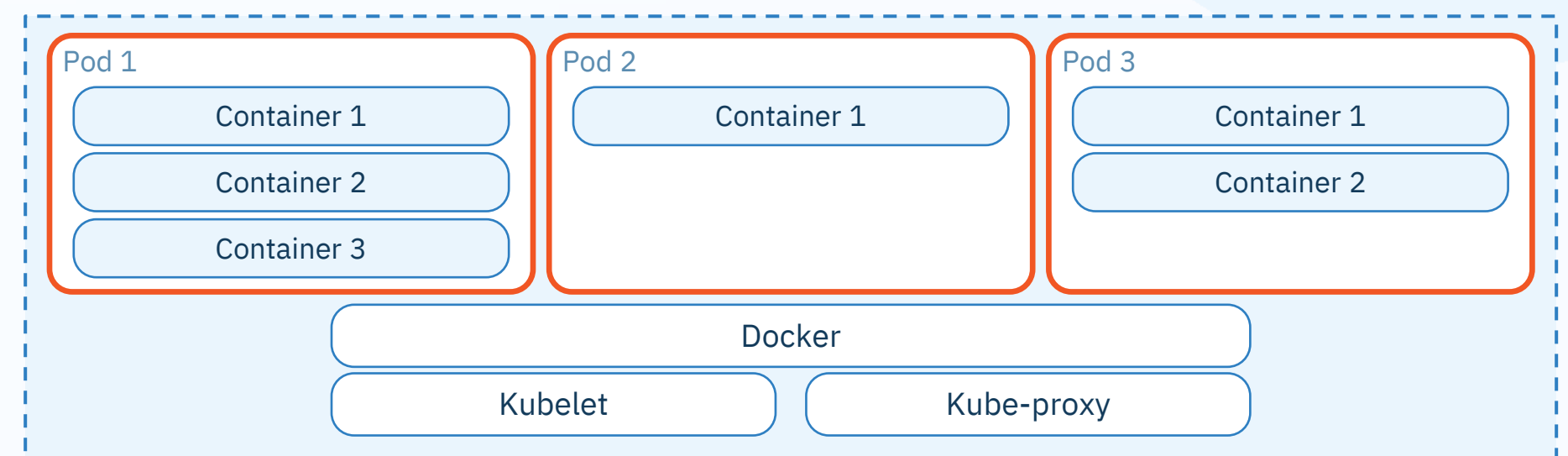
- ◆ Runs controllers that interact with underlying cloud providers
- ◆ Links clusters into a cloud provider's API



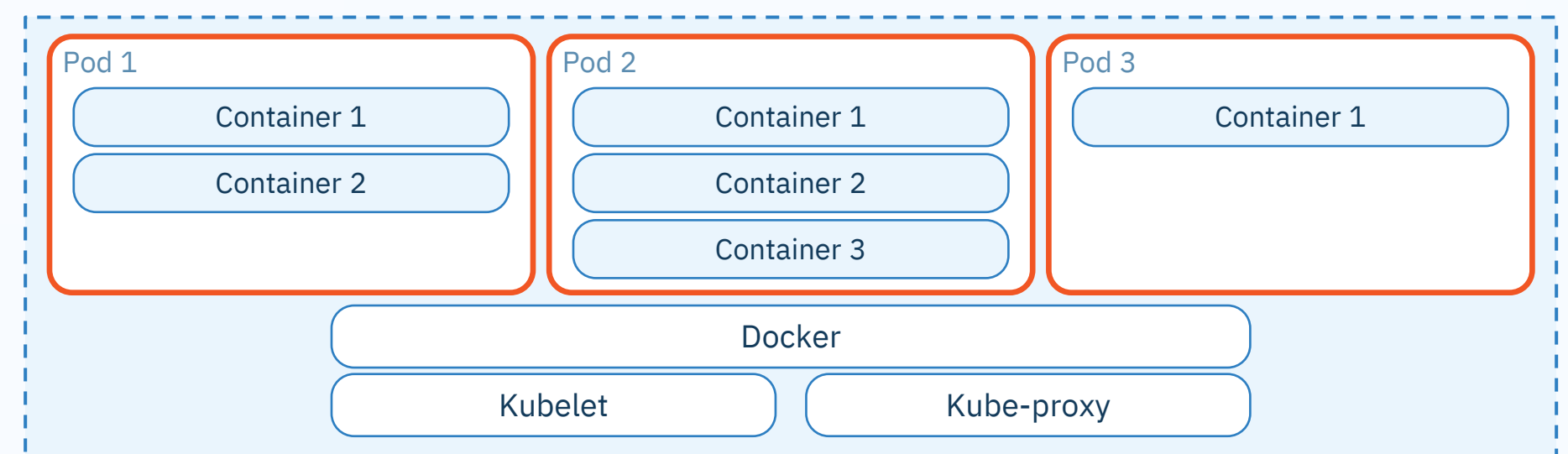
Nodes

- ◆ Are the worker machines in Kubernetes
- ◆ May be a virtual or physical machine
- ◆ Managed by the control plane
- ◆ Contain the services necessary to run applications
- ◆ Nodes include pods which are the smallest deployment entity in Kubernetes

Worker node 1



Worker node 2

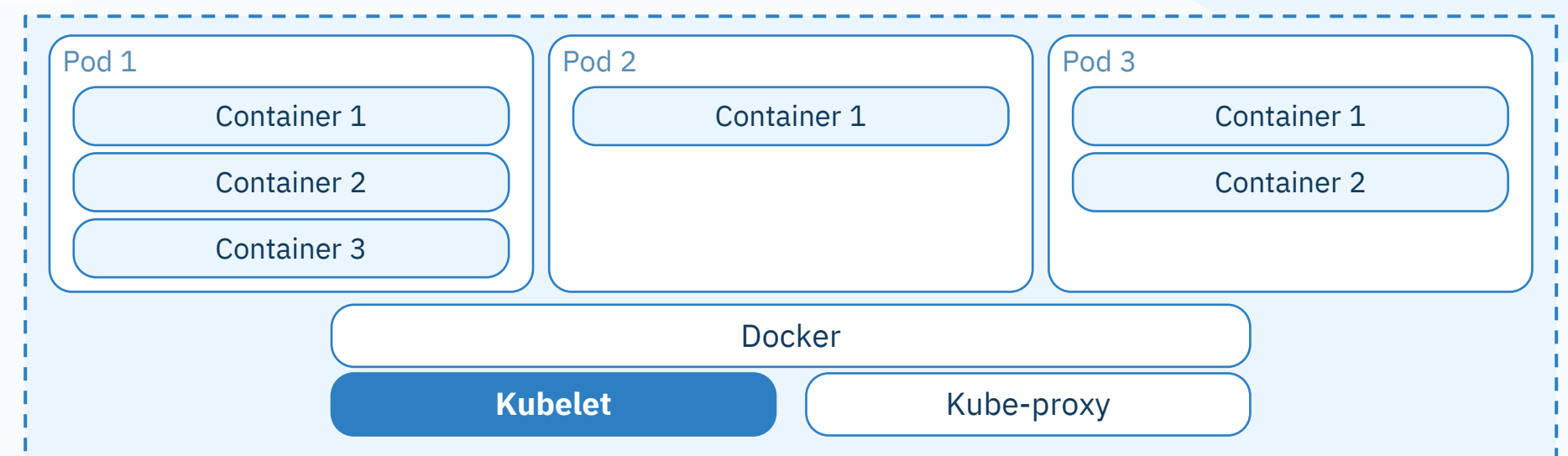




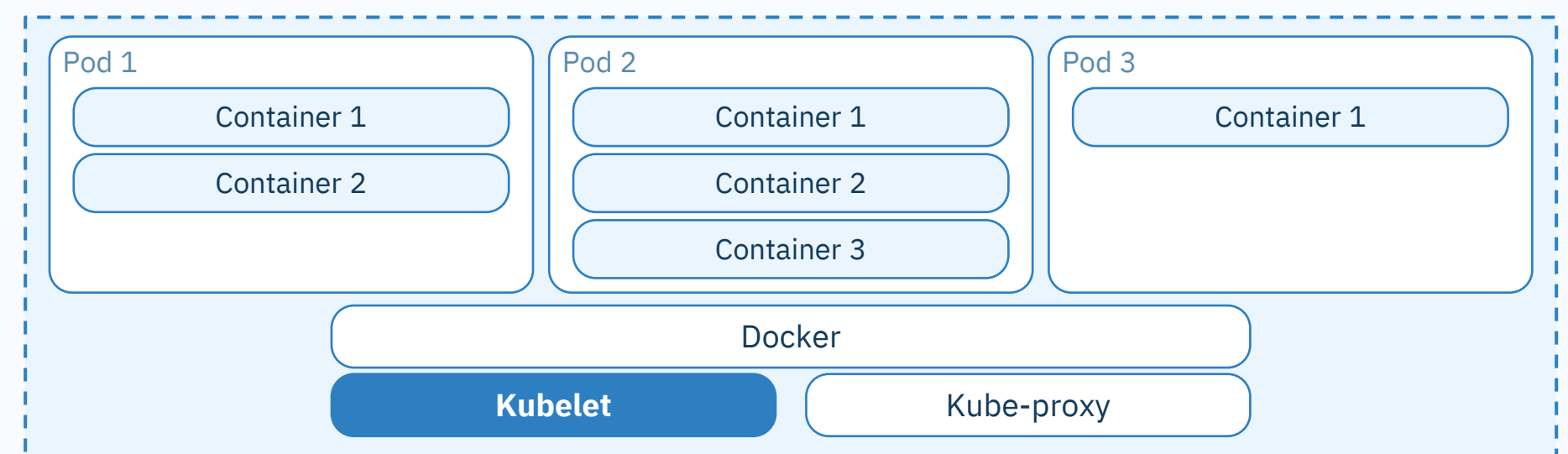
Kubelet

- ◆ Communicates with the API server
- ◆ Ensure that pods and their associated containers are running
- ◆ Reports to the control plane on the pods' health and status

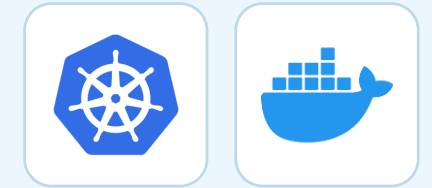
Worker node 1



Worker node 2

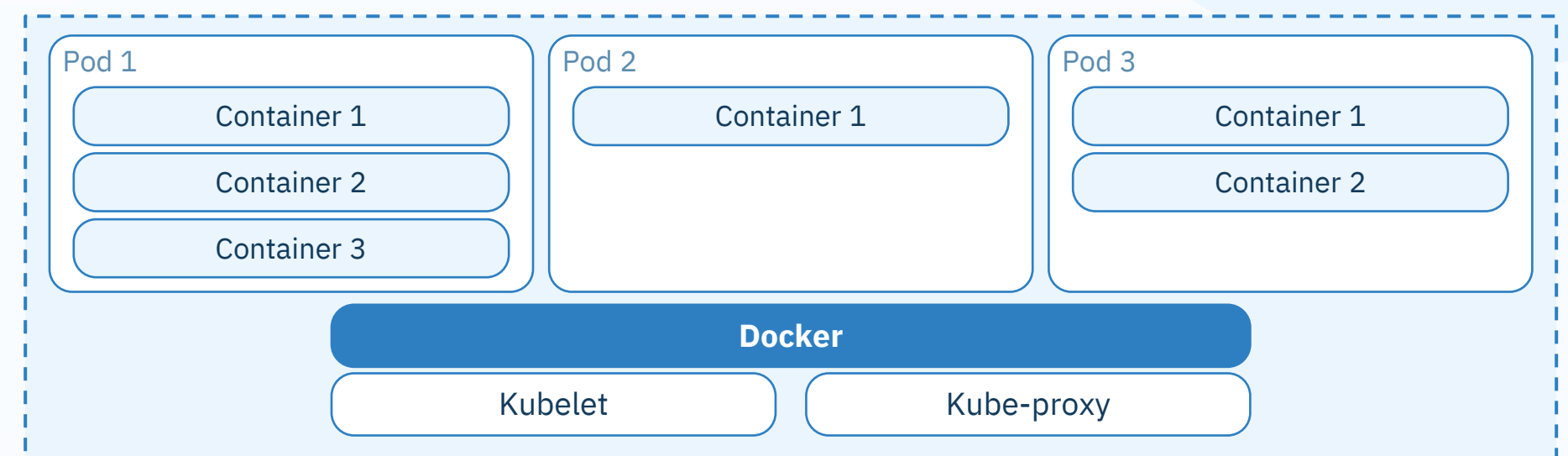


Container runtime

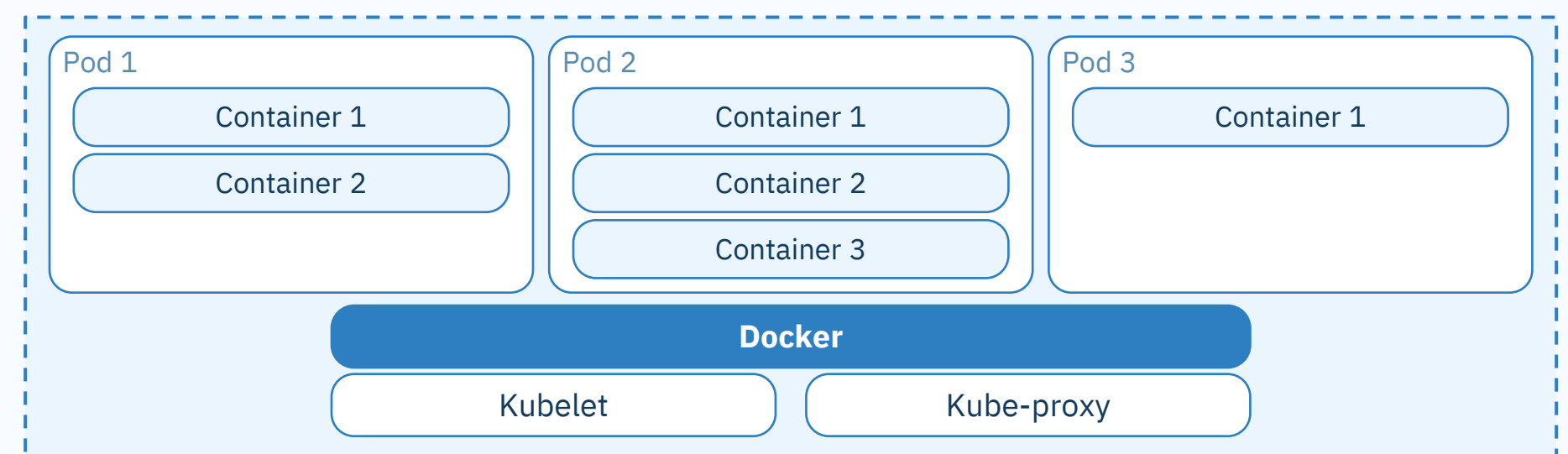


- ◆ Downloads images and runs containers
- ◆ Kubernetes implements an interface so that this component is pluggable
- ◆ Docker is a well-known runtime

Worker node 1



Worker node 2

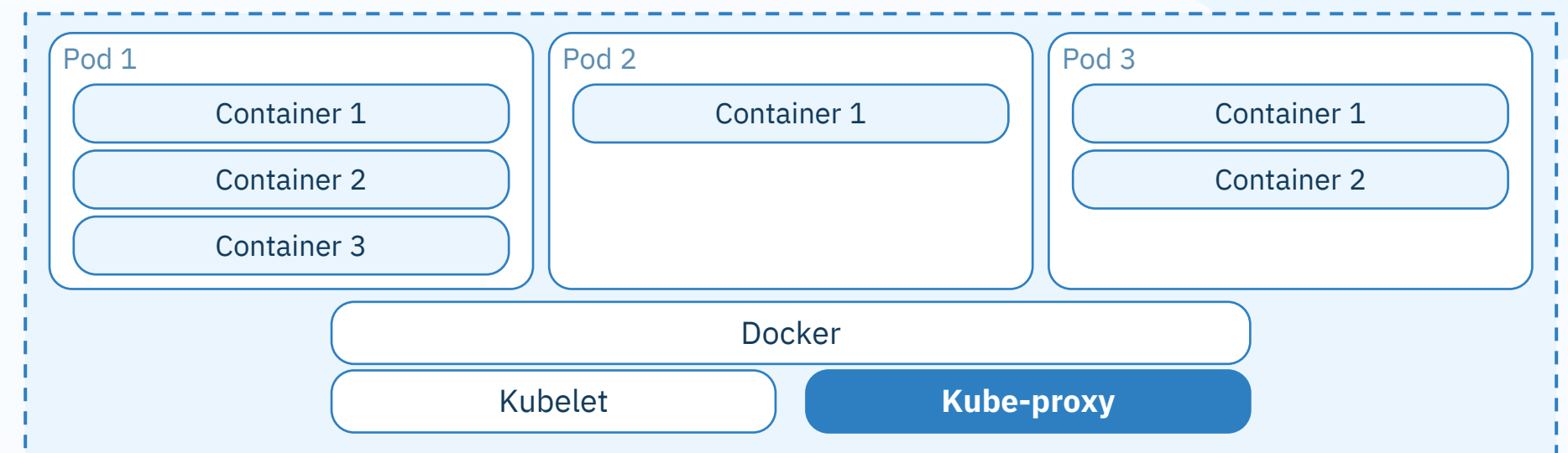




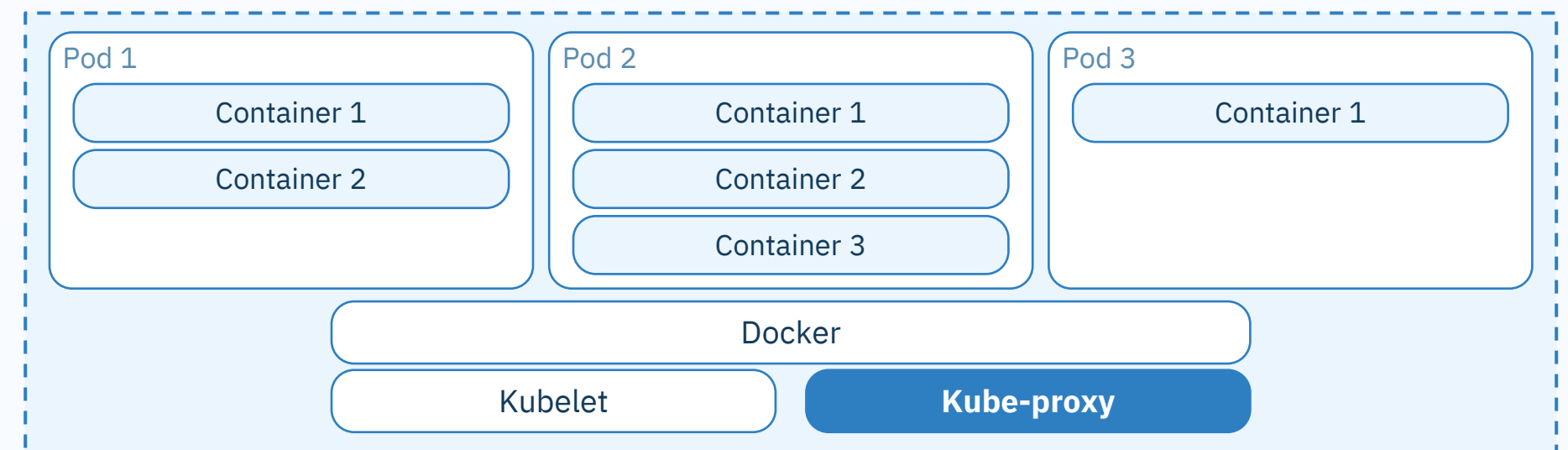
Kubernetes proxy

- ◆ Network proxy
- ◆ Maintains network rules that allow communication to pods

Worker node 1



Worker node 2





Kubectl – The Kubernetes CLI

Kubectl is the Kubernetes command line interface.

Kubectl: Kube Command Tool Line

Helps users deploy applications, inspect and manage cluster resources, view logs, and more

Provides many features for users who work with clusters and manage running cluster workloads

Key command types:

- ◆ Imperative commands
- ◆ Imperative object configuration
- ◆ Declarative object configuration



Kubectl – The Kubernetes CLI

Kubectl is the Kubernetes command line interface.

Kubectl: Kube Command Tool Line

Helps users deploy applications, inspect and manage cluster resources, view logs, and more

Provides many features for users who work with clusters and manage running cluster workloads

Key command types:

- ◆ Imperative commands
- ◆ Imperative object configuration
- ◆ Declarative object configuration



Kubectl command structure

Kubectl commands use the following structure:

```
kubectl [command] [type] [name] [flags]
```

[command] = any operation to be performed (create, get, apply, delete)

[type] = resource type (pod, deployment, replicaset)

[name] = resource name (if applicable)

[flags] = special options or modifiers that override default values

Imperative commands



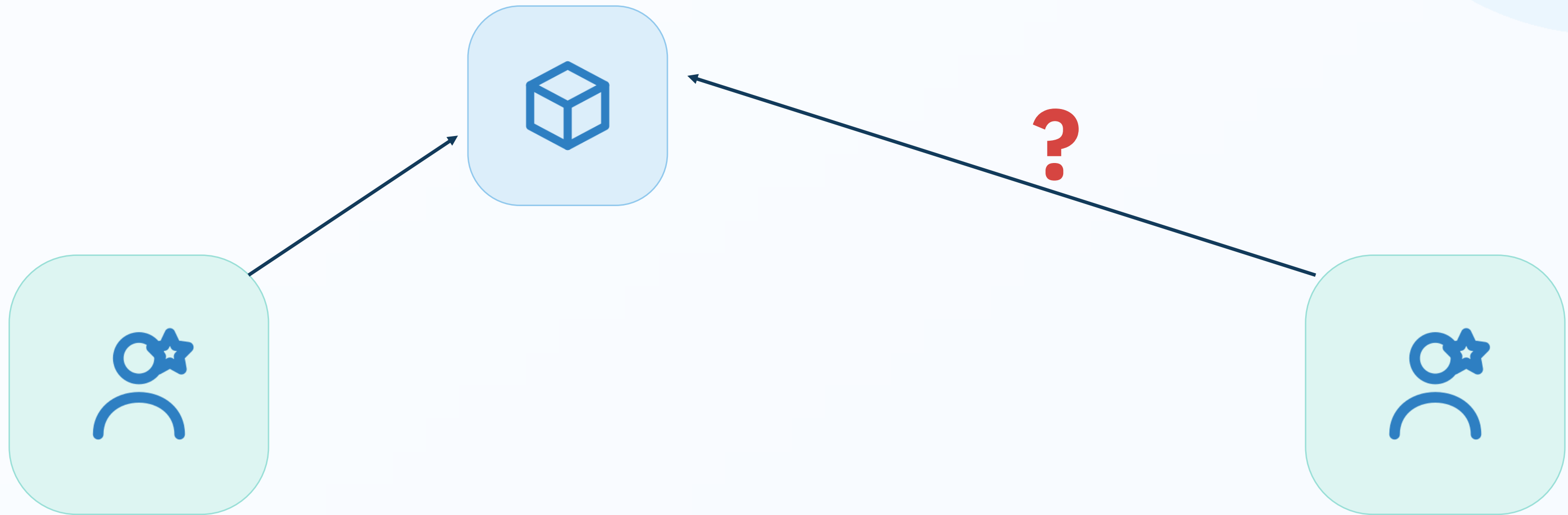
Imperative commands create, update, and delete live objects directly:

- ◆ Operations specified as arguments or flags
- ◆ Easiest to learn

```
kubectl run nginx --image nginx
```

- ◆ No audit trail
- ◆ Not very flexible (no template, no integration)

Imperative command limitations





Imperative object configuration

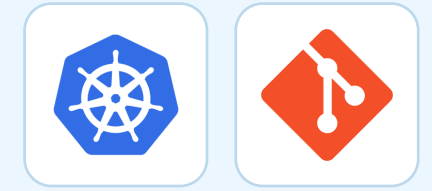
The kubectl command specifies required operations, optional flags, and at least one file name

- ◆ The configuration file specified must contain a full definition of the objects in YAML or JSON format

```
kubectl create -f nginx.yaml
```

- ◆ Configuration templates help replicate identical results

Imperative object configuration



The kubectl command specifies required operations, optional flags, and at least one file name

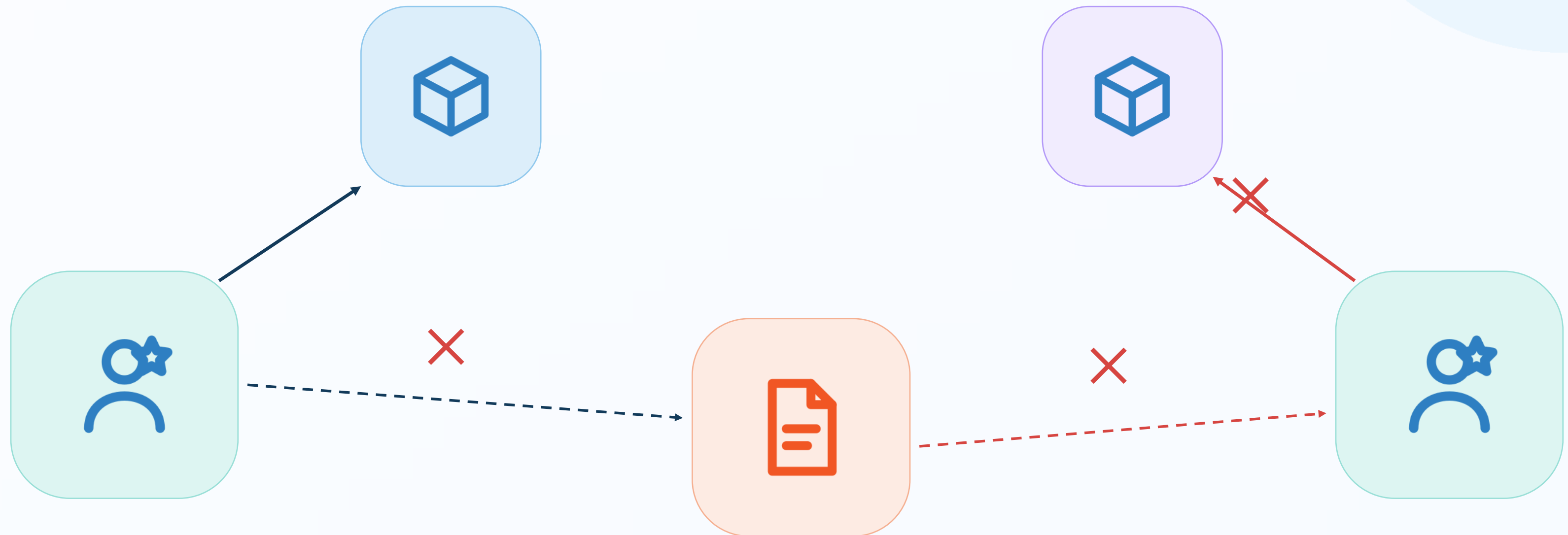
Advantages:

- ◆ May be stored in a source control system like Git
- ◆ Can integrate with change processes
- ◆ Provides audit trails and templates for creating new objects

Disadvantages:

- ◆ Requires basic understanding of the object schema

Imperative approach limitations



Declarative object configuration



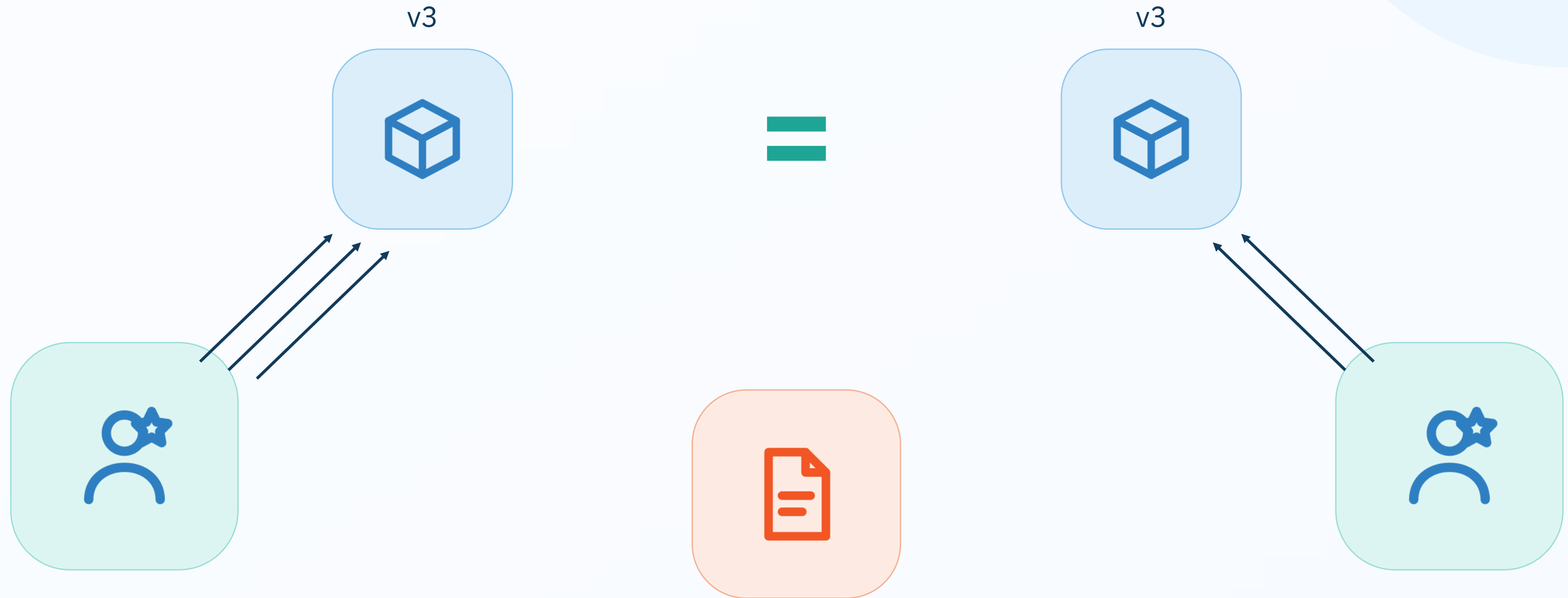
Declarative object configuration stores configuration data in files:

- ◆ Operations are identified by Kubectl, not the user
- ◆ Works on directories or individual files

```
kubectl apply -f nginx/
```

- ◆ Configuration files define desired state

Declarative configuration benefits





Kubectl commonly used commands

Command	Description
apply	Apply/change a configuration of a resource using a file or stdin
create	Create one or more resources using a file or stdin
describe	Describe or detail a file/container
get	Access a file/container or other resource
delete	Delete a file/container
autoscale	Apply autoscaling to the selected file/container
edit	Make changes to a file/container
exec	Execute a command on a container in a specific pod
expose	Make a running file/container available
label	Apply a label to a file/container



Kubectl 'get' command examples

Kubectl 'get' commands deal with listing different objects or resources.

```
kubectl get services
```

```
kubectl get pods -all-namespaces
```

```
kubectl get deployment my-dep
```

```
kubectl get pods
```

Kubectl 'apply' command examples

Kubectl 'apply' commands create resources using YAML or JSON files.

```
kubectl apply -f ./my1.yaml -f ./my2.yaml
```

```
kubectl apply -f https://git.io/vPieo
```



Kubectl 'scale' command examples

Kubectl 'scale' commands scale (increase/decrease) the number of replicas.

```
kubectl scale --replicas=3 rs/foo
```

```
kubectl scale --replicas=3 -f foo.yaml
```

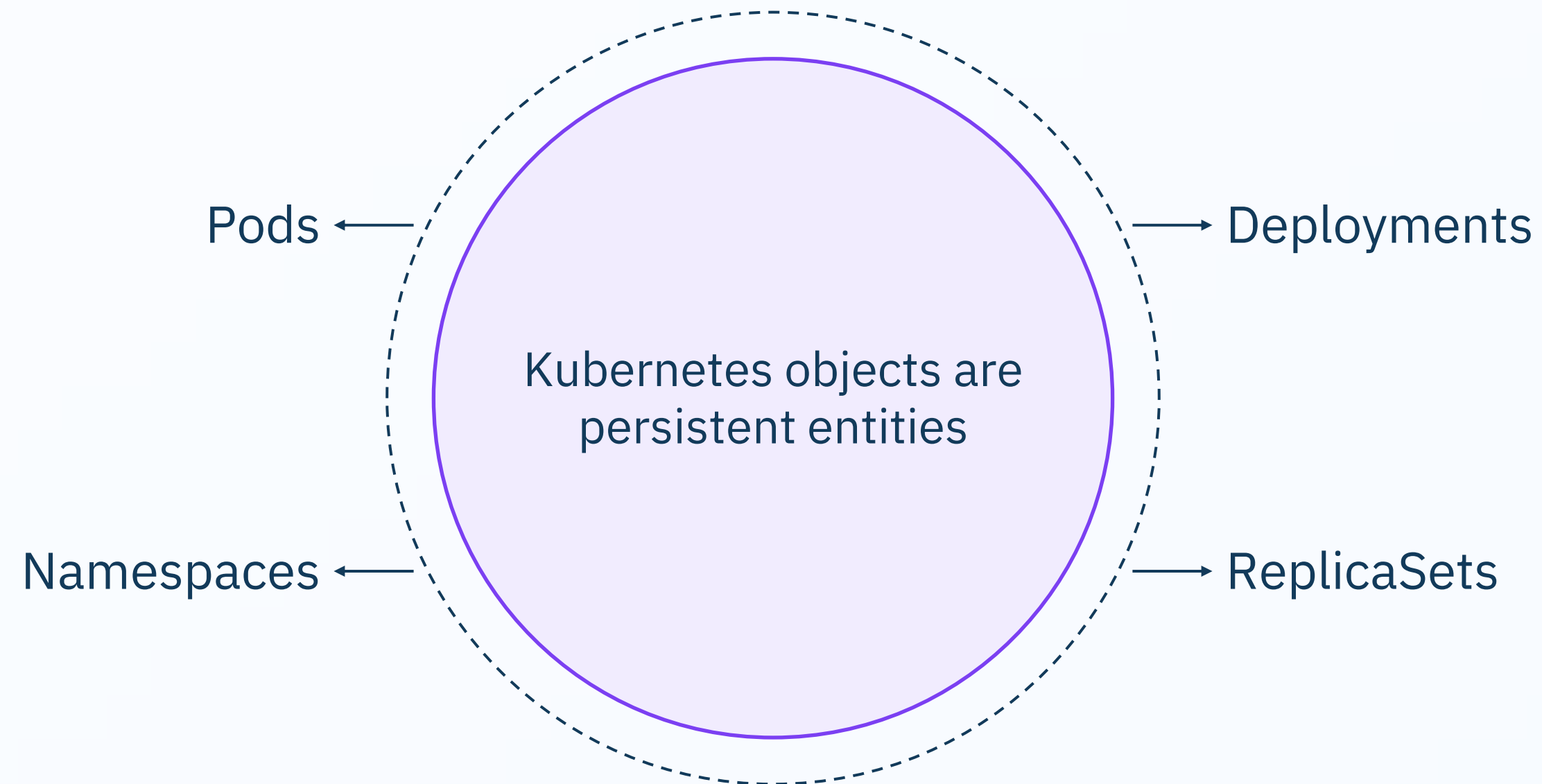


 Extra

Kubernetes objects



What are Kubernetes objects?





What are Kubernetes objects?

- ◆ Kubernetes objects consist of two main fields.

Object spec

- ◆ Provided by user
- ◆ Defines desired state

Status

- ◆ Provided by Kubernetes
- ◆ Defines current state

- ◆ To work with Kubernetes objects, use the Kubernetes API directly with the client libraries, the kubectl command-line interface, or both

Labels and selectors



Labels are key/value pairs attached to objects

- ◆ Intended for identification of objects
- ◆ Not unique
- ◆ Help organize and group objects

Label selectors are the core grouping method in Kubernetes

- ◆ Identify and group a set of objects

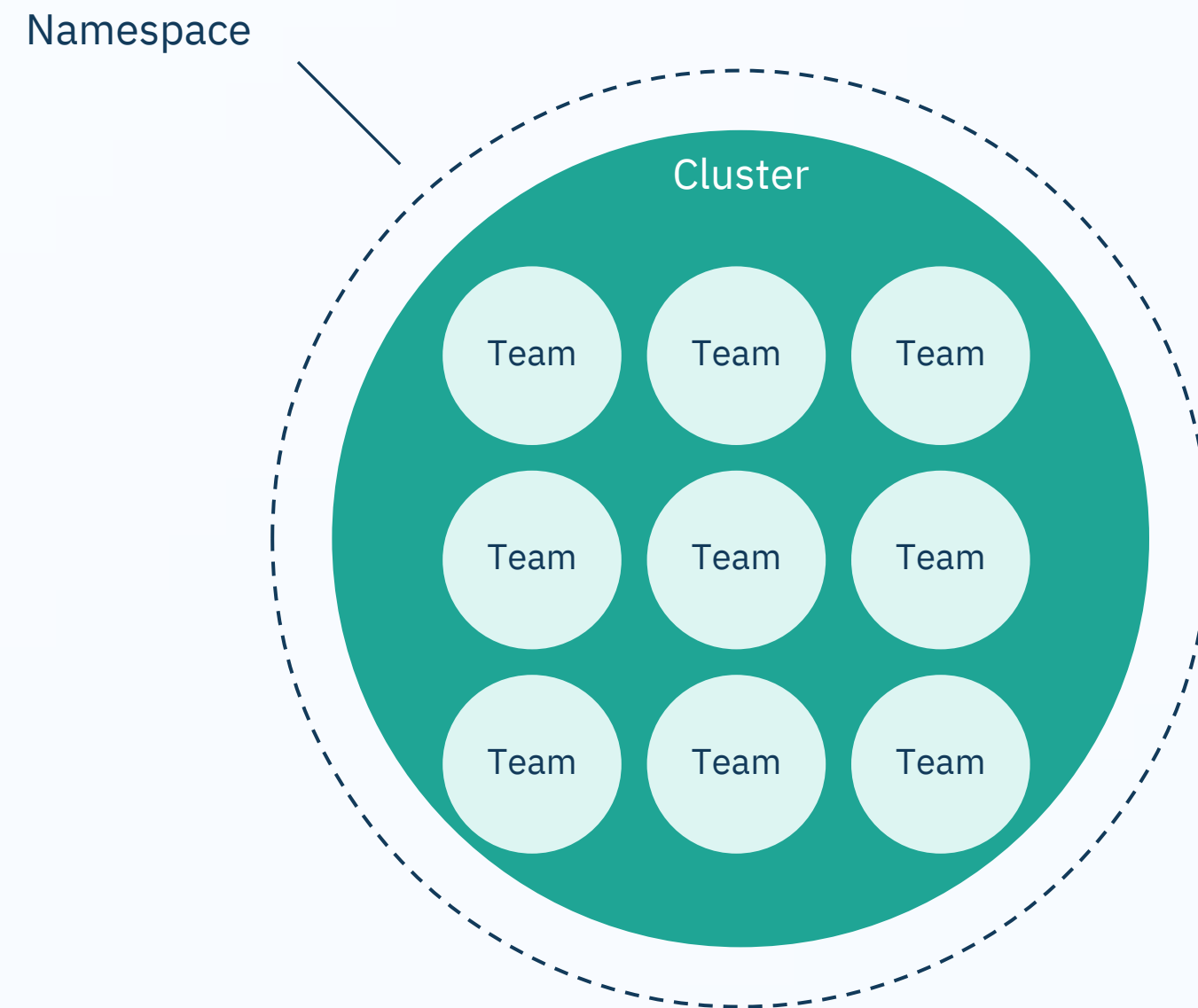


Namespaces and names

- ◆ Namespaces provide a mechanism for isolating groups of resources within a single cluster.

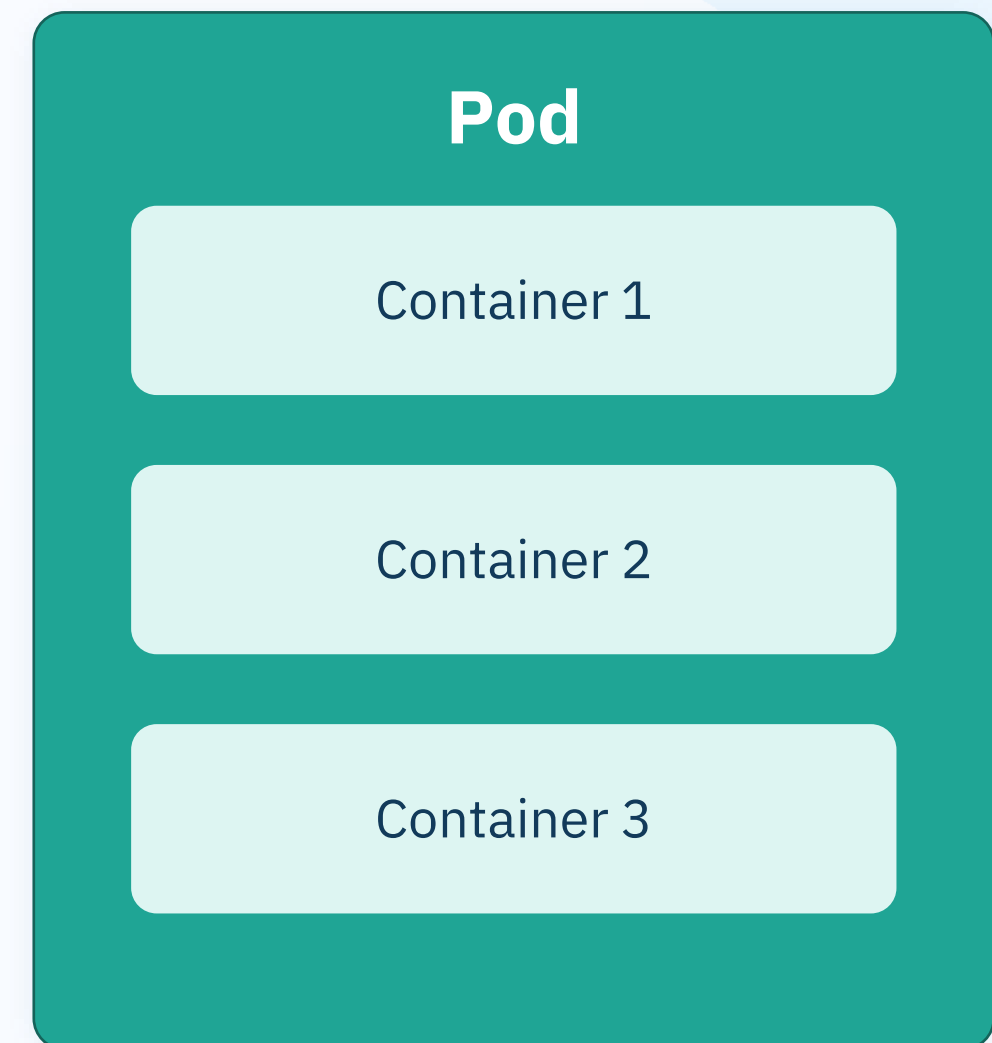
Namespaces provide a scope for object names

- ◆ Each object has a name
- ◆ Names are unique for a resource type within a namespace



Pods

- ◆ Simplest unit in Kubernetes
- ◆ Represents processes running in your cluster
- ◆ Encapsulates one or more containers
- ◆ Replicating a Pod serves to scale applications horizontally



Pods



- ◆ Simplest unit in Kubernetes
- ◆ Represents processes running in your cluster
- ◆ Encapsulates one or more containers
- ◆ Replicating a Pod serves to scale applications horizontally

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
spec:
  containers:
  - name: nginx
    image: nginx:1.7.9
    ports:
    - containerPort: 80
```

ReplicaSet



A ReplicaSet is a set of horizontally scaled running Pods.

- ◆ A ReplicaSet configuration file defines:
 - Number of replicas
 - Pod template
 - Selector to identify which Pods it can acquire
- ◆ Generally encapsulated by a Deployment

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.7.9
          ports:
            - containerPort: 80
```

Deployment

A Deployment is a higher-level object that provides updates for Pods and ReplicaSets.

Deployments:

- ◆ Run multiple replicas of an application
- ◆ Suitable for stateless applications
- ◆ Update triggers a rollout

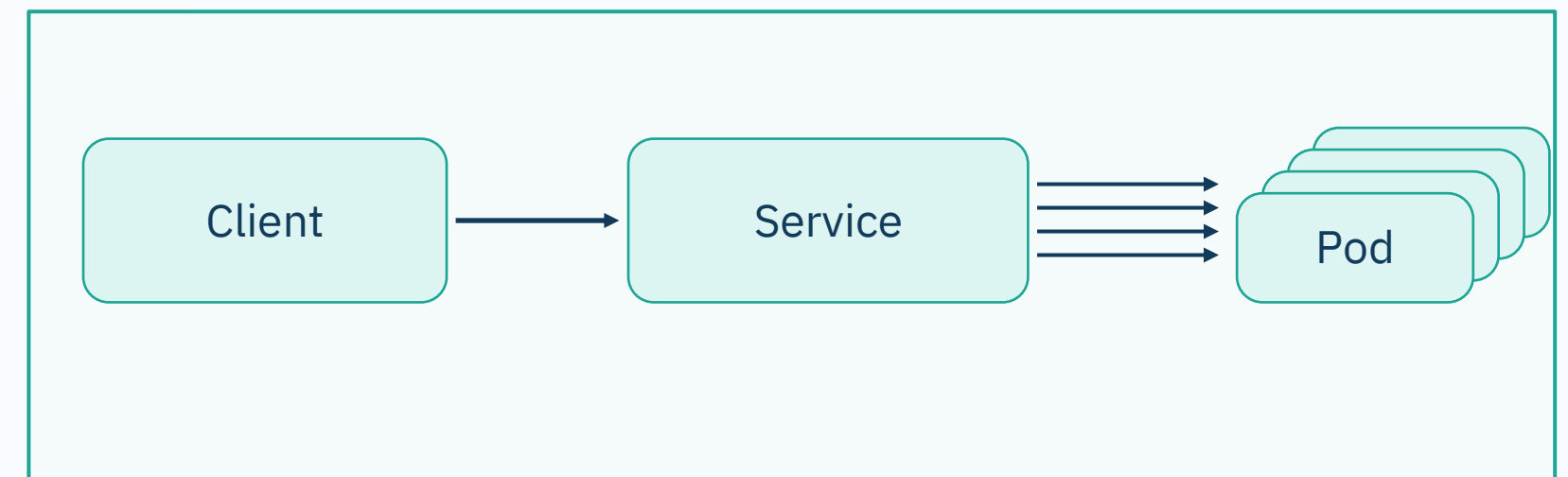


```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.7.9
        ports:
        - containerPort: 80
```

Service



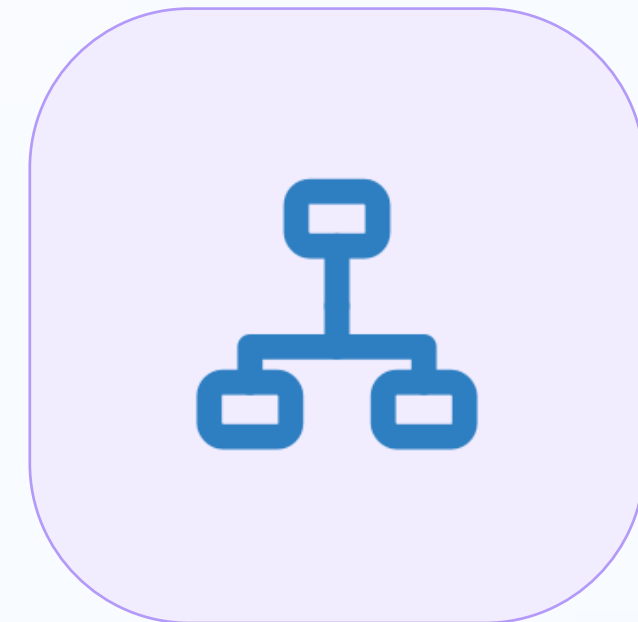
- ◆ Is a REST object, like Pods
- ◆ Is a logical abstraction for a set of Pods in a cluster
- ◆ Provides policies for accessing the Pods and cluster
- ◆ Acts as a load balancer across the Pods
- ◆ Is assigned a unique IP address for accessing applications deployed on Pods
- ◆ Eliminates the need for a separate service discovery process



— KUBERNETES OBJECTS

Service properties

- ◆ Supports multiple protocols such as TCP (default), UDP, and others
- ◆ Supports multiple port definitions
 - The port number with the same name can vary in each backend Pod
- ◆ Can have an optional selector
- ◆ Optionally maps incoming ports to a targetPort





Why is a Service needed?

- ◆ Pods in a cluster are volatile
- ◆ This volatility leads to discoverability issues because of changing IP addresses
- ◆ A Service keeps track of the changes and exposes a single IP address or a DNS name
- ◆ A Service utilizes selectors to target a set of Pods

Native Kubernetes applications

- ◆ Update API endpoints when there are changes in the Pods of a Service

Non-Native Kubernetes applications

- ◆ Use a virtual-IP-based bridge or load balancer between the applications and the backend Pods

Service types

◆ There are four types of Services:

ClusterIP

NodePort

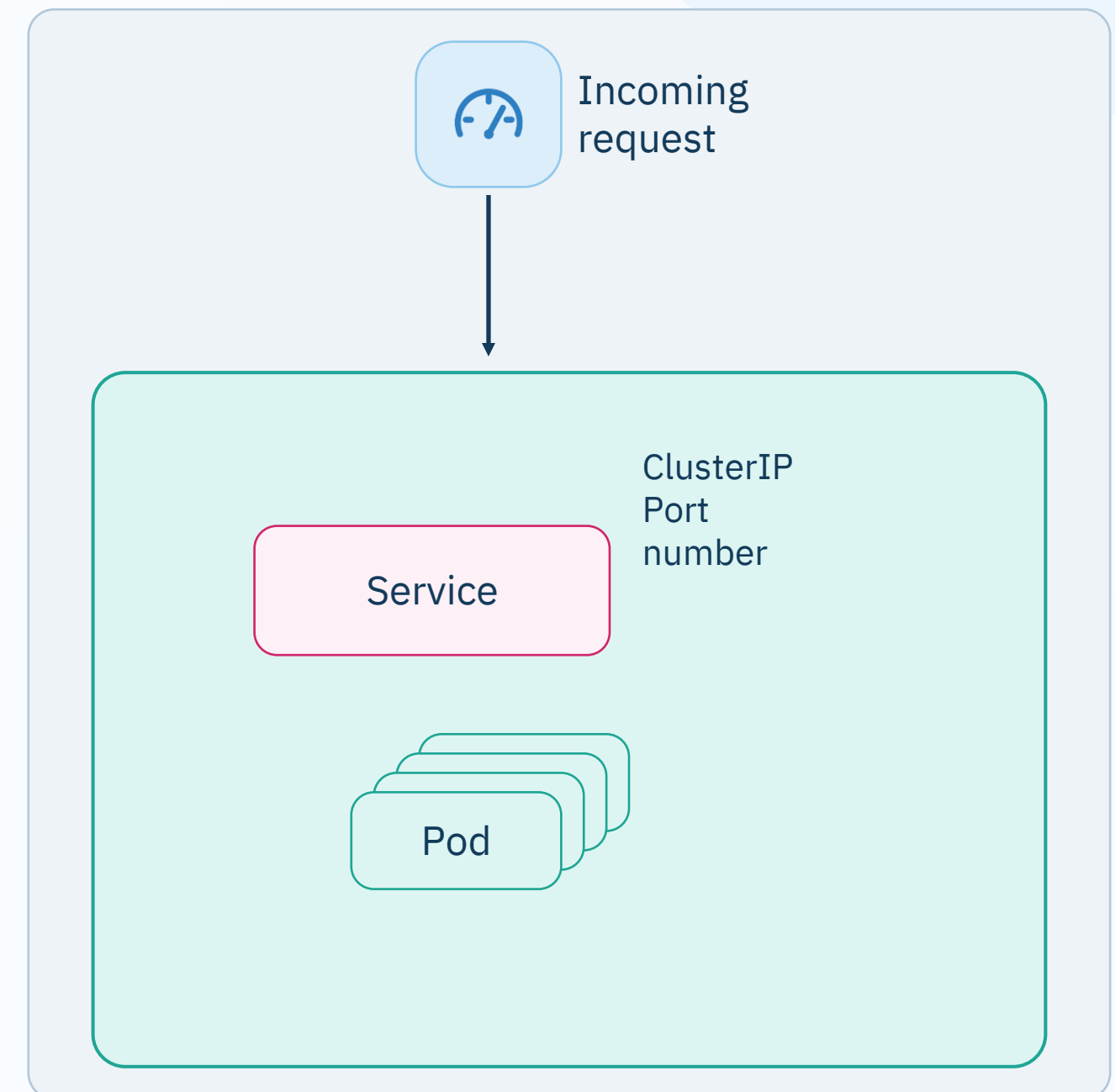
LoadBalancer

External Name



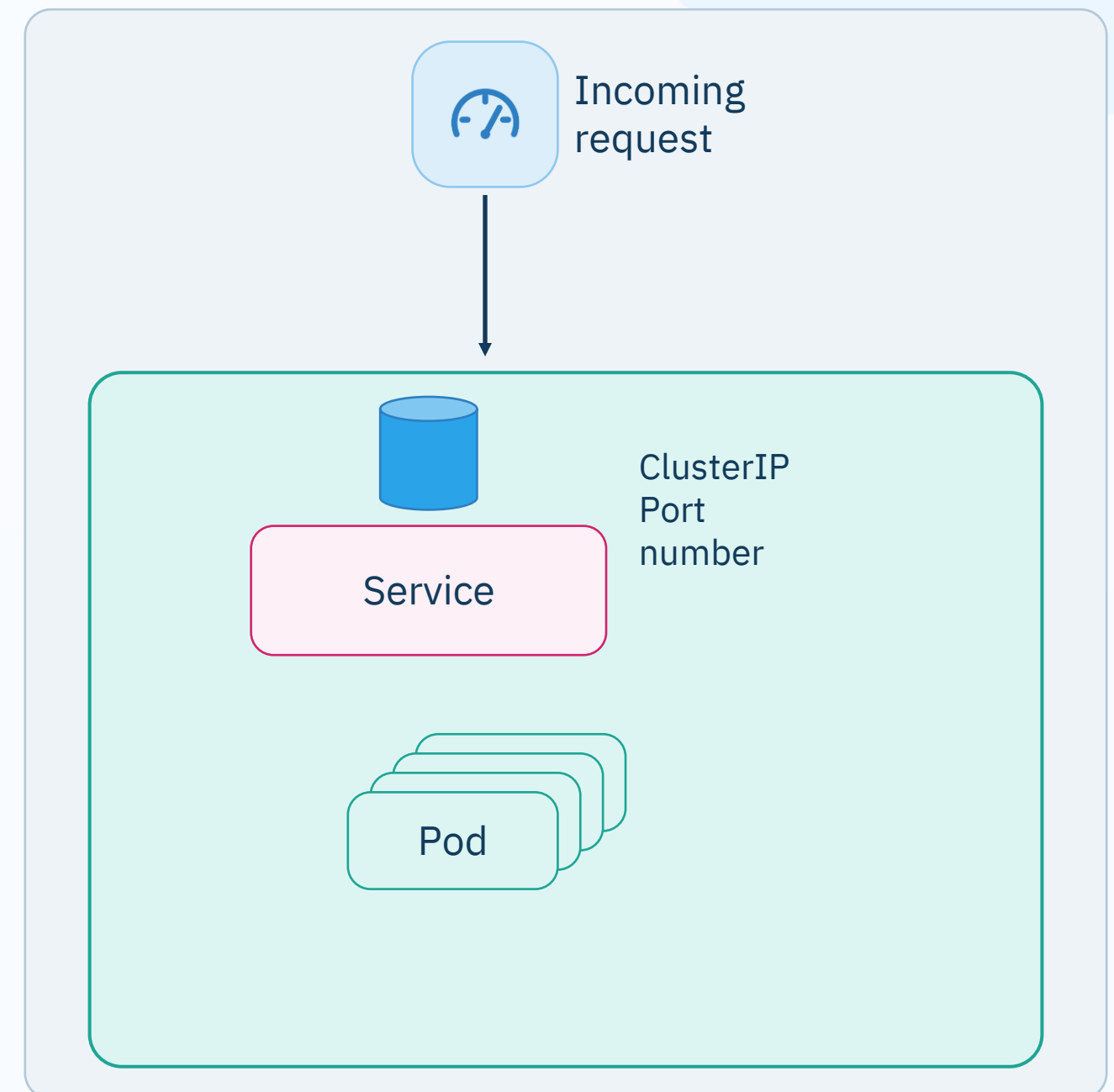
Service - ClusterIP

- ◆ Is the default and most common Service type
- ◆ Is assigned a cluster-internal IP address to the ClusterIP Service that makes the Service only reachable within the cluster
- ◆ Cannot make requests to Service (Pods) from outside the cluster
- ◆ You set the ClusterIP address in the Service definition file
- ◆ Provides inter-service communication within the cluster



Service - NodePort

- ◆ Is an extension of ClusterIP Service
- ◆ Creates and routes the incoming requests automatically to the ClusterIP Service
- ◆ Exposes the Service on each Node's IP address at a static port
- ◆ Exposes a single Service, with no load-balancing requirements for multiple services

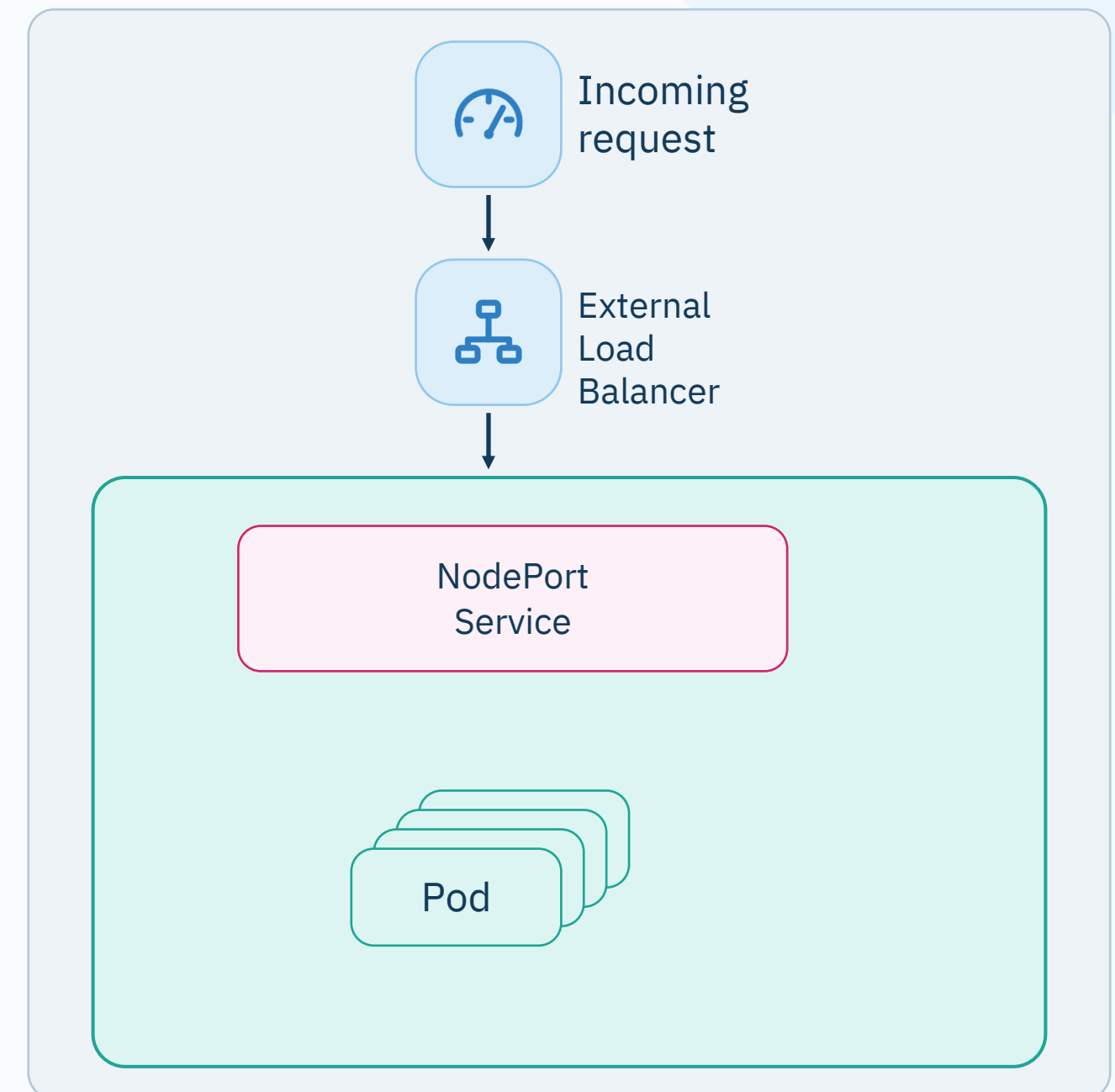


— KUBERNETES OBJECTS

Service – External Load Balancer (ELB)

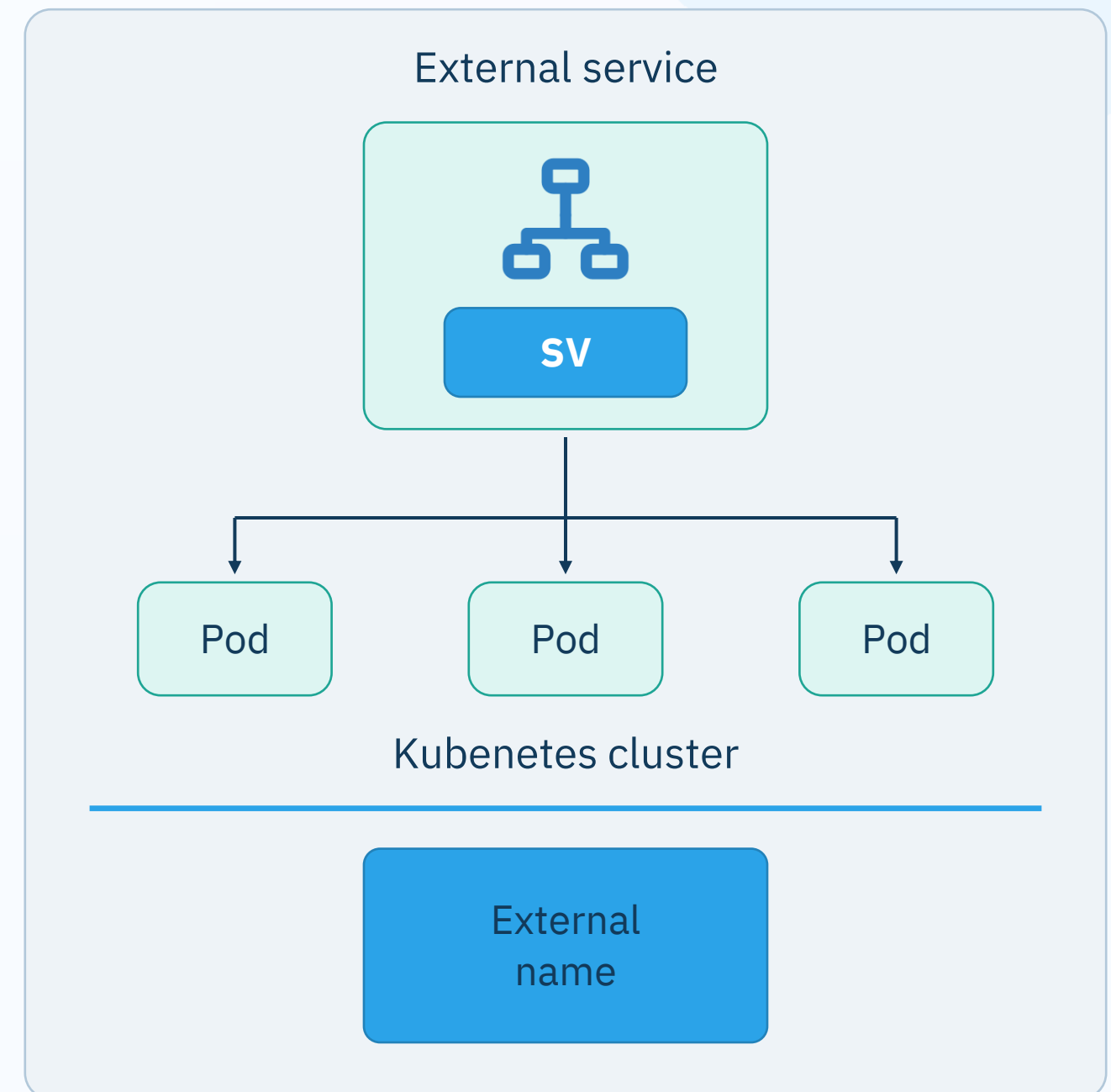


- ◆ An extension of the NodePort Service, an External Load Balancer or ELB, creates NodePort and ClusterIP Services automatically
- ◆ Integrates and automatically directs traffic to the NodePort Service with a cloud provider's ELB
- ◆ To expose a Service to the Internet, you need a new ELB with an IP address
- ◆ Use a cloud provider's ELB to host your cluster



Service – External Name

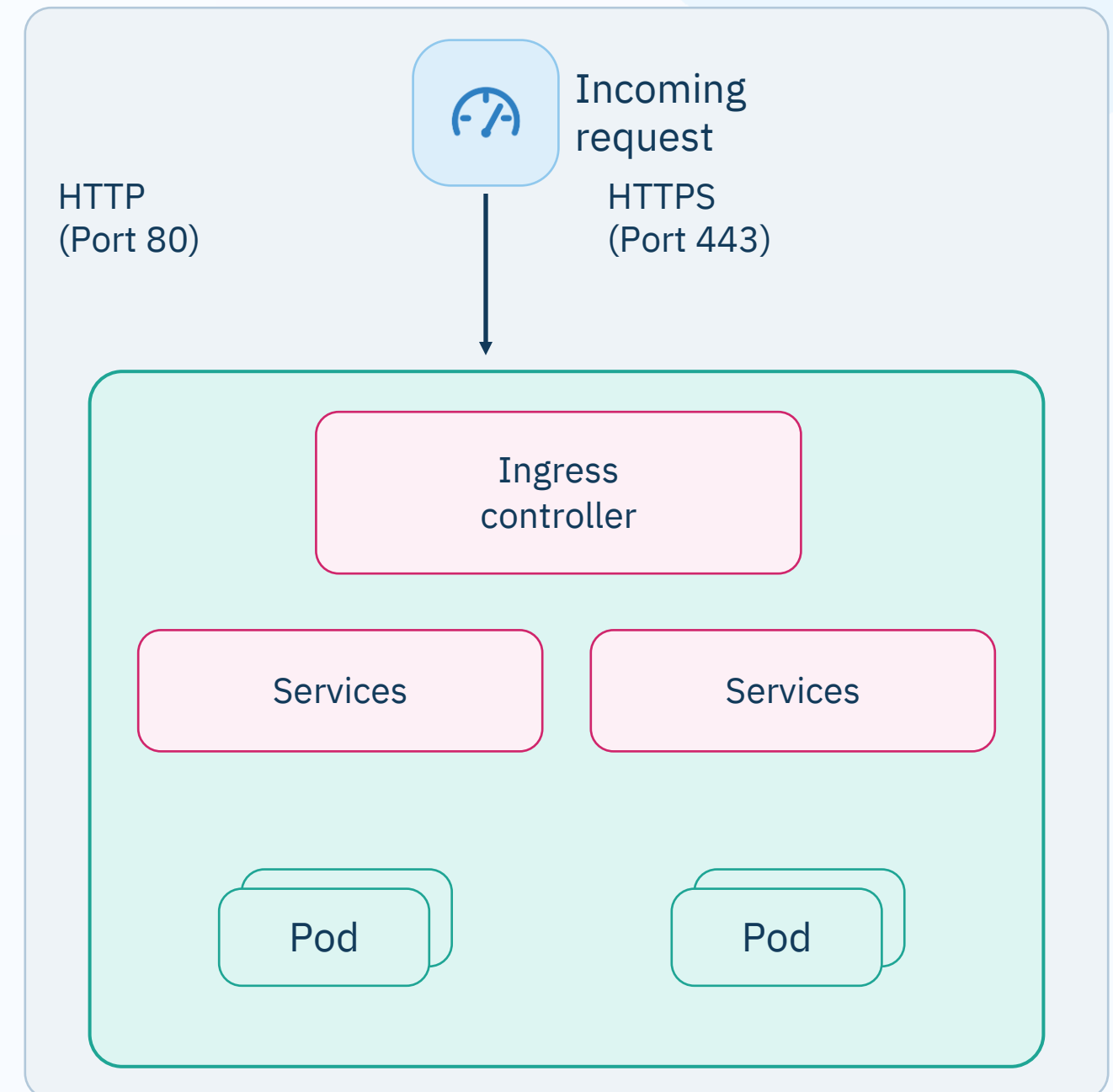
- ◆ Maps to a DNS name and not to any selector
- ◆ Requires a 'spec.externalName' parameter
- ◆ Maps the Service to contents of the externalName field that returns a CNAME record and its value
- ◆ Used to create a Service that represents an external storage and enable Pods from different namespaces to talk to each other



— KUBERNETES OBJECTS

Ingress

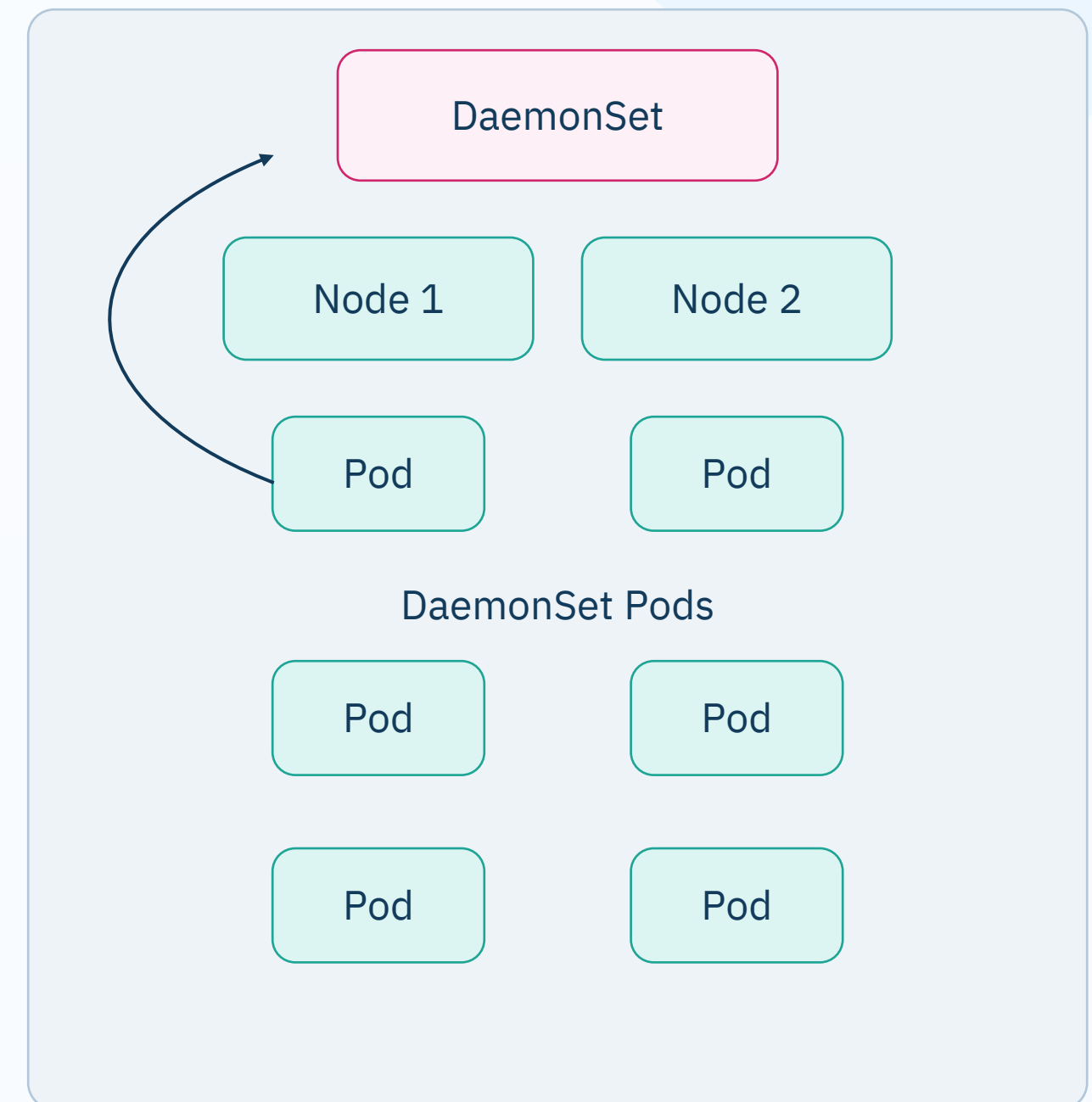
- ◆ Is an API object (combined with a controller) that provides routing rules to manage external users' access to multiple services in a Kubernetes cluster
- ◆ In production, Ingress exposes applications to the Internet via port 80 (HTTP) or port 443 (HTTPS)
- ◆ An ELB is expensive and is managed outside the cluster while the cluster monitors Ingress



DaemonSet

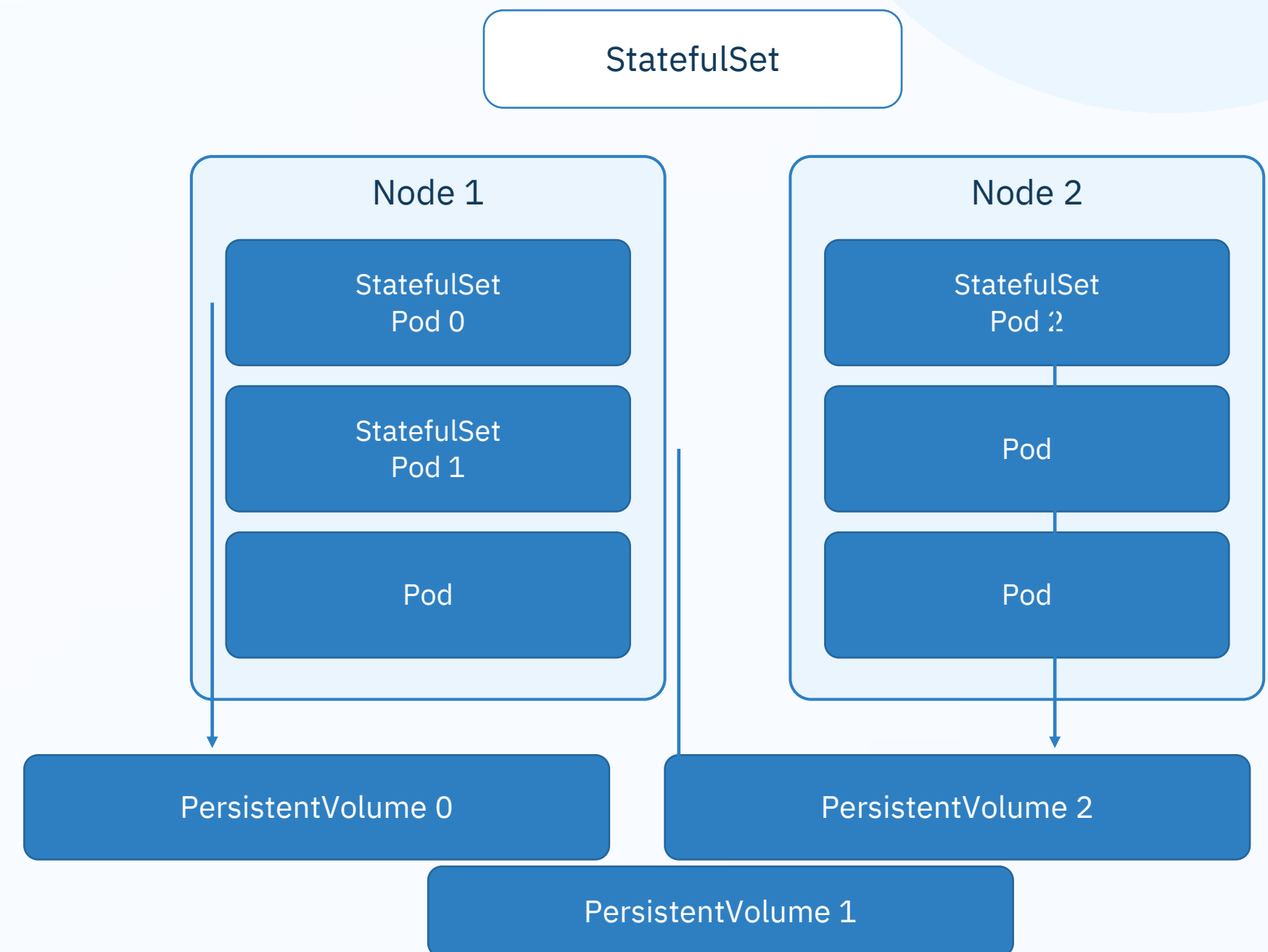


- ◆ Is an object that makes sure that Nodes run a copy of a Pod
- ◆ As Nodes are added to a cluster, Pods are added to the Nodes
- ◆ Pods are garbage collected when removed from a cluster
- ◆ If you delete a DaemonSet, all Pods are removed
- ◆ Ideally used for storage, logs, and monitoring Nodes



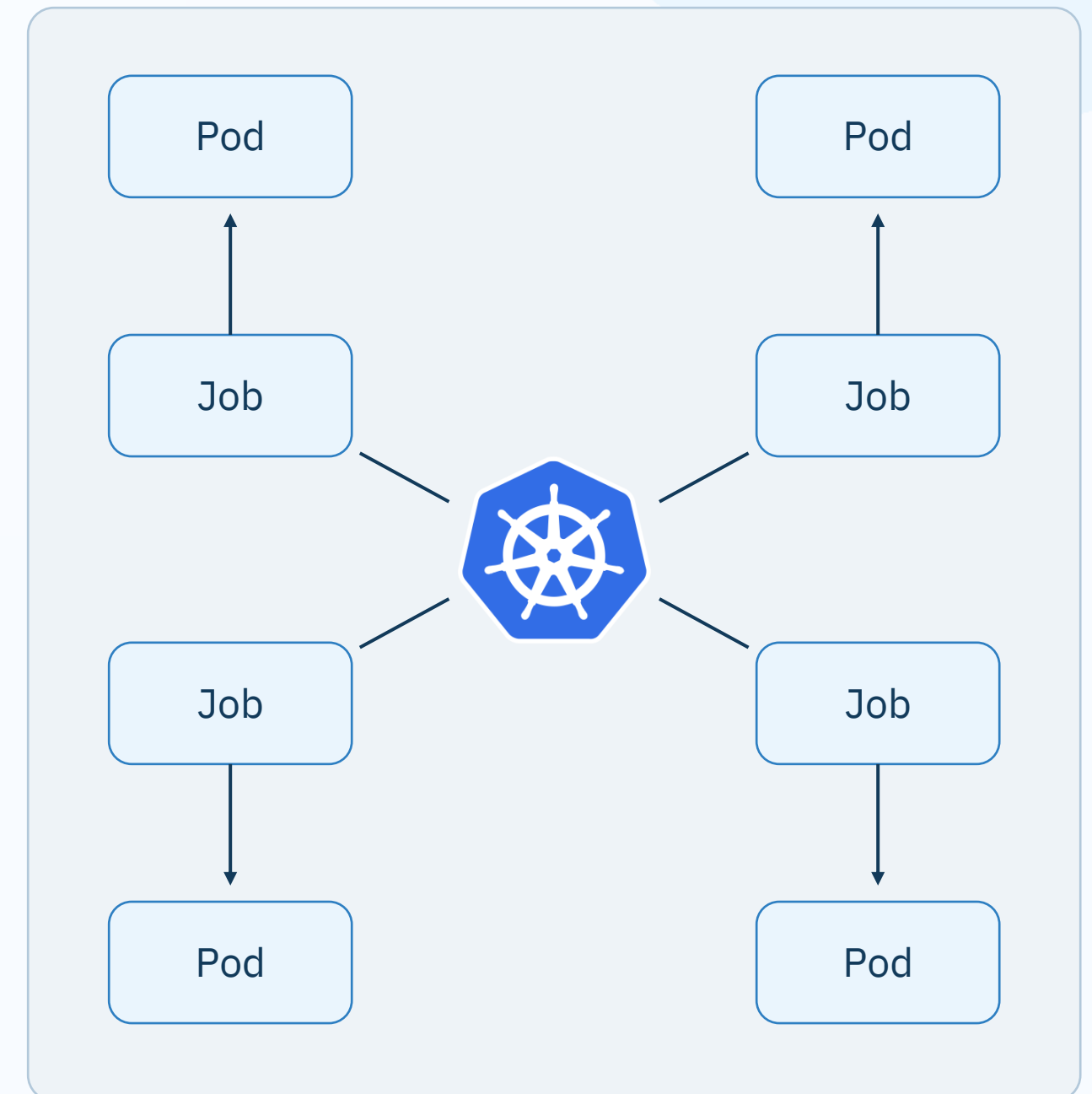
StatefulSet

- ◆ Is an object that manages stateful applications
- ◆ Manages deployment and scaling of Pods
- ◆ Provides guarantees about the ordering and uniqueness of Pods
- ◆ Maintains a sticky identity for each of your Pods
- ◆ Provides persistent storage volumes for your workloads



Job

- ◆ Is an object that creates Pods and tracks its completion process
- ◆ Jobs are retried until completed
- ◆ Deleting a Job will remove the created Pods
- ◆ Suspending a Job will delete its active Pods until the Job resumes
- ◆ A job can run several Pods in parallel
- ◆ A *CronJob* is regularly used to create Jobs on an iterative schedule



 Extra

Continuous improvement and social coding

— CONTINUOUS IMPROVEMENT AND SOCIAL CODING

Pursue continuous improvement

- ◆ Continuous Delivery enables continuous improvement
- ◆ Know where you are and when things are broken



Everyone is responsible

If a build breaks:

- ◆ Don't ask, "Whose fault is it?"
- ◆ Do ask, "Why did this fail?" and "What can we do to stop this from occurring again?"
- ◆ How did the system fail the people?

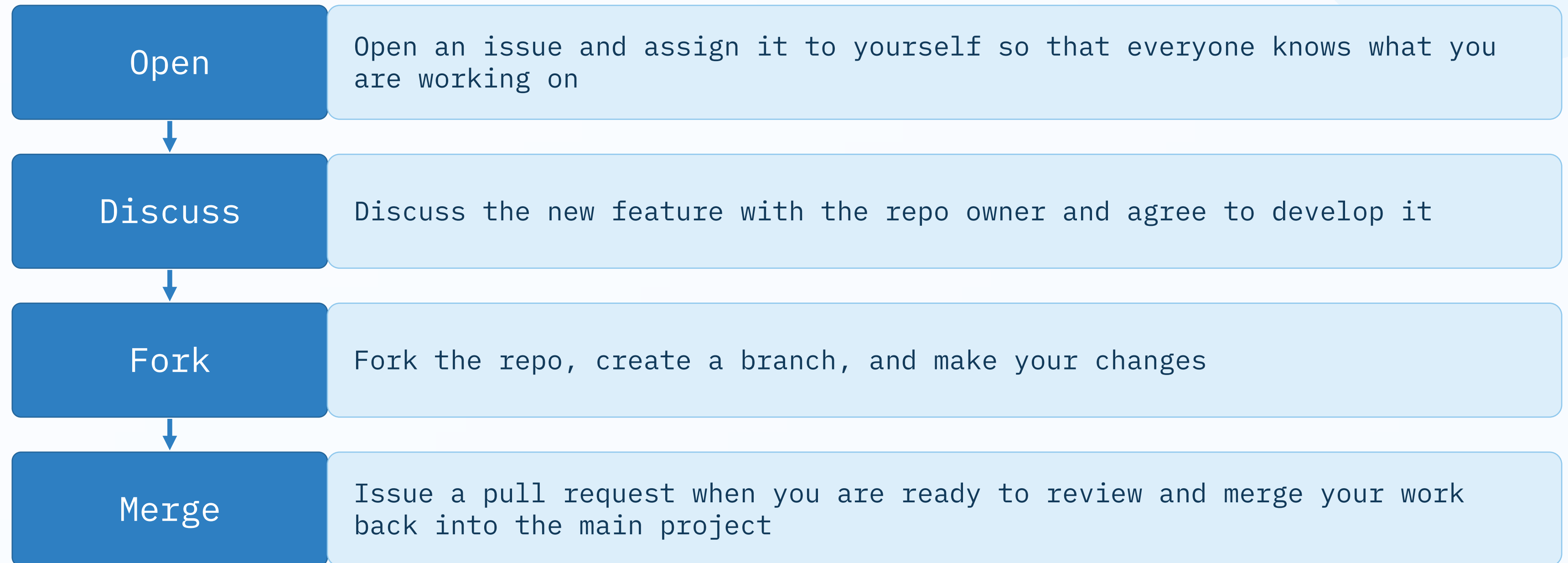
What is social coding?

- ◆ Open source for inner source
 - Open source communities already did this
 - Enterprises are now doing this too
 - Developers used to work in private repos
 - Code could not be reused
- ◆ All repositories are public
- ◆ Everyone is encouraged to contribute
- ◆ Anarchy is controlled via pull requests





Social coding steps in practice



Social coding principles

- ◆ Repo owners are in full control
 - Can ask for changes
 - Can ask for more test cases to be written
- ◆ Developers are considered as equal team members
- ◆ Developers can leverage another team's code
- ◆ Company saves money as code is reused

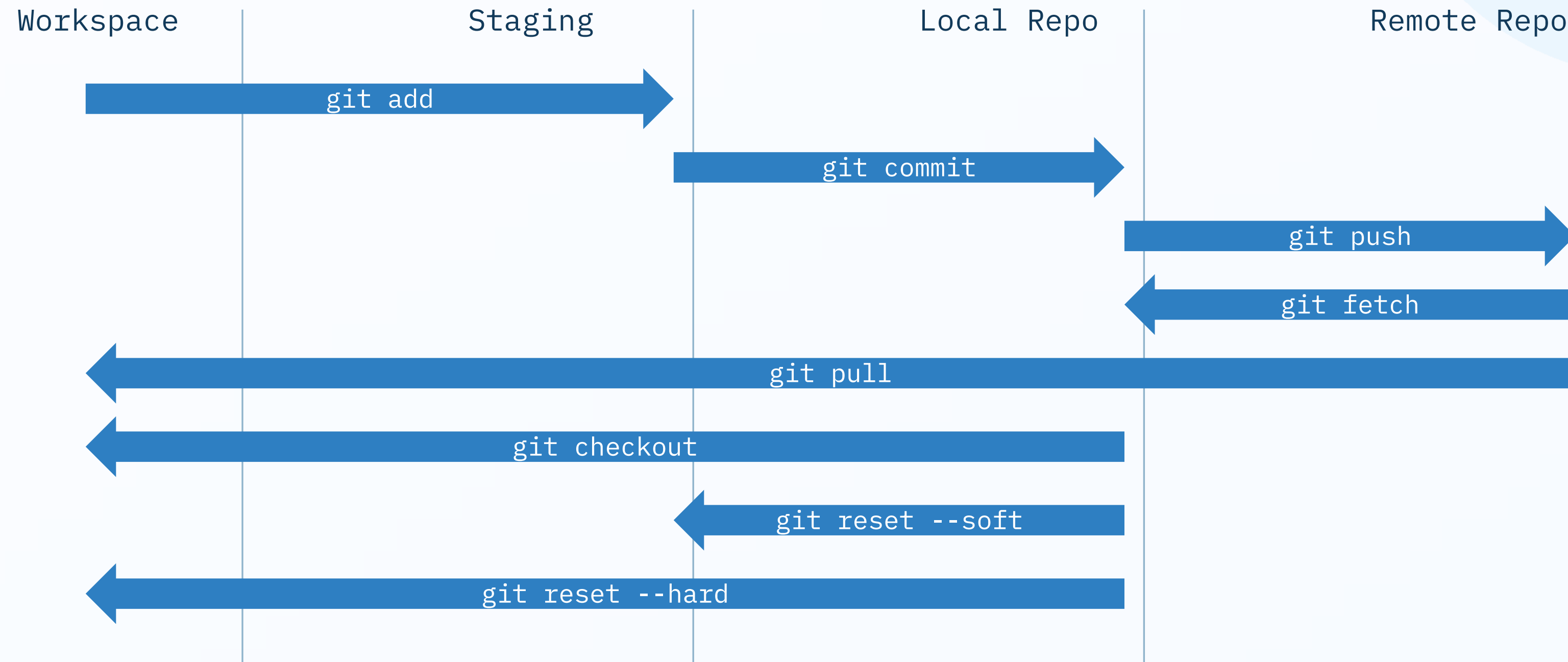
What is Git?

- ◆ A distributed source code management (SCM)
- ◆ Code can be stored locally or kept in a remote repository
- ◆ Every developer has a full copy
- ◆ Works locally without any server
- ◆ Works remotely with GitHub, GitLab, and BitBucket



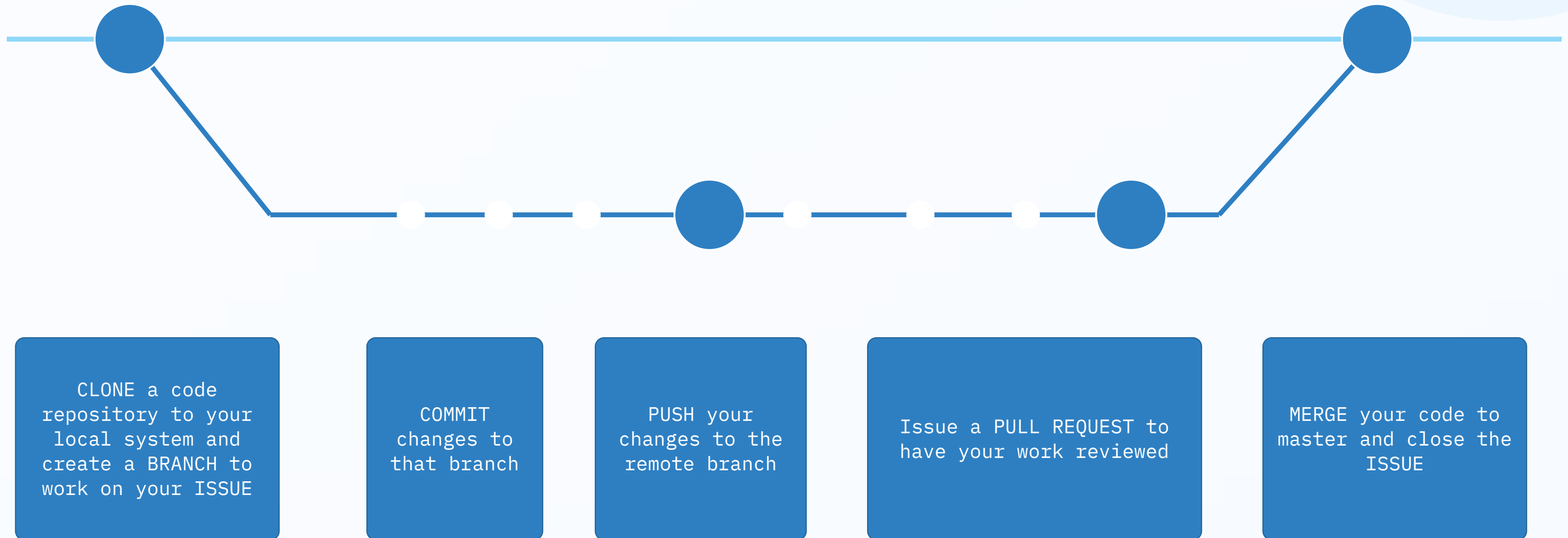


Git workflow





Git Feature Branch workflow





Example: Create feature branch

```
git checkout main
git pull
git checkout -b feature-1
... write some code here ...
git add .
git commit -m 'initial working version of my new feature'
git push -u origin feature-1
```

Always start with the latest code from master

Always create a new branch to work on

Set up a remote branch and track it



Before a pull request

Before making a pull request, get latest changes from main branch

```
git checkout main
git pull
git checkout my-new-feature
git merge main
... fix any merge conflicts here ...
git add .
git commit -m 'merged updates from main'
git push
```

Pull the latest changes from main

Merge any changes from main with your branch

Finally, push your changes to the remote branch



After a pull request

After a pull request is merged, get latest changes from main branch

```
git checkout main
git pull
git branch -d my-new-feature
git checkout -b my-new-feature-2
```

Pull the latest changes from main

Delete the old branch

Create a new branch for the new feature

 Extra

Jenkins, CircleCI and Travis CI



Jenkins overview

- ◆ Oldest of the three tools, implements both CI and CD
- ◆ Has a large ecosystem of plugins
- ◆ Uses the 'Groovy' language in a Jenkinsfile
- ◆ Enables developers to treat their CI/CD pipelines as code

```
pipeline {  
  agent { dockerfile { filename 'Dockerfile.build' } }  
  stages {  
    stage('Checkout') {  
      steps { checkout scm }  
    }  
    stage('Unit Testing') { // Perform unit testing  
      steps {  
        script { sh "python -m unittest discover -s tests/unit" }  
      }  
    }  
  }  
}
```

How Jenkins works



- ◆ Set up your project on the Jenkins website
- ◆ Create a Jenkinsfile in the root of your project folder
- ◆ Build runs in an isolated environment (VM or Docker)
- ◆ Depending on how you configure Jenkins, you can:
 - Execute a build when you push to master
 - Execute a build whenever you issue a pull request
 - Execute a build on many other events

This is a drawback. other tools allow you to specify a pipeline simply by adding a file to your repository, making them very repeatable. Requiring you to use a website for some configuration means manual steps that are not automatable and may not be reproducible.



Jenkinsfile

```
pipeline {
  agent { dockerfile { filename 'Dockerfile.build' } }
  stages {
    stage('Checkout') { // Checkout (git clone ...) the projects repository
      steps {
        checkout scm
      }
    }
    stage('Setup') { // Install any dependencies you need to perform testing
      steps {
        script {
          sh "pip install -r requirements.txt"
        }
      }
    }
    stage('Linting') { // Run pylint against your code
      steps {
        script {
          sh "pylint **/*.py"
        }
      }
    }
    stage('Unit Testing') { // Perform unit testing
      steps {
        script {
          sh "python -m unittest discover -s tests/unit"
        }
      }
    }
  }
}
```

Checks out code

Installs dependencies

Performs linting

Runs unit tests

CircleCI

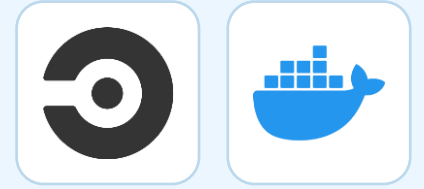


- ◆ Provided as a service, implements both CI and CD
- ◆ It allows you to treat your CI/CD pipeline as code (common theme)
- ◆ Uses circle.yml file to control workflow

```
jobs:
  build:
    executor: python/default
    steps:
      - checkout
      - python/install-packages:
          pkg-manager: pip
      - run:
          name: Run tests
          command: nosetests -v --with-spec --spec-color
```

How CircleCI works

- ◆ Set up your project on the CircleCI website
- ◆ Create a `.circleci/config.yml` file in the root of your project
- ◆ Build runs in an isolated environment (VM or Docker)
- ◆ Depending on how you configure CircleCI:
 - Whenever you push to master a build is executed
 - Whenever you issue a pull request a build is executed
- ◆ You may see a pattern emerging...





circle.yml

```
version: 2.1
orbs:
  python: circleci/python@2.0.3

jobs:
  build: # this can be any name you choose
    docker:
      - image: cimg/python:3.9
        environment:
          DATABASE_URL: postgresql://tester:passw0rd@localhost/testdb
      - image: circleci/postgres:13.2
        environment:
          POSTGRES_USER: tester
          POSTGRES_PASSWORD: passw0rd
          POSTGRES_DB: testdb
    steps:
      - checkout
      - python/install-packages:
          pkg-manager: pip
      - run:
          name: Run tests
          command: nosetests -v --with-spec --spec-color
```

Builds upon Python image

Builds PostgreSQL

Defines steps to run

Travis CI



- ◆ Provided as a service, implements both CI and CD
- ◆ Allows you to treat your CI/CD pipeline as code
- ◆ Uses a .travis.yml file to define workflow

```
language: python
python:
  - "3.9"

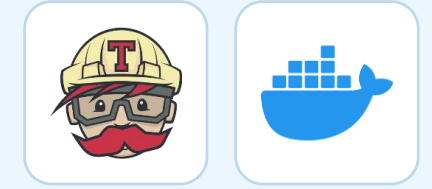
install: "pip3 install -r requirements.txt"

script: nosetests -v --with-spec --spec-color

after_success:
  - codecov
```

— JENKINS, CIRCLECI AND TRAVIS CI

How Travis CI works



- ◆ Set up your project on the Travis CI website
- ◆ Create a `.travis.yml` file in the root of your project
- ◆ Build runs in an isolated environment (VM or Docker)
- ◆ Depending on how you configure Travis CI:
 - Whenever you push to master a build is executed
 - Whenever you issue a pull request a build is executed
- ◆ They all service the same goal: enabling CI

Like the other tools, you can specify what actions trigger a build, but unlike the other tools, this is not done in the `.travis.yml` file. It can only be done manually from the Travis CI admin UI, just like Jenkins, which is a drawback.

.travis.yml



```
language: python
python:
  - "3.9"

services:
  - postgresql

env:
  - DATABASE_URI=postgres://postgres:@localhost:5432/testdb

before_script:
  - psql -c 'create database testdb;' -U postgres

# command to install dependencies
install: "pip3 install -r requirements.txt"

# command to run tests
script: nosetests

after_success:
  - codecov
```

Sets language

Requests PostgreSQL

Installs dependencies

Runs tests

Runs code coverage